



SE4040 – EAD

Group ID - 06

Assignment - Smart Solar Microgrid Trading System

Y4S2

Submitted by:

Registration number	Name
IT22546920	Lakshan H.G.J.S.
IT22220424	Kaushalya P.L.P.D.
IT22070630	Perera K.D.N.

Table of Contents

Table of Contents.....	2
1. Introduction.....	3
2. System Architecture.....	4
2.1. Technology Stack.....	4
3. Use Case Diagram.....	5
4. Data Flow Diagram (DFD)	6
5. Database Design.....	8
6. Screenshots of All User Interfaces.....	9
6.1. Web Application (React)	9
6.2. Android Application (Kotlin)	14
7. Source Code.....	19
8. References.....	202
9. Individual Contribution.....	203
10. Challenges.....	204
10.1. Verifying that Android Authentication Is Not Performed Locally	204
10.2. Enforcing the 12-Hour and 7-Day Rules Centrally	204
10.3. Coordinating Role-Based Access Across Two Independent Clients.....	204
10.4. Google Maps API key configuration	204
10.5. git workflow coordination across three members	205
11. Git Repository link.....	205

1. Introduction

SolariX is a client-server system built for the SE4040 Enterprise Application Development module. It supports the trading of solar-generated energy between property owners with rooftop solar arrays (Prosumers) and a network of microgrid charging/drop-off hubs managed by a utility operator.

The system has three client-facing roles:

- Backoffice — administrative staff who manage system users, prosumer accounts and microgrid node (hub) configuration. Backoffice access is web-only.
- Grid Operator — field staff who monitor bookings and battery slot availability via the web portal, and who physically verify and finalize energy transfers using the Android app's QR scanner.
- Solar Prosumer — property owners who register with their National Identity Card (NIC) as their primary identifier, and who reserve, modify and cancel energy drop-off/charging slots through the Android app.

Architecturally, the system follows a strict three-tier, FAT-service model: the Web application (React) and the Android application (Kotlin, native) act purely as user-interface layers. Neither client holds authoritative business logic — every rule (the 7-day reservation window, the 12-hour modification/cancellation notice period, node-deactivation guards, QR verification) is enforced centrally by the C# Web API, which is the only component with a driver to the MongoDB data store.

This report documents the system's architecture, database design, API surface, role-based access control, deployment process, and each member's individual contribution to the implementation.

2. System Architecture

The diagram below shows the high-level architecture of SolariX. Both client applications communicate exclusively over RESTful HTTP with the central Web API; there is no direct database access from either client.

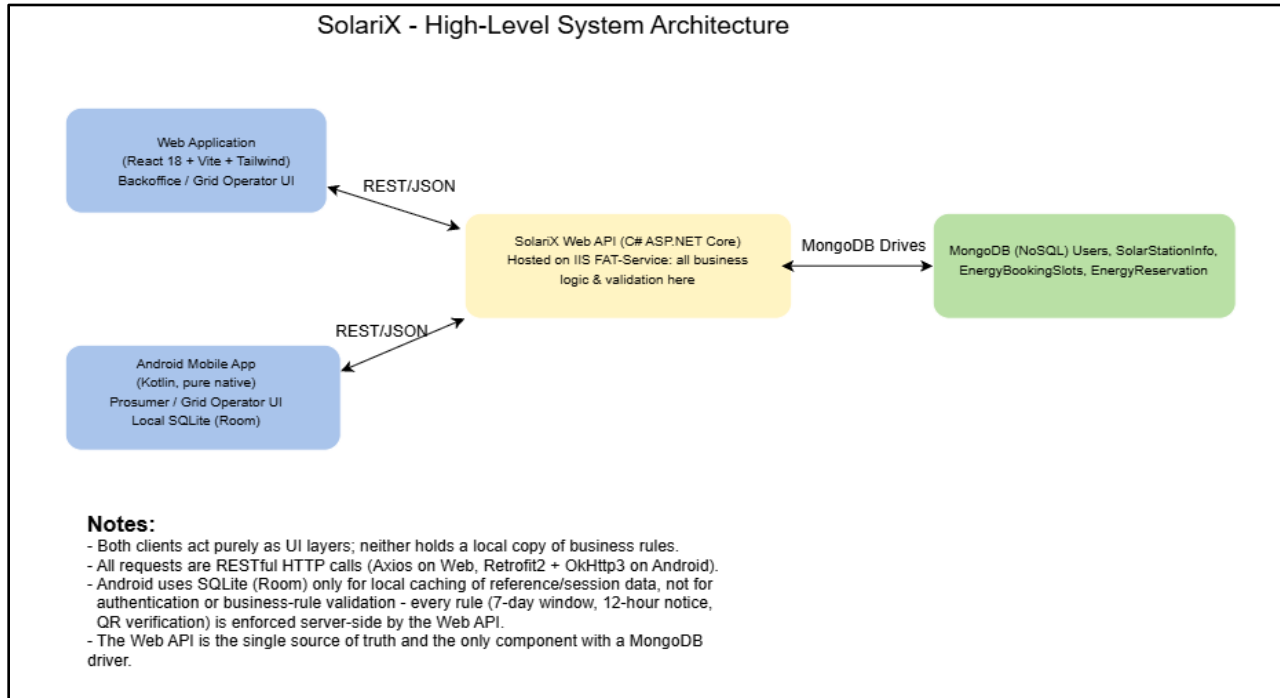


Figure 1 - SolariX high-level client-server architecture

2.1. Technology Stack

Layer	Technology	Notes
Web Application	React 18, Vite, Tailwind CSS, Axios	Backoffice and Grid Operator UI only; calls the Web API for every operation
Mobile Application	Kotlin (pure native Android), Room (SQLite), Retrofit2 + OkHttp3, ZXing, Google Maps SDK	Prosumer and Grid Operator UI; no cross-platform framework used
Web Service / API	C# ASP.NET Core, hosted on IIS	FAT-service pattern — all business logic and validation resides here
Database	MongoDB (NoSQL)	Accessed only by the Web API via the official MongoDB .NET driver

3. Use Case Diagram

The use case diagram below summarizes the functions available to each of the three roles.

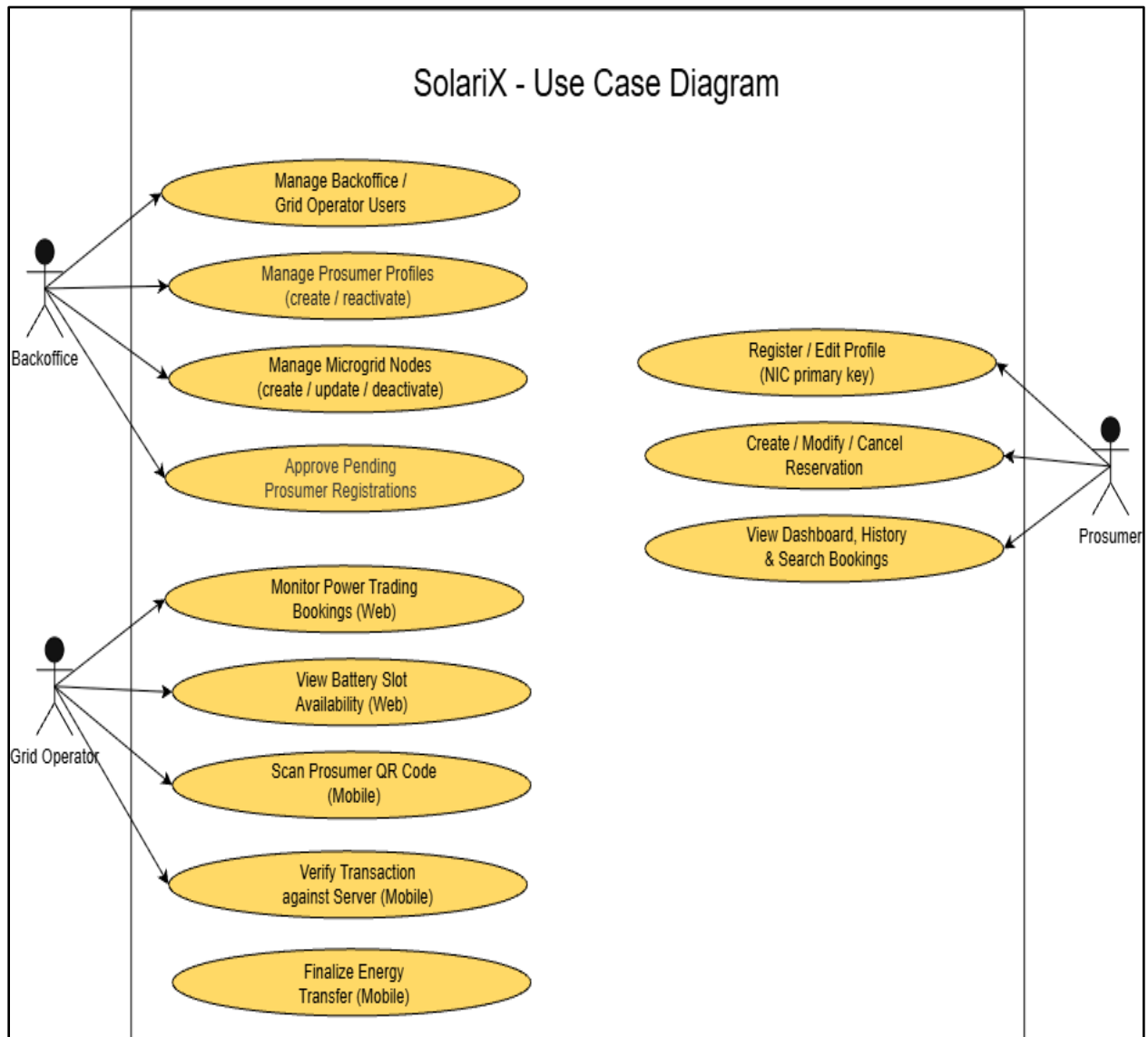


Figure 2 - SolariX use case diagram by role

4. Data Flow Diagram (DFD)

The Level 0 (context) diagram below treats SolariX as a single process exchanging data with its three external actors and the MongoDB data store.

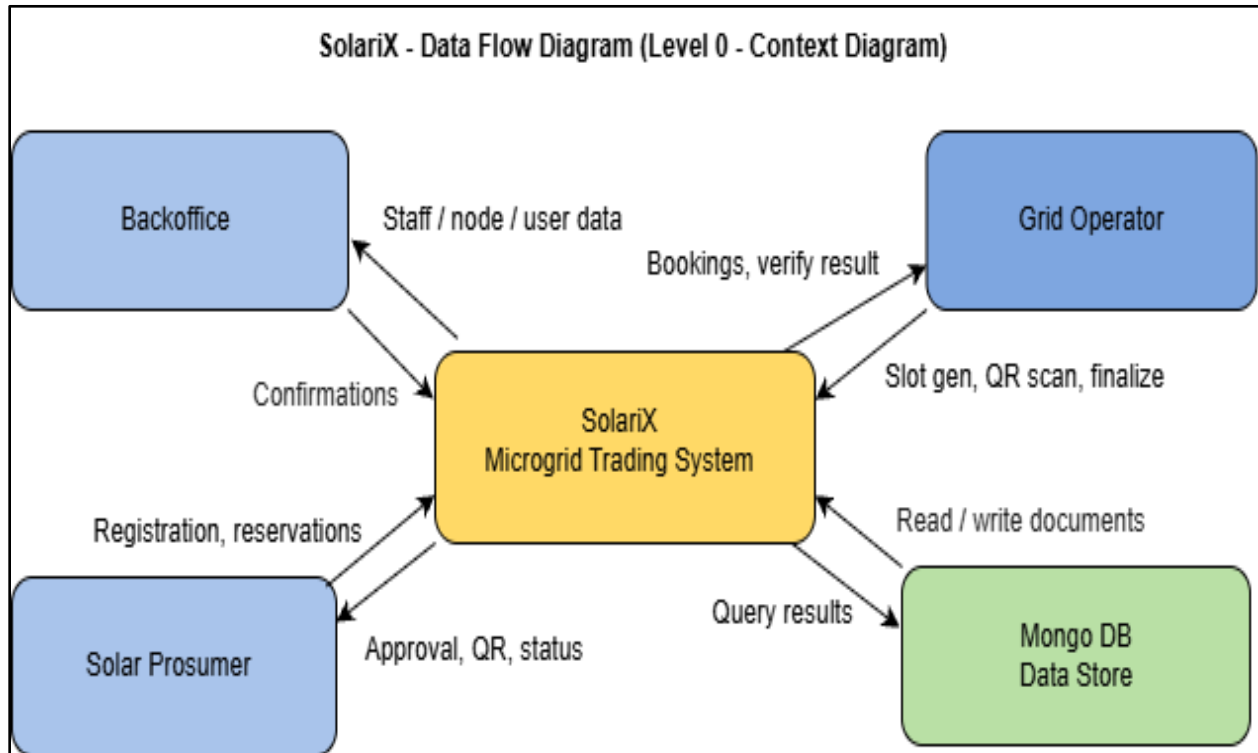


Figure 3 - DFD Level 0 (context diagram)

The Level 1 diagram below decomposes Process 0 into its four constituent processes, matching the system's four API controllers, and shows exactly which MongoDB collection each process reads from and writes to.

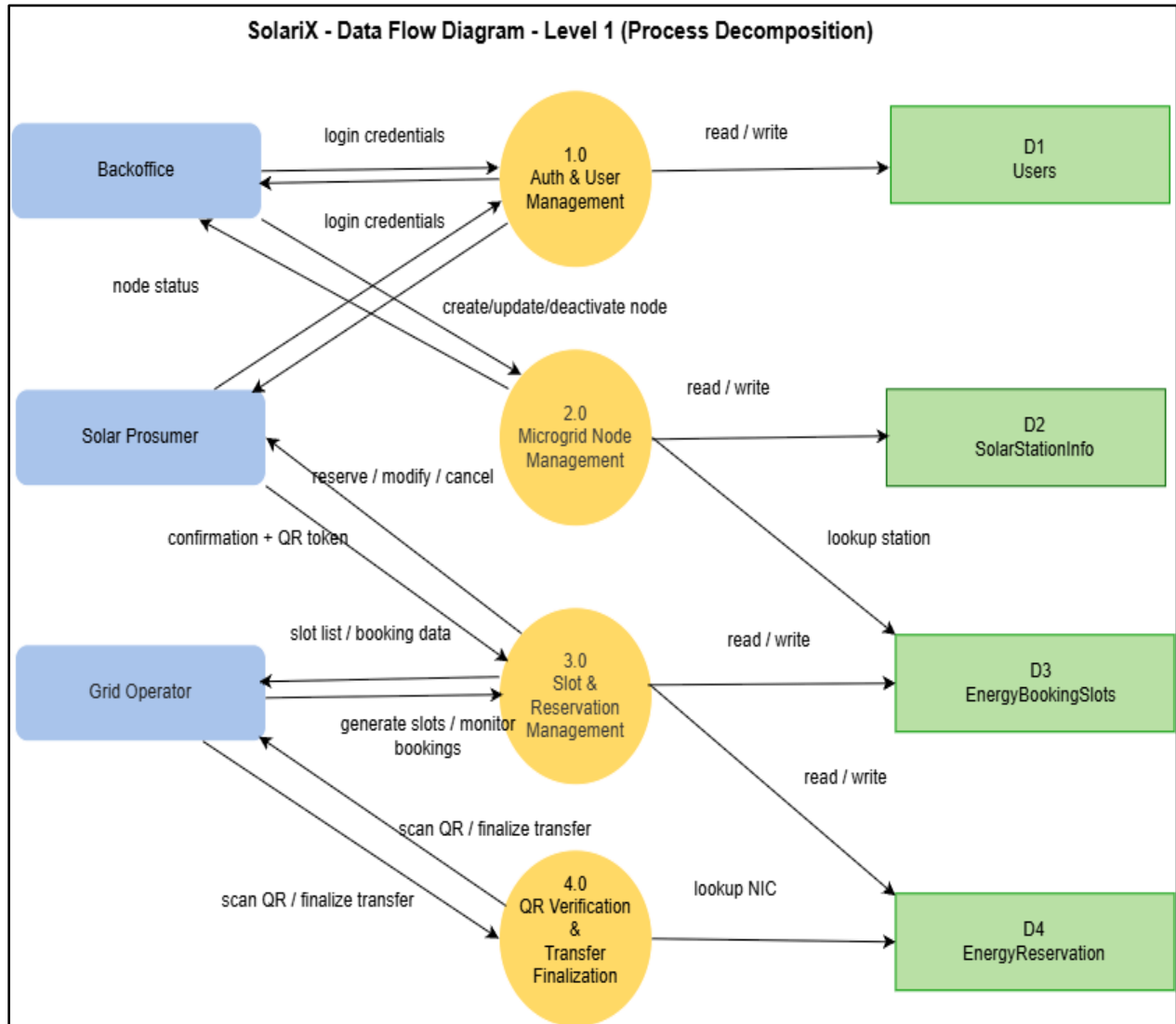


Figure 4 - DFD Level 1 (process decomposition)

5. Database Design

SolariX uses a single MongoDB database (SolarixDb) containing four collections. Relationships between collections are maintained at the application level (referenced by ID/NIC) rather than enforced by the database, consistent with the NoSQL/document-store model.

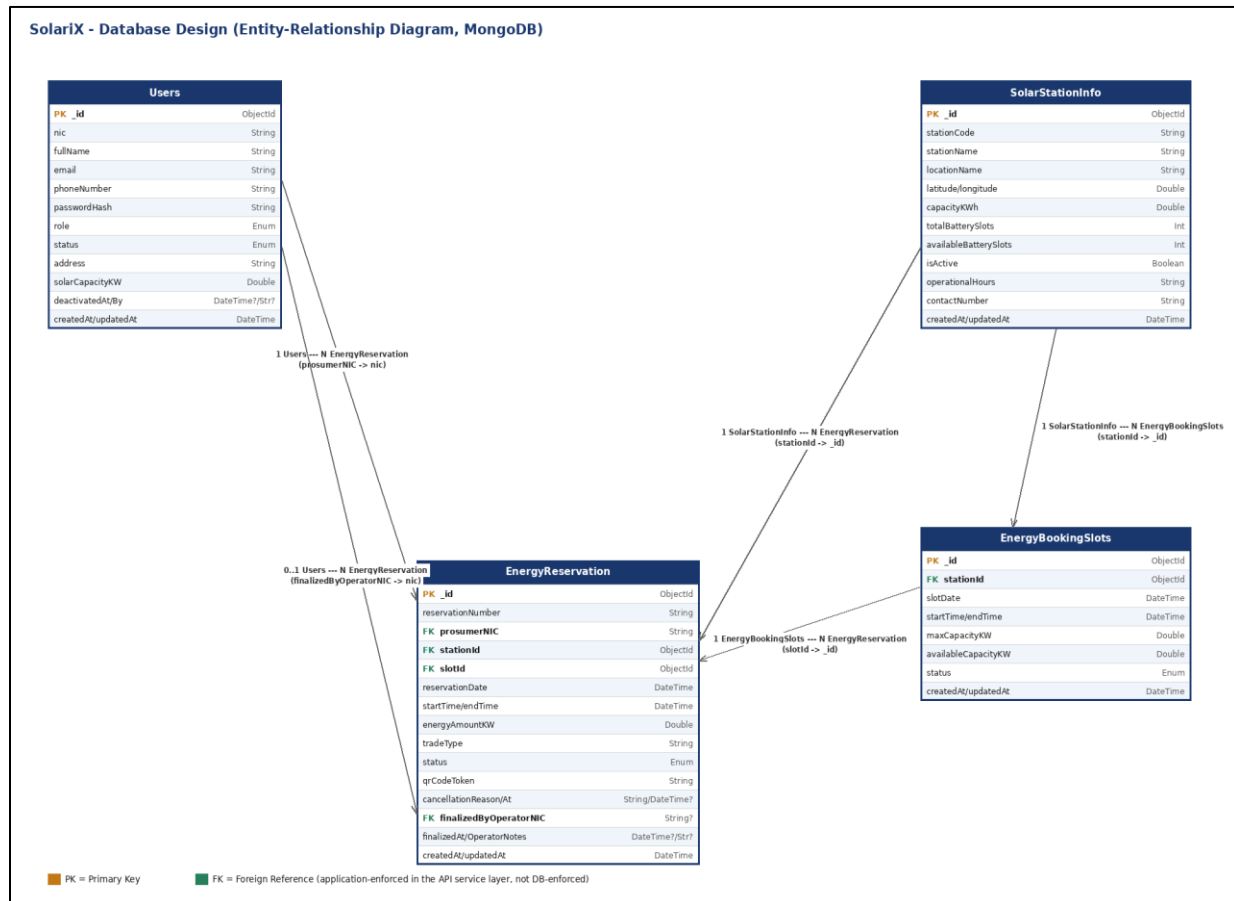


Figure 5 - SolariX database Entity-Relationship Diagram (MongoDB)

SolarixDb is a MongoDB (NoSQL) database. The PK/FK references shown above are maintained at the application level by the Web API's service layer (e.g. the node deactivation guard and QR/reservation lookups) rather than enforced by the database itself, which is standard for a document-store model.

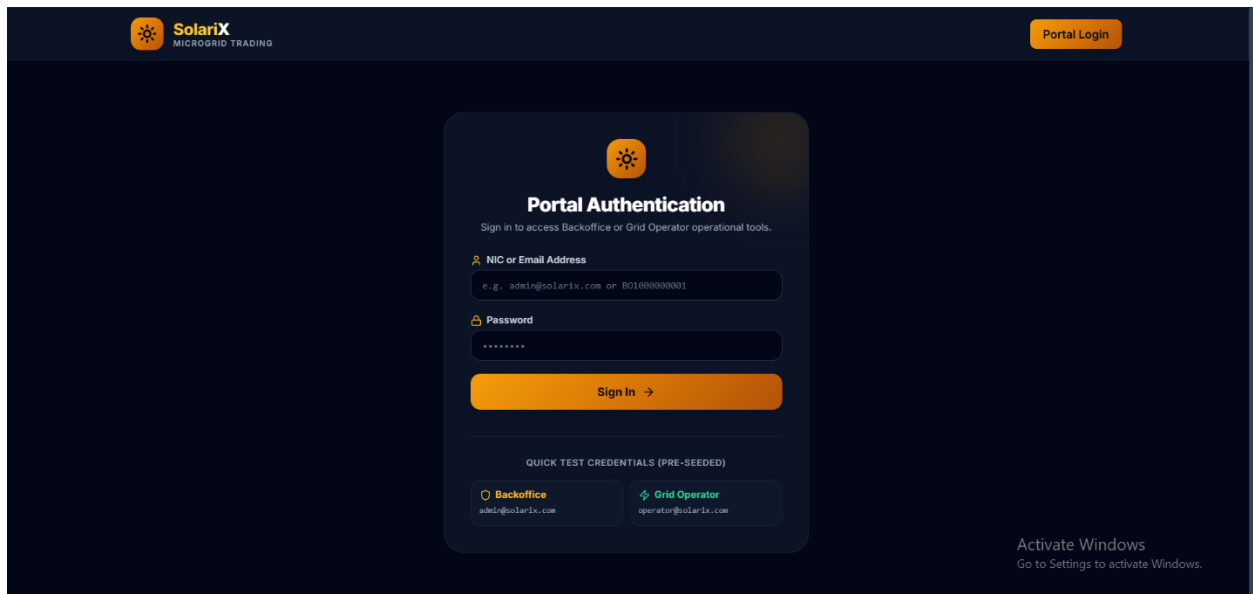
6. Screenshots of All User Interfaces

6.1. Web Application (React)

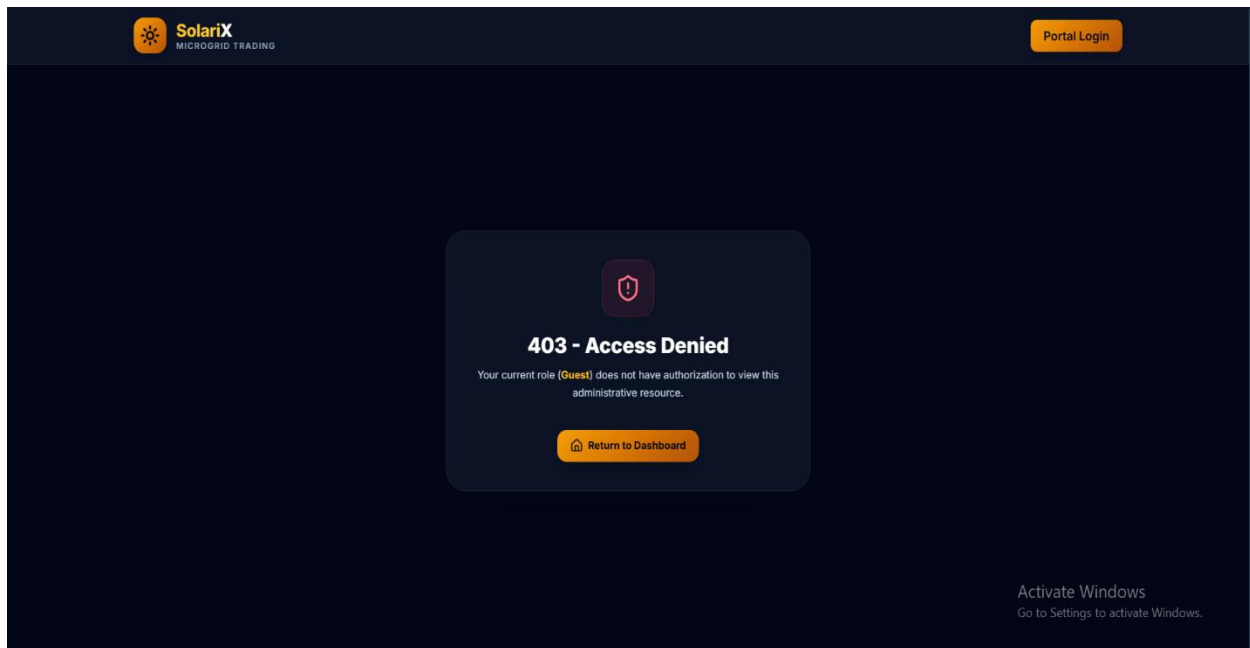
Home Page UI



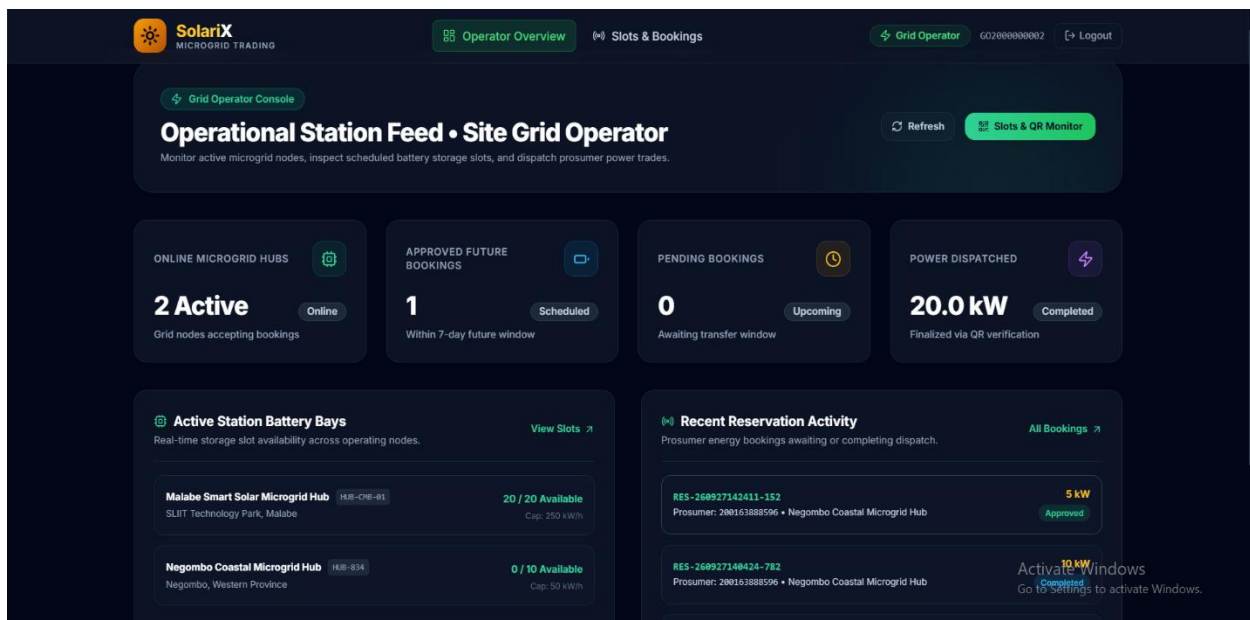
Login UI



Unauthorized UI



Operator Dashboard UI



Slot Management UI

SolarIX MICROGRID TRADING

Operator Overview Slots & Bookings Grid Operator G02000000002 Logout

Microgrid Slot & Reservation Management

Grid Operator console for viewing battery slot readiness, auditing prosumer trading requests, and finalizing on-site QR transactions.

Battery Storage Slots & Schedule

Operational hours and slot capacities read from central FAT service.

Malabe Smart Solar Microgrid Hub (HUB-CMB-01) 09/29/2026 Batch Daily Slots

Time Slot	Available	Available Cap.
01:30 PM - 02:30 PM	Available	25 / 25 kW
02:30 PM - 03:30 PM	Available	25 / 25 kW
03:30 PM - 04:30 PM	Available	25 / 25 kW
04:30 PM - 05:30 PM	Available	25 / 25 kW
05:30 PM - 06:30 PM	Available	25 / 25 kW
06:30 PM - 07:30 PM	Available	25 / 25 kW
07:30 PM - 08:30 PM	Available	25 / 25 kW
08:30 PM - 09:30 PM	Available	25 / 25 kW
09:30 PM - 10:30 PM	Available	25 / 25 kW
10:30 PM - 11:30 PM	Available	25 / 25 kW

Power Trading Booking Monitor

Search reservation or NIC... ALL Approved Completed Cancelled

Activate Windows
Go to Settings to activate Windows.

Power Trading Booking UI

SolarIX MICROGRID TRADING

Operator Overview Slots & Bookings Grid Operator G02000000002 Logout

Power Trading Booking Monitor

Real-time feed of solar reservations across microgrid hubs with QR validation dispatch.

Verify & Complete QR

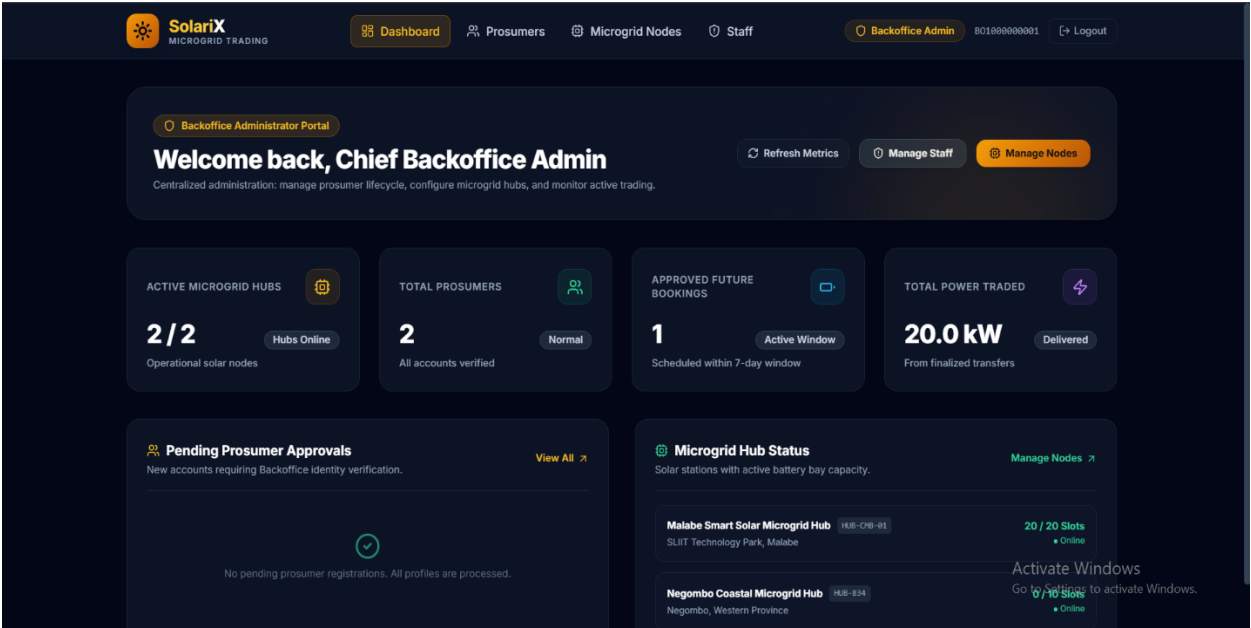
RESERVATION #	PROSUMER NIC	HUB / STATION	SCHEDULED SLOT	TRADE SPEC	STATUS	OPERATOR ACTION
RES-260927142411-152	200163888596	Negombo Coastal Microgrid Hub	9/28/2026 02:30 PM	5 kW (Charge)	Approved	Finalize
RES-260927140424-782	200163888596	Negombo Coastal Microgrid Hub	9/29/2026 01:30 PM	10 kW (DropOff)	Completed	Done (G02000000002)
RES-260927091642-157	200163888596	Negombo Coastal Microgrid Hub	9/30/2026 01:30 PM	6 kW (Charge)	Approved	Finalize
RES-260926052625-495	200163888596	Malabe Smart Solar Microgrid Hub	9/26/2026 09:30 PM	5 kW (DropOff)	Approved	Finalize
RES-260925170334-360	200163888596	Malabe Smart Solar Microgrid Hub	9/28/2026 01:30 PM	10 kW (DropOff)	Completed	Done (G02000000002)

Power Trading Booking Monitor

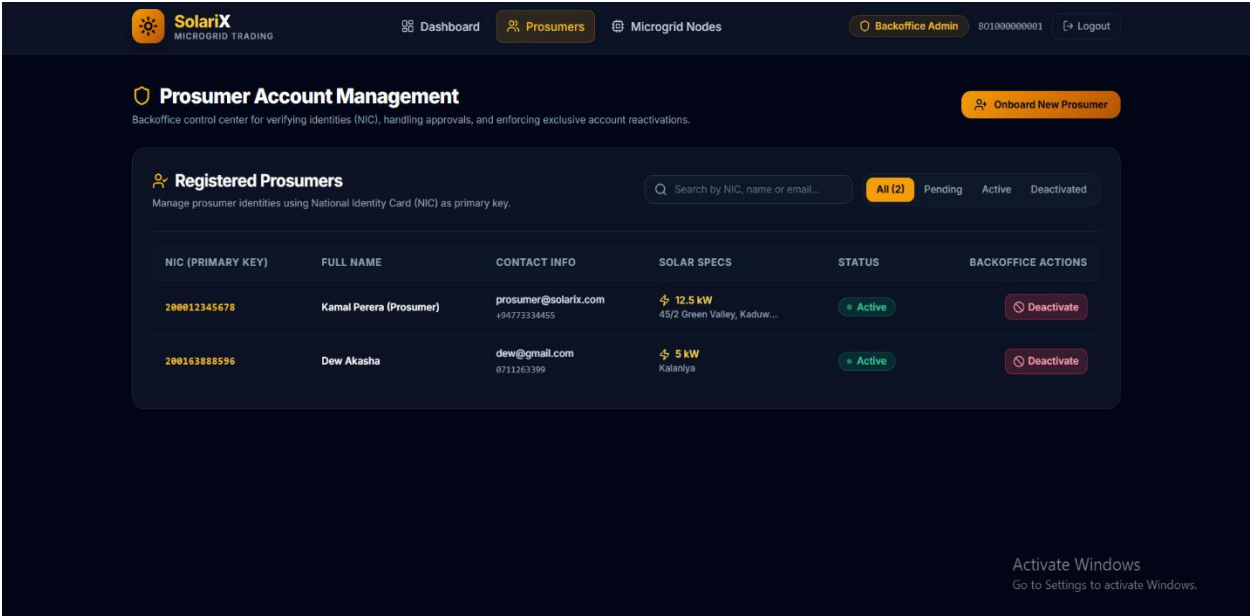
Search reservation or NIC... ALL Approved Completed Cancelled

Activate Windows
Go to Settings to activate Windows.

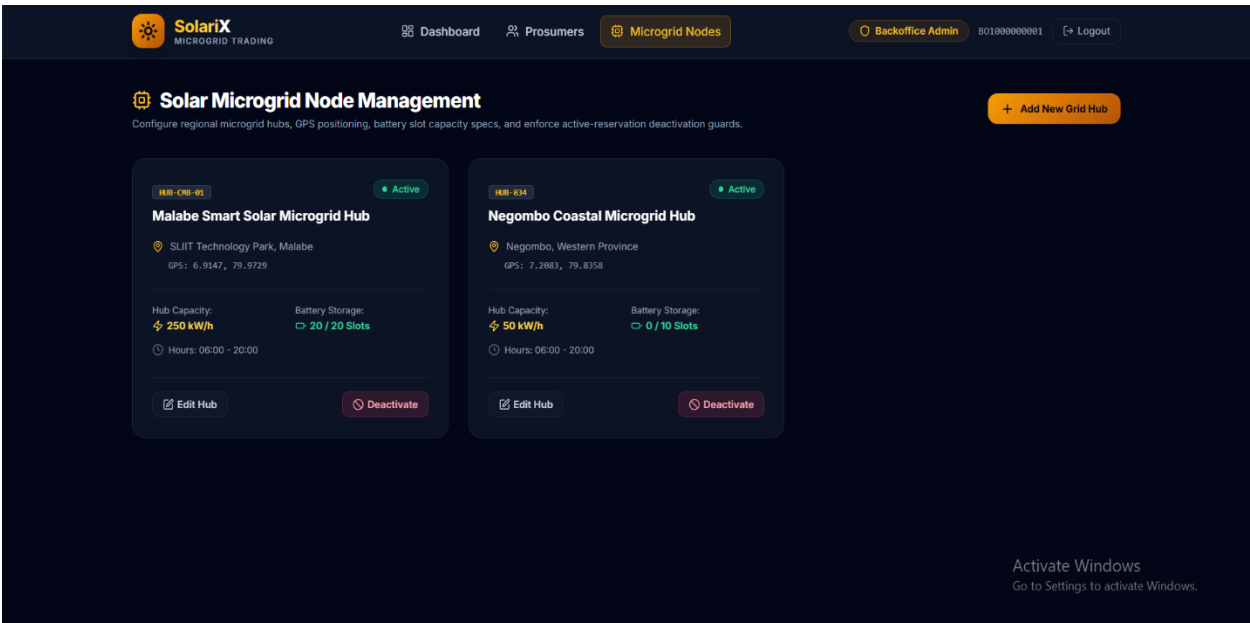
Backoffice Dashboard UI



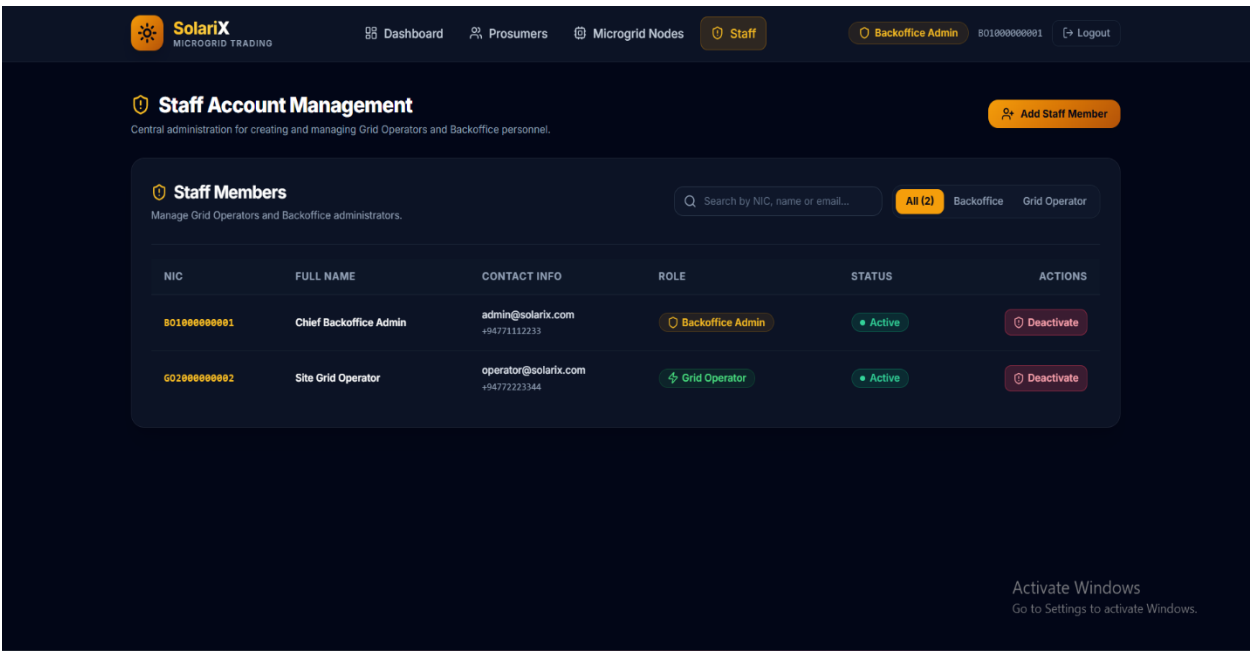
Prosumer Management UI



Node Management UI



Staff Management UI



Screen	Route	Purpose
Home	/	Public landing page
Login	/login	Authenticates Backoffice and Grid Operator users
Unauthorized	/unauthorized	Shown when a role attempts to access a restricted route
Backoffice Dashboard	/backoffice/dashboard	Administrative summary metrics
Prosumer Management	/backoffice/prosumers	Create / approve / reactivate prosumer accounts
Node Management	/backoffice/nodes	Create / update / deactivate microgrid nodes
Staff Management	/backoffice/staff	Create Backoffice / Grid Operator user accounts
Operator Dashboard	/operator/dashboard	Live booking summary for Grid Operators
Slot Management	/operator/slots	Generate daily slots; view live booking data

6.2. Android Application (Kotlin)

LoginActivity

10:22

SolariX
Smart Solar Microgrid Energy Trading Platform

Welcome to SolariX
Sign in to manage solar energy trades

NIC or Email Address

Password

Server: <http://10.0.2.2:5000/api/> (Tap to change)

Login

New Prosumer? Register Account

RegisterActivity

10:19

Prosumer Registration
Join the decentralised clean energy microgrid

National Identity Card (NIC)
200158333696

Unique national identifier (e.g. 199912345678 or 991234567V)

Full Name
Dew Nawo

Email Address
dew@gmail.com

Phone Number
0711369985

Password

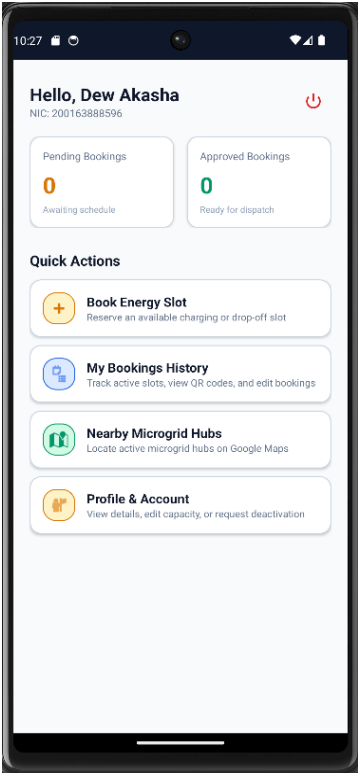
Physical Address
Kaduwela

Solar Capacity (kW)
5.0

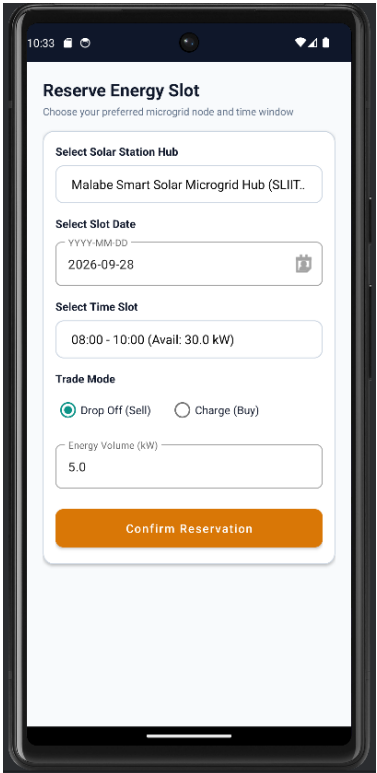
Register

Already registered? Sign In

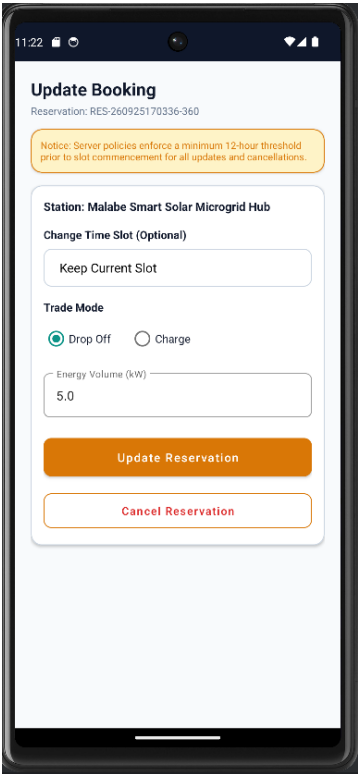
ProsumerDashboardActivity



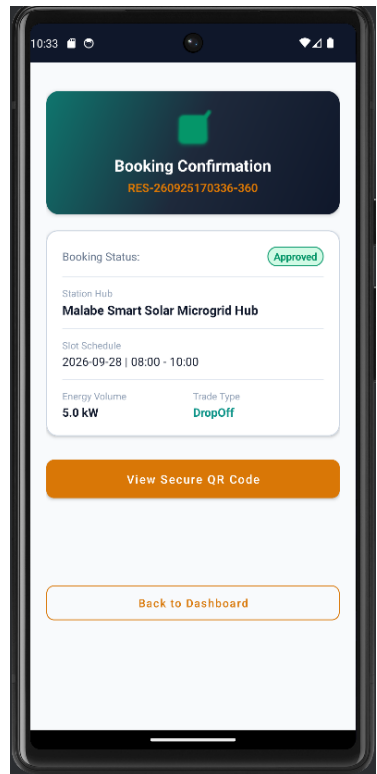
CreateBookingActivity



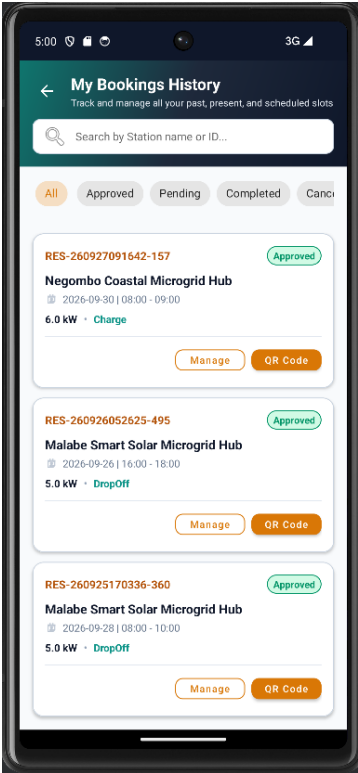
EditBookingActivity



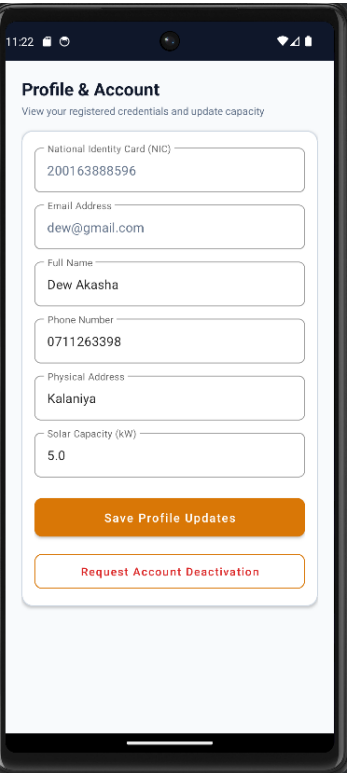
BookingSummaryActivity



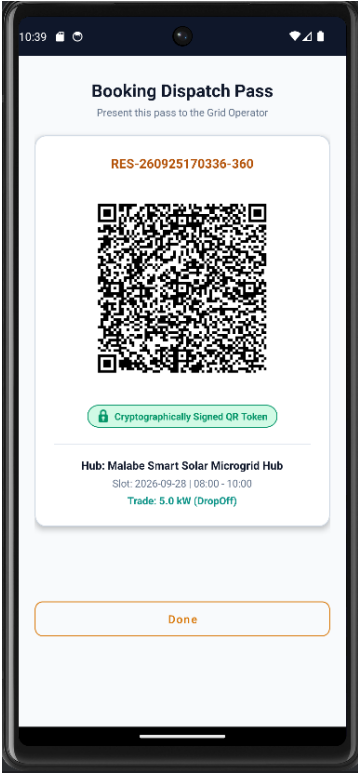
BookingHistoryActivity



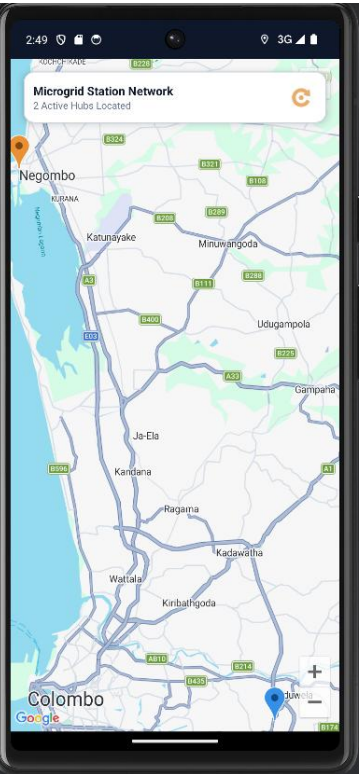
ProsumerProfileActivity



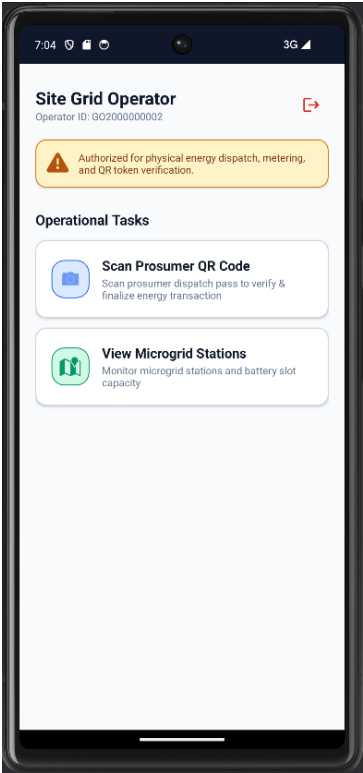
QRDisplayActivity



StationsMapActivity



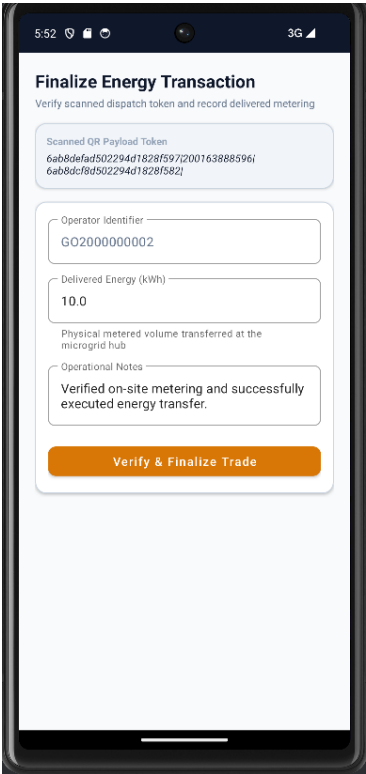
OperatorDashboardActivity



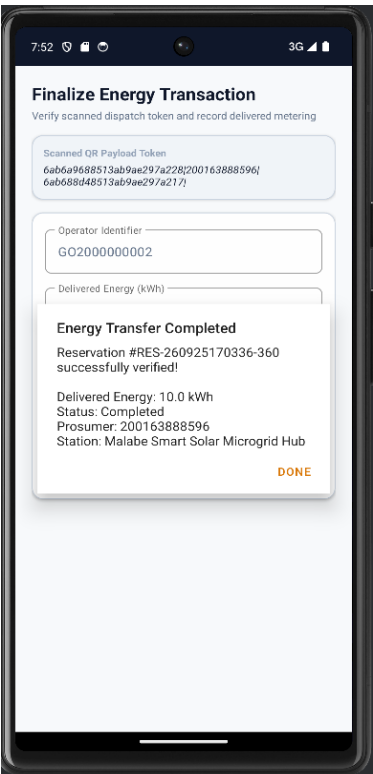
QRScannerActivity



FinalizeEnergyTransaction



JobCompletionActivity



Screen (Activity)	Reached via	Purpose
LoginActivity	App launch	Authenticates Prosumer / Grid Operator; blocks Backoffice with an explanatory dialog
RegisterActivity	Login screen link	Prosumer self-registration using NIC as primary key
ProsumerDashboardActivity	Post-login (Prosumer)	Active/pending reservation counts, quick actions
CreateBookingActivity	Dashboard FAB	Schedules a new reservation within the 7-day window
EditBookingActivity	Reservation card tap	Modifies an existing booking (12-hour rule enforced)
BookingSummaryActivity	After create/edit	Confirms the action and displays the result
BookingHistoryActivity	Dashboard menu	Full reservation history with search
ProsumerProfileActivity	Dashboard menu	Edit profile; request account deactivation
QRDisplayActivity	Approved booking	Displays the reservation's secure QR token
StationsMapActivity	Dashboard menu	Google Maps view of nearby microgrid nodes
OperatorDashboardActivity	Post-login (Operator)	Grid Operator home screen
QRScannerActivity	Operator dashboard	Camera-based QR scan of a prosumer's reservation token
JobCompletionActivity	Successful QR scan	Operator enters delivered kWh and finalizes the transfer

7. Source Code

This section reproduces the core business-logic and integration files that implement SolariX's FAT-service architecture and its two hard business rules: the 7-day reservation window and the 12-hour modification/cancellation notice period. Full source is included in the submitted project archive; every .cs file carries a header comment block and every method carries an inline comment describing its purpose, per the assignment's marking requirement.

BookingService.cs (business rule enforcement — excerpt)

```
/// <summary>
/// Central booking rule engine. All reservation-related business rules
/// (7-day scheduling window, 12-hour modification/cancellation notice)
/// are enforced here, and only here, consistent with the FAT-service
/// architecture required by the assignment specification.
/// </summary>
public class BookingService
{
    // Rejects a reservation request scheduled more than 7 days ahead.
    private void ValidateSchedulingWindow(DateTime reservationDate)
    {
        if (reservationDate > DateTime.UtcNow.AddDays(7))
            throw new InvalidOperationException(
                "Reservations must be scheduled within 7 days.");
    }

    // Rejects an update/cancellation made with less than 12 hours' notice.
    private void ValidateNoticePeriod(DateTime slotStartTime)
    {
        if ((slotStartTime - DateTime.UtcNow).TotalHours < 12)
            throw new InvalidOperationException(
                "Updates and cancellations require at least 12 hours' notice.");
    }
}
```

RetrofitClient.kt (Android — API-only client configuration)

```
// mobile-client/.../data/remote/RetrofitClient.kt
// Singleton Retrofit client factory. All network calls from the Android
// app pass through this client; there is no local database fallback
// for authentication or business rules.
object RetrofitClient {
    private const val DEFAULT_BASE_URL = "http://10.0.2.2:5000/api/"
```

```

    @Volatile
    private var currentBaseUrl: String = DEFAULT_BASE_URL

    fun setBaseUrl(newBaseUrl: String) {
        currentBaseUrl = if (newBaseUrl.endsWith("/")) newBaseUrl
            else "$newBaseUrl/"
        apiService = null // force recreation on URL change
    }
}

```

AuthController.cs

```

using Microsoft.AspNetCore.Mvc;
using SolariX.Api.DTOs;
using SolariX.Api.Services;

namespace SolariX.Api.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    [Produces("application/json")]
    public class AuthController : ControllerBase
    {
        private readonly IUserService _userService;

        public AuthController(IUserService userService)
        {
            _userService = userService;
        }

        [HttpPost("login")]
        [ProducesResponseType(typeof(LoginResponse), StatusCodes.Status200OK)]
        [ProducesResponseType(StatusCodes.Status401Unauthorized)]
        [ProducesResponseType(StatusCodes.Status400BadRequest)]
        public async Task<IActionResult> Login([FromBody] LoginRequest request)
        {
            if (!ModelState.IsValid)
            {
                return BadRequest(ModelState);
            }

            try
            {

```

```

        var response = await _userService.AuthenticateAsync(request);
        if (response == null)
        {
            return Unauthorized(new { message = "Invalid NIC/Email or password." });
        }

        return Ok(response);
    }
    catch (InvalidOperationException ex)
    {
        return BadRequest(new { message = ex.Message });
    }
    catch (Exception ex)
    {
        return StatusCode(StatusCode.Status500InternalServerError, new { message =
ex.Message });
    }

    [HttpPost("register-prosumer")]
    [ProducesResponseType(typeof(UserResponse), StatusCode.Status201Created)]
    [ProducesResponseType(StatusCode.Status400BadRequest)]
    public async Task<IActionResult> RegisterProsumer([FromBody]
RegisterProsumerRequest request)
    {
        if (!ModelState.IsValid)
        {
            return BadRequest(ModelState);
        }

        try
        {
            var created = await _userService.RegisterProsumerAsync(request);
            return CreatedAtAction(nameof(GetProfile), new { nic = created.NIC }, created);
        }
        catch (ArgumentException ex)
        {
            return BadRequest(new { message = ex.Message });
        }
        catch (Exception ex)
        {
            return StatusCode(StatusCode.Status500InternalServerError, new { message =
ex.Message });
        }
    }

```

```

[HttpGet("profile/{nic}")]
[ProducesResponseType(typeof(UserResponse), StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status404NotFound)]
public async Task<IActionResult> GetProfile(string nic)
{
    var user = await _userService.GetUserByNicAsync(nic);
    if (user == null)
    {
        return NotFound(new { message = $"User with NIC '{nic}' was not found." });
    }

    return Ok(user); } }

```

JobCompletionActivity.kt

```
package com.solarix.mobile.ui.operator
```

```

import android.os.Bundle
import android.view.View
import android.widget.Toast
import androidx.appcompat.app.AlertDialog
import androidx.appcompat.app.AppCompatActivity
import androidx.lifecycle.LifecycleScope
import com.solarix.mobile.data.remote.dtos.VerifyAndCompleteRequest
import com.solarix.mobile.data.repository.ReservationRepository
import com.solarix.mobile.databinding.ActivityJobCompletionBinding
import com.solarix.mobile.utils.SessionManager
import kotlinx.coroutines.launch

```

```
class JobCompletionActivity : AppCompatActivity() {
```

```

    private lateinit var binding: ActivityJobCompletionBinding
    private lateinit var reservationRepository: ReservationRepository
    private lateinit var sessionManager: SessionManager
    private var qrToken: String = ""

```

```

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityJobCompletionBinding.inflate(layoutInflater)
        setContentView(binding.root)
    }

```

```

reservationRepository = ReservationRepository(this)
sessionManager = SessionManager.getInstance(this)

qrToken = intent.getStringExtra("SCANNED_QR_TOKEN") ?: ""
if (qrToken.isBlank()) {
    Toast.makeText(this, "No QR token received.", Toast.LENGTH_SHORT).show()
    finish()
    return
}

setupUI()
setupListeners()
}

private fun setupUI() {
    val operatorNic = sessionManager.getUserNic() ?: "OPERATOR"
    binding.tvScannedToken.text = qrToken
    binding.etOperatorNic.setText(operatorNic)
}

private fun setupListeners() {
    binding.btnFinalizeTransfer.setOnClickListener {
        finalizeTransaction()
    }
}

private fun finalizeTransaction() {
    val operatorNic = sessionManager.getUserNic()
    if (operatorNic.isNullOrBlank()) {
        Toast.makeText(this, "Operator session invalid.", Toast.LENGTH_SHORT).show()
        return
    }

    val energyStr = binding.etDeliveredEnergy.text.toString().trim()
    val energy = energyStr.toDoubleOrNull()
    if (energy == null || energy <= 0) {
        Toast.makeText(this, "Please enter a valid delivered energy volume.",
Toast.LENGTH_SHORT).show()
        return
    }
}

```

```

    }

    val notes = binding.etOperatorNotes.text.toString().trim()

    val request = VerifyAndCompleteRequest(
        qrToken = qrToken,
        operatorId = operatorNic,
        energyDeliveredKWh = energy,
        notes = notes.ifBlank { null }
    )

    binding.progressBar.visibility = View.VISIBLE
    binding.btnFinalizeTransfer.isEnabled = false

    lifecycleScope.launch {
        val result = reservationRepository.verifyAndComplete(request)
        binding.progressBar.visibility = View.GONE
        binding.btnFinalizeTransfer.isEnabled = true

        result.onSuccess { reservation ->
            AlertDialog.Builder(this@JobCompletionActivity)
                .setTitle("Energy Transfer Completed")
                .setMessage(
                    "Reservation #${reservation.reservationNumber} successfully verified!\n\n" +
                    "Delivered Energy: $energy kWh\n" +
                    "Status: ${reservation.status}\n" +
                    "Prosumer: ${reservation.prosumerNIC}\n" +
                    "Station: ${reservation.stationName}"
                )
                .setPositiveButton("Done") { _, _ ->
                    finish()
                }
                .setCancelable(false)
                .show()
        }.onFailure { error ->
            // Displays server error message (e.g. invalid HMAC signature or reservation not
            Approved)
            AlertDialog.Builder(this@JobCompletionActivity)
                .setTitle("Transaction Verification Failed")
                .setMessage(error.message ?: "Failed to verify QR token or complete reservation.")
        }
    }

```

```

        .setPositiveButton("OK", null)
        .show()
    }
}
}
}

```

AuthRepository.kt

```

package com.solarix.mobile.data.repository
import android.content.Context
import com.solarix.mobile.data.local.AppDatabase
import com.solarix.mobile.data.local.entities.UserEntity
import com.solarix.mobile.data.remote.RetrofitClient
import com.solarix.mobile.data.remote.dtos.LoginRequest
import com.solarix.mobile.data.remote.dtos.LoginResponse
import com.solarix.mobile.data.remote.dtos.RegisterProsumerRequest
import com.solarix.mobile.data.remote.dtos.UpdateProfileRequest
import com.solarix.mobile.data.remote.dtos.UserResponse
import com.solarix.mobile.utils.SessionManager
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.withContext

class AuthRepository(private val context: Context) {
    private val api = RetrofitClient.getService(context)
    private val db = AppDatabase.getDatabase(context)
    private val session = SessionManager.getInstance(context)

    suspend fun login(identifier: String, password: String): Result<LoginResponse> =
        withContext(Dispatchers.IO) {
            try {
                val response = api.login(LoginRequest(identifier.trim(), password))
                if (response.isSuccessful && response.body() != null) {
                    val body = response.body()!!
                    val entity = UserEntity(
                        nic = body.nic,
                        fullName = body.fullName,
                        email = body.email,
                        role = body.role,
                        status = body.status,

```

```

        token = body.token
    )
    session.saveFullUserSession(entity)
    Result.success(body)
} else {
    val msg = RetrofitClient.parseErrorMessage(response)
    Result.failure(Exception(msg))
}
} catch (e: Exception) {
    Result.failure(e)
}
}

```

```

suspend fun registerProsumer(request: RegisterProsumerRequest): Result<UserResponse> =
    withContext(Dispatchers.IO) {
        try {
            val response = api.registerProsumer(request)
            if (response.isSuccessful && response.body() != null) {
                Result.success(response.body()!!)
            } else {
                val msg = RetrofitClient.parseErrorMessage(response)
                Result.failure(Exception(msg))
            }
        } catch (e: Exception) {
            Result.failure(e)
        }
    }
}

```

```

suspend fun getProfile(nic: String): Result<UserResponse> =
    withContext(Dispatchers.IO) {
        try {
            val response = api.getProfile(nic)
            if (response.isSuccessful && response.body() != null) {
                val profile = response.body()!!
                // Update SQLite cache
                val existing = db.userDao().getUserByNic(nic)
                if (existing != null) {
                    db.userDao().update(
                        existing.copy(
                            fullName = profile.fullName,

```

```

        email = profile.email,
        phoneNumber = profile.phoneNumber,
        address = profile.address,
        solarCapacityKW = profile.solarCapacityKW
    )
    )
}
Result.success(profile)
} else {
    val msg = RetrofitClient.parseErrorMessage(response)
    Result.failure(Exception(msg))
}
} catch (e: Exception) {
    Result.failure(e)
}
}

```

```

suspend fun updateProfile(nic: String, request: UpdateProfileRequest): Result<UserResponse>
=
withContext(Dispatchers.IO) {
    try {
        val response = api.updateProfile(nic, request)
        if (response.isSuccessful && response.body() != null) {
            val updated = response.body()!!
            val existing = db.userDao().getUserByNic(nic)
            if (existing != null) {
                db.userDao().update(
                    existing.copy(
                        fullName = updated.fullName,
                        phoneNumber = updated.phoneNumber,
                        address = updated.address,
                        solarCapacityKW = updated.solarCapacityKW
                    )
                )
            }
            Result.success(updated)
        } else {
            val msg = RetrofitClient.parseErrorMessage(response)
            Result.failure(Exception(msg))
        }
    }
}

```

```

        } catch (e: Exception) {
            Result.failure(e)
        }
    }

suspend fun deactivateAccount(nic: String): Result<String> =
    withContext(Dispatchers.IO) {
        try {
            val response = api.deactivateAccount(nic, requestedByNic = nic)
            if (response.isSuccessful) {
                val msg = response.body()?.get("message") ?: "Account deactivated successfully."
                session.clearSession()
                Result.success(msg)
            } else {
                val msg = RetrofitClient.parseErrorMessage(response)
                Result.failure(Exception(msg))
            }
        } catch (e: Exception) {
            Result.failure(e)
        }
    }

suspend fun logout() {
    session.clearSession()
}
}

```

RegisterActivity.kt

```

package com.solarix.mobile.ui.auth

import android.os.Bundle
import android.view.View
import android.widget.Toast
import androidx.appcompat.app.AlertDialog
import androidx.appcompat.app.AppCompatActivity
import androidx.lifecycle.lifecycleScope
import com.solarix.mobile.data.remote.dtos.RegisterProsumerRequest
import com.solarix.mobile.data.repository.AuthRepository

```

```

import com.solarix.mobile.databinding.ActivityRegisterBinding
import kotlinx.coroutines.launch

class RegisterActivity : AppCompatActivity() {

    private lateinit var binding: ActivityRegisterBinding
    private lateinit var authRepository: AuthRepository

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityRegisterBinding.inflate(layoutInflater)
        setContentView(binding.root)

        authRepository = AuthRepository(this)

        binding.btnRegister.setOnClickListener {
            val nic = binding.etNic.text.toString().trim()
            val fullName = binding.etFullName.text.toString().trim()
            val email = binding.etEmail.text.toString().trim()
            val phone = binding.etPhone.text.toString().trim()
            val password = binding.etPassword.text.toString().trim()
            val address = binding.etAddress.text.toString().trim()
            val capacityStr = binding.etCapacity.text.toString().trim()
            val capacity = capacityStr.toDoubleOrNull() ?: 5.0

            if (nic.isBlank() || fullName.isBlank() || email.isBlank() || phone.isBlank() ||
password.isBlank()) {
                Toast.makeText(this, "Please fill in all required fields.",
Toast.LENGTH_SHORT).show()
                return@setOnClickListener
            }

            performRegistration(
                RegisterProsumerRequest(
                    nic = nic,
                    fullName = fullName,
                    email = email,
                    phoneNumber = phone,
                    password = password,
                    address = address,

```

```

        solarCapacityKW = capacity
    )
)
}

binding.tvBackToLogin.setOnClickListener {
    finish()
}
}

private fun performRegistration(request: RegisterProsumerRequest) {
    binding.progressBar.visibility = View.VISIBLE
    binding.btnRegister.isEnabled = false

    lifecycleScope.launch {
        val result = authRepository.registerProsumer(request)
        binding.progressBar.visibility = View.GONE
        binding.btnRegister.isEnabled = true

        result.onSuccess { user ->
            AlertDialog.Builder(this@RegisterActivity)
                .setTitle("Registration Submitted")
                .setMessage("Your prosumer account with NIC '${user.nic}' has been created with
status '${user.status}'. You may now log in.")
                .setPositiveButton("Sign In") { _, _ ->
                    finish()
                }
                .setCancelable(false)
                .show()
        }.onFailure { error ->
            AlertDialog.Builder(this@RegisterActivity)
                .setTitle("Registration Failed")
                .setMessage(error.message ?: "Could not complete registration. Ensure the NIC and
Email are unique.")
                .setPositiveButton("OK", null)
                .show()
        }
    }
}
}
}
}

```

StationsMapActivity.kt

```
package com.solarix.mobile.ui.map

import android.Manifest
import android.content.Intent
import android.content.pm.PackageManager
import android.os.Bundle
import android.view.View
import android.widget.Toast
import androidx.activity.result.contract.ActivityResultContracts
import androidx.appcompat.app.AppCompatActivity
import androidx.core.content.ContextCompat
import androidx.lifecycle.LifecycleScope
import com.google.android.gms.maps.CameraUpdateFactory
import com.google.android.gms.maps.GoogleMap
import com.google.android.gms.maps.OnMapReadyCallback
import com.google.android.gms.maps.SupportMapFragment
import com.google.android.gms.maps.model.BitmapDescriptorFactory
import com.google.android.gms.maps.model.LatLng
import com.google.android.gms.maps.model.LatLngBounds
import com.google.android.gms.maps.model.Marker
import com.google.android.gms.maps.model.MarkerOptions

import com.solarix.mobile.R
import com.solarix.mobile.data.remote.dtos.StationResponse
import com.solarix.mobile.data.repository.StationRepository
import com.solarix.mobile.databinding.ActivityStationsMapBinding
import com.solarix.mobile.ui.prosumer.CreateBookingActivity
import kotlinx.coroutines.launch

class StationsMapActivity : AppCompatActivity(), OnMapReadyCallback {

    private lateinit var binding: ActivityStationsMapBinding
    private lateinit var stationRepository: StationRepository
    private var googleMap: GoogleMap? = null

    private var stationsList: List<StationResponse> = emptyList()
    private val stationMarkerMap = HashMap<Marker, StationResponse>()
```

```

private var selectedStation: StationResponse? = null

private val requestLocationPermissionLauncher =
    registerForActivityResult(ActivityResultContracts.RequestMultiplePermissions()) {
permissions ->
        val fineLocationGranted =
permissions[Manifest.permission.ACCESS_FINE_LOCATION] ?: false
        val coarseLocationGranted =
permissions[Manifest.permission.ACCESS_COARSE_LOCATION] ?: false

        if (fineLocationGranted || coarseLocationGranted) {
            enableUserLocation()
        }
    }

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    binding = ActivityStationsMapBinding.inflate(layoutInflater)
    setContentView(binding.root)

    stationRepository = StationRepository(this)

    val mapFragment = supportFragmentManager.findFragmentById(R.id.mapFragment) as?
SupportMapFragment
    mapFragment?.getMapAsync(this)

    setupListeners()
}

override fun onMapReady(map: GoogleMap) {
    googleMap = map

    map.uiSettings.isZoomControlsEnabled = true
    map.uiSettings.isCompassEnabled = true

    checkLocationPermissions()
    loadStationMarkers()

    map.setOnMarkerClickListener { marker ->
        val station = stationMarkerMap[marker]

```

```

        if (station != null) {
            showStationDetails(station)
        }
        false
    }

    map.setOnMapClickListener {
        hideStationDetails()
    }
}

private fun checkLocationPermissions() {
    if (ContextCompat.checkSelfPermission(this,
Manifest.permission.ACCESS_FINE_LOCATION)
        == PackageManager.PERMISSION_GRANTED
    ) {
        enableUserLocation()
    } else {
        requestLocationPermissionLauncher.launch(
            arrayOf(
                Manifest.permission.ACCESS_FINE_LOCATION,
                Manifest.permission.ACCESS_COARSE_LOCATION
            )
        )
    }
}

private fun enableUserLocation() {
    try {
        googleMap?.isMyLocationEnabled = true
    } catch (e: SecurityException) {
        e.printStackTrace()
    }
}

private fun setupListeners() {
    binding.btnBack.setOnClickListener {
        onBackPressedDispatcher.onBackPressed()
    }
}

```

```

binding.btnRefreshMap.setOnClickListener {
    loadStationMarkers()
}

binding.btnCloseDetails.setOnClickListener {
    hideStationDetails()
}

binding.btnBookAtStation.setOnClickListener {
    val station = selectedStation ?: return@setOnClickListener
    val intent = Intent(this, CreateBookingActivity::class.java).apply {
        putExtra("STATION_ID", station.id)
    }
    startActivity(intent)
}

private fun loadStationMarkers() {
    binding.tvStationCount.text = "Loading microgrid nodes..."

    lifecycleScope.launch {
        val result = stationRepository.getStations(activeOnly = true)
        result.onSuccess { stations ->
            stationsList = stations
            binding.tvStationCount.text = "${stations.size} Active Hubs Located"
            renderMarkersOnMap(stations)
        }.onFailure { error ->
            binding.tvStationCount.text = "Failed to load hubs"
            Toast.makeText(this@StationsMapActivity, "Error: ${error.message}",
                Toast.LENGTH_SHORT).show()
        }
    }
}

private fun renderMarkersOnMap(stations: List<StationResponse>) {
    val map = googleMap ?: return
    map.clear()
    stationMarkerMap.clear()

    if (stations.isEmpty()) return

```

```

val boundsBuilder = LatLngBounds.Builder()

for (station in stations) {
    val latLng = LatLng(station.latitude, station.longitude)
    boundsBuilder.include(latLng)

    val markerHue = if (station.availableBatterySlots > 0) {
        BitmapDescriptorFactory.HUE_AZURE
    } else {
        BitmapDescriptorFactory.HUE_ORANGE
    }

    val markerOptions = MarkerOptions()
        .position(latLng)
        .title(station.stationName)
        .snippet("Slots: ${station.availableBatterySlots}/${station.totalBatterySlots} |
${station.capacityKWh} kWh")
        .icon(BitmapDescriptorFactory.defaultMarker(markerHue))

    val marker = map.addMarker(markerOptions)
    if (marker != null) {
        stationMarkerMap[marker] = station
    }
}

try {
    val bounds = boundsBuilder.build()
    map.animateCamera(CameraUpdateFactory.newLatLngBounds(bounds, 120))
} catch (e: Exception) {
    // If only 1 station or bounding failed, center on first
    val first = stations.first()
    map.animateCamera(CameraUpdateFactory.newLatLngZoom(LatLng(first.latitude,
first.longitude), 12f))
}

private fun showStationDetails(station: StationResponse) {
    selectedStation = station
    binding.tvDetailStationName.text = "${station.stationName} (${station.stationCode})"
}

```

```

        binding.tvDetailLocation.text = station.locationName
        binding.tvDetailCapacity.text = "${station.capacityKWh} kWh"
        binding.tvDetailSlots.text = "${station.availableBatterySlots} / ${station.totalBatterySlots}"
        binding.tvDetailHours.text = station.operationalHours ?: "06:00 - 20:00"

        binding.cardStationDetails.visibility = View.VISIBLE
    }

    private fun hideStationDetails() {
        selectedStation = null
        binding.cardStationDetails.visibility = View.GONE
    }
}

```

JobCompletionActivity.kt

```

package com.solarix.mobile.ui.operator

import android.os.Bundle
import android.view.View
import android.widget.Toast
import androidx.appcompat.app.AlertDialog
import androidx.appcompat.app.AppCompatActivity
import androidx.lifecycle.LifecycleScope
import com.solarix.mobile.data.remote.dtos.VerifyAndCompleteRequest
import com.solarix.mobile.data.repository.ReservationRepository
import com.solarix.mobile.databinding.ActivityJobCompletionBinding
import com.solarix.mobile.utils.SessionManager
import kotlinx.coroutines.launch

class JobCompletionActivity : AppCompatActivity() {

    private lateinit var binding: ActivityJobCompletionBinding
    private lateinit var reservationRepository: ReservationRepository
    private lateinit var sessionManager: SessionManager
    private var qrToken: String = ""

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityJobCompletionBinding.inflate(layoutInflater)
    }
}

```

```

setContentView(binding.root)

reservationRepository = ReservationRepository(this)
sessionManager = SessionManager.getInstance(this)

qrToken = intent.getStringExtra("SCANNED_QR_TOKEN") ?: ""
if (qrToken.isBlank()) {
    Toast.makeText(this, "No QR token received.", Toast.LENGTH_SHORT).show()
    finish()
    return
}

setupUI()
setupListeners()
}

private fun setupUI() {
    val operatorNic = sessionManager.getUserNic() ?: "OPERATOR"
    binding.tvScannedToken.text = qrToken
    binding.etOperatorNic.setText(operatorNic)
}

private fun setupListeners() {
    binding.btnFinalizeTransfer.setOnClickListener {
        finalizeTransaction()
    }
}

private fun finalizeTransaction() {
    val operatorNic = sessionManager.getUserNic()
    if (operatorNic.isNullOrEmpty()) {
        Toast.makeText(this, "Operator session invalid.", Toast.LENGTH_SHORT).show()
        return
    }

    val energyStr = binding.etDeliveredEnergy.text.toString().trim()
    val energy = energyStr.toDoubleOrNull()
    if (energy == null || energy <= 0) {
        Toast.makeText(this, "Please enter a valid delivered energy volume.",
Toast.LENGTH_SHORT).show()

```

```

        return
    }

    val notes = binding.etOperatorNotes.text.toString().trim()

    val request = VerifyAndCompleteRequest(
        qrToken = qrToken,
        operatorId = operatorNic,
        energyDeliveredKWh = energy,
        notes = notes.ifBlank { null }
    )

    binding.progressBar.visibility = View.VISIBLE
    binding.btnFinalizeTransfer.isEnabled = false

    lifecycleScope.launch {
        val result = reservationRepository.verifyAndComplete(request)
        binding.progressBar.visibility = View.GONE
        binding.btnFinalizeTransfer.isEnabled = true

        result.onSuccess { reservation ->
            AlertDialog.Builder(this@JobCompletionActivity)
                .setTitle("Energy Transfer Completed")
                .setMessage(
                    "Reservation #${reservation.reservationNumber} successfully verified!\n\n" +
                    "Delivered Energy: $energy kWh\n" +
                    "Status: ${reservation.status}\n" +
                    "Prosumer: ${reservation.prosumerNIC}\n" +
                    "Station: ${reservation.stationName}"
                )
                .setPositiveButton("Done") { _, _ ->
                    finish()
                }
                .setCancelable(false)
                .show()
        }.onFailure { error ->
            // Displays server error message (e.g. invalid HMAC signature or reservation not
Approved)
            AlertDialog.Builder(this@JobCompletionActivity)
                .setTitle("Transaction Verification Failed")

```

```

        .setMessage(error.message ?: "Failed to verify QR token or complete reservation.")
        .setPositiveButton("OK", null)
        .show()
    }
}
}
}

```

QRScannerActivity.kt

```

package com.solarix.mobile.ui.map

import android.Manifest
import android.content.Intent
import android.content.pm.PackageManager
import android.os.Bundle
import android.view.View
import android.widget.Toast
import androidx.activity.result.contract.ActivityResultContracts
import androidx.appcompat.app.AppCompatActivity
import androidx.core.content.ContextCompat
import androidx.lifecycle.lifecycleScope
import com.google.android.gms.maps.CameraUpdateFactory
import com.google.android.gms.maps.GoogleMap
import com.google.android.gms.maps.OnMapReadyCallback
import com.google.android.gms.maps.SupportMapFragment
import com.google.android.gms.maps.model.BitmapDescriptorFactory
import com.google.android.gms.maps.model.LatLng
import com.google.android.gms.maps.model.LatLngBounds
import com.google.android.gms.maps.model.Marker
import com.google.android.gms.maps.model.MarkerOptions
import com.solarix.mobile.R
import com.solarix.mobile.data.remote.dtos.StationResponse
import com.solarix.mobile.data.repository.StationRepository
import com.solarix.mobile.databinding.ActivityStationsMapBinding
import com.solarix.mobile.ui.prosumer.CreateBookingActivity
import kotlinx.coroutines.launch

class StationsMapActivity : AppCompatActivity(), OnMapReadyCallback {

```

```

private lateinit var binding: ActivityStationsMapBinding
private lateinit var stationRepository: StationRepository
private var googleMap: GoogleMap? = null

private var stationsList: List<StationResponse> = emptyList()
private val stationMarkerMap = HashMap<Marker, StationResponse>()
private var selectedStation: StationResponse? = null

private val requestLocationPermissionLauncher =
    registerForActivityResult(ActivityResultContracts.RequestMultiplePermissions()) {
permissions ->
        val fineLocationGranted =
permissions[Manifest.permission.ACCESS_FINE_LOCATION] ?: false
        val coarseLocationGranted =
permissions[Manifest.permission.ACCESS_COARSE_LOCATION] ?: false

        if (fineLocationGranted || coarseLocationGranted) {
            enableUserLocation()
        }
    }

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    binding = ActivityStationsMapBinding.inflate(layoutInflater)
    setContentView(binding.root)

    stationRepository = StationRepository(this)

    val mapFragment = supportFragmentManager.findFragmentById(R.id.mapFragment) as?
SupportMapFragment
    mapFragment?.getMapAsync(this)

    setupListeners()
}

override fun onMapReady(map: GoogleMap) {
    googleMap = map

    map.uiSettings.isZoomControlsEnabled = true
    map.uiSettings.isCompassEnabled = true

```

```

checkLocationPermissions()
loadStationMarkers()

map.setOnMarkerClickListener { marker ->
    val station = stationMarkerMap[marker]
    if (station != null) {
        showStationDetails(station)
    }
    false
}

map.setOnMapClickListener {
    hideStationDetails()
}
}

private fun checkLocationPermissions() {
    if (ContextCompat.checkSelfPermission(this,
Manifest.permission.ACCESS_FINE_LOCATION)
    == PackageManager.PERMISSION_GRANTED
    ) {
        enableUserLocation()
    } else {
        requestLocationPermissionLauncher.launch(
            arrayOf(
                Manifest.permission.ACCESS_FINE_LOCATION,
                Manifest.permission.ACCESS_COARSE_LOCATION
            )
        )
    }
}

private fun enableUserLocation() {
    try {
        googleMap?.isMyLocationEnabled = true
    } catch (e: SecurityException) {
        e.printStackTrace()
    }
}

```

```

private fun setupListeners() {
    binding.btnBack.setOnClickListener {
        onBackPressedDispatcher.onBackPressed()
    }

    binding.btnRefreshMap.setOnClickListener {
        loadStationMarkers()
    }

    binding.btnCloseDetails.setOnClickListener {
        hideStationDetails()
    }

    binding.btnBookAtStation.setOnClickListener {
        val station = selectedStation ?: return@setOnClickListener
        val intent = Intent(this, CreateBookingActivity::class.java).apply {
            putExtra("STATION_ID", station.id)
        }
        startActivity(intent)
    }
}

private fun loadStationMarkers() {
    binding.tvStationCount.text = "Loading microgrid nodes..."

    lifecycleScope.launch {
        val result = stationRepository.getStations(activeOnly = true)
        result.onSuccess { stations ->
            stationsList = stations
            binding.tvStationCount.text = "${stations.size} Active Hubs Located"
            renderMarkersOnMap(stations)
        }.onFailure { error ->
            binding.tvStationCount.text = "Failed to load hubs"
            Toast.makeText(this@StationsMapActivity, "Error: ${error.message}",
Toast.LENGTH_SHORT).show()
        }
    }
}

```

```

private fun renderMarkersOnMap(stations: List<StationResponse>) {
    val map = googleMap ?: return
    map.clear()
    stationMarkerMap.clear()

    if (stations.isEmpty()) return

    val boundsBuilder = LatLngBounds.Builder()

    for (station in stations) {
        val latLng = LatLng(station.latitude, station.longitude)
        boundsBuilder.include(latLng)

        val markerHue = if (station.availableBatterySlots > 0) {
            BitmapDescriptorFactory.HUE_AZURE
        } else {
            BitmapDescriptorFactory.HUE_ORANGE
        }

        val markerOptions = MarkerOptions()
            .position(latLng)
            .title(station.stationName)
            .snippet("Slots: ${station.availableBatterySlots}/${station.totalBatterySlots} |
${station.capacityKWh} kWh")
            .icon(BitmapDescriptorFactory.defaultMarker(markerHue))

        val marker = map.addMarker(markerOptions)
        if (marker != null) {
            stationMarkerMap[marker] = station
        }
    }

    try {
        val bounds = boundsBuilder.build()
        map.animateCamera(CameraUpdateFactory.newLatLngBounds(bounds, 120))
    } catch (e: Exception) {
        // If only 1 station or bounding failed, center on first
        val first = stations.first()
        map.animateCamera(CameraUpdateFactory.newLatLngZoom(LatLng(first.latitude,
first.longitude), 12f))
    }
}

```

```

    }
}

private fun showStationDetails(station: StationResponse) {
    selectedStation = station
    binding.tvDetailStationName.text = "${station.stationName} (${station.stationCode})"
    binding.tvDetailLocation.text = station.locationName
    binding.tvDetailCapacity.text = "${station.capacityKWh} kWh"
    binding.tvDetailSlots.text = "${station.availableBatterySlots} / ${station.totalBatterySlots}"
    binding.tvDetailHours.text = station.operationalHours ?: "06:00 - 20:00"

    binding.cardStationDetails.visibility = View.VISIBLE
}

private fun hideStationDetails() {
    selectedStation = null
    binding.cardStationDetails.visibility = View.GONE
}
}

```

BookingHistoryActivity.kt

```

package com.solarix.mobile.ui.prosumer

import android.content.Intent
import android.os.Bundle
import android.text.Editable
import android.text.TextWatcher
import android.view.View
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
import androidx.lifecycle.LifecycleScope
import androidx.recyclerview.widget.LinearLayoutManager
import com.solarix.mobile.data.remote.dtos.ReservationResponse
import com.solarix.mobile.data.repository.ReservationRepository
import com.solarix.mobile.databinding.ActivityBookingHistoryBinding
import com.solarix.mobile.ui.prosumer.adapters.BookingsAdapter
import com.solarix.mobile.utils.SessionManager
import kotlinx.coroutines.launch

```

```

class BookingHistoryActivity : AppCompatActivity() {

    private lateinit var binding: ActivityBookingHistoryBinding
    private lateinit var reservationRepository: ReservationRepository
    private lateinit var sessionManager: SessionManager
    private lateinit var adapter: BookingsAdapter

    private var allReservations: List<ReservationResponse> = emptyList()
    private var currentFilterStatus: String = "All"
    private var currentSearchQuery: String = ""

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityBookingHistoryBinding.inflate(layoutInflater)
        setContentView(binding.root)

        reservationRepository = ReservationRepository(this)
        sessionManager = SessionManager.getInstance(this)

        setupRecyclerView()
        setupFilters()
        setupListeners()

        // Check if an initial status filter was passed
        val initialStatus = intent.getStringExtra("FILTER_STATUS")
        if (!initialStatus.isNullOrBlank()) {
            currentFilterStatus = initialStatus
            selectChipForStatus(initialStatus)
        }
    }

    override fun onResume() {
        super.onResume()
        loadBookings()
    }

    private fun setupRecyclerView() {
        adapter = BookingsAdapter(
            bookings = emptyList(),
            onViewQRClick = { item ->

```

```

        val intent = Intent(this, QRDisplayActivity::class.java).apply {
            putExtra("RESERVATION_DATA", item)
        }
        startActivity(intent)
    },
    onEditClick = { item ->
        val intent = Intent(this, EditBookingActivity::class.java).apply {
            putExtra("RESERVATION_DATA", item)
        }
        startActivity(intent)
    }
)
binding.rvBookings.layoutManager = LinearLayoutManager(this)
binding.rvBookings.adapter = adapter
}

private fun setupFilters() {
    binding.chipGroupStatus.setOnCheckedChangeListener { _, checkedIds ->
        val checkedId = checkedIds.firstOrNull() ?: View.NO_ID
        currentFilterStatus = when (checkedId) {
            binding.chipApproved.id -> "Approved"
            binding.chipPending.id -> "Pending"
            binding.chipCompleted.id -> "Completed"
            binding.chipCancelled.id -> "Cancelled"
            else -> "All"
        }
        applyFilters()
    }
}

binding.etSearchQuery.addTextChangedListener(object : TextWatcher {
    override fun beforeTextChanged(s: CharSequence?, start: Int, count: Int, after: Int) {}
    override fun onTextChanged(s: CharSequence?, start: Int, before: Int, count: Int) {
        currentSearchQuery = s?.toString()?.trim() ?: ""
        applyFilters()
    }
    override fun afterTextChanged(s: Editable?) {}
})
}

private fun selectChipForStatus(status: String) {

```

```

when (status.lowercase()) {
    "approved" -> binding.chipApproved.isChecked = true
    "pending" -> binding.chipPending.isChecked = true
    "completed" -> binding.chipCompleted.isChecked = true
    "cancelled" -> binding.chipCancelled.isChecked = true
    else -> binding.chipAll.isChecked = true
}
}

private fun setupListeners() {
    binding.btnBack.setOnClickListener {
        onBackPressedDispatcher.onBackPressed()
    }

    binding.swipeRefreshBookings.setOnRefreshListener {
        loadBookings()
    }
}

private fun loadBookings() {
    val nic = sessionManager.getUserNic() ?: return
    binding.progressBar.visibility = View.VISIBLE

    lifecycleScope.launch {
        val result = reservationRepository.getReservationsByProsumer(nic)
        binding.progressBar.visibility = View.GONE
        binding.swipeRefreshBookings.isRefreshing = false

        result.onSuccess { reservations ->
            allReservations = reservations
            applyFilters()
        }.onFailure { error ->
            Toast.makeText(this@BookingHistoryActivity, "Error loading bookings:
${error.message}", Toast.LENGTH_SHORT).show()
            applyFilters()
        }
    }
}

private fun applyFilters() {

```

```

var filtered = allReservations

// Status Filter
if (!currentFilterStatus.equals("All", ignoreCase = true)) {
    filtered = filtered.filter { it.status.equals(currentFilterStatus, ignoreCase = true) }
}

// Search Query (by station name or reservation number)
if (currentSearchQuery.isNotBlank()) {
    val q = currentSearchQuery.lowercase()
    filtered = filtered.filter {
        it.stationName.lowercase().contains(q) ||
        it.reservationNumber.lowercase().contains(q) ||
        it.stationId.lowercase().contains(q)
    }
}

adapter.updateData(filtered)

if (filtered.isEmpty()) {
    binding.tvEmptyState.visibility = View.VISIBLE
} else {
    binding.tvEmptyState.visibility = View.GONE
}
}
}

```

BookingSummaryActivity.kt

```

package com.solarix.mobile.ui.prosumer

import android.content.Intent
import android.os.Bundle
import android.view.View
import androidx.appcompat.app.AppCompatActivity
import androidx.core.content.ContextCompat
import com.solarix.mobile.R
import com.solarix.mobile.data.remote.dtos.ReservationResponse
import com.solarix.mobile.databinding.ActivityBookingSummaryBinding

```

```

class BookingSummaryActivity : AppCompatActivity() {

    private lateinit var binding: ActivityBookingSummaryBinding
    private var reservation: ReservationResponse? = null

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityBookingSummaryBinding.inflate(layoutInflater)
        setContentView(binding.root)

        @Suppress("DEPRECATION")
        reservation = intent.getSerializableExtra("RESERVATION_DATA") as?
ReservationResponse

        if (reservation == null) {
            finish()
            return
        }

        populateSummary(reservation!!)

        binding.btnViewQR.setOnClickListener {
            val intent = Intent(this, QRDisplayActivity::class.java).apply {
                putExtra("RESERVATION_DATA", reservation)
            }
            startActivity(intent)
        }

        binding.btnBackToDashboard.setOnClickListener {
            val intent = Intent(this, ProsumerDashboardActivity::class.java).apply {
                flags = Intent.FLAG_ACTIVITY_CLEAR_TOP or
Intent.FLAG_ACTIVITY_SINGLE_TOP
            }
            startActivity(intent)
            finish()
        }
    }

    private fun populateSummary(res: ReservationResponse) {
        binding.tvReservationNumber.text = if (res.reservationNumber.isNotBlank()) {

```

```

        res.reservationNumber
    } else {
        "#RES-#{res.id.takeLast(6).uppercase()}"
    }

    binding.tvStationName.text = if (res.stationName.isNotBlank()) res.stationName else "Hub:
    ${res.stationId}"

    val datePart = res.reservationDate.split("T").firstOrNull() ?: res.reservationDate
    val startPart = res.startTime.split("T").lastOrNull()?.take(5) ?: res.startTime
    val endPart = res.endTime.split("T").lastOrNull()?.take(5) ?: res.endTime
    binding.tvSlotTiming.text = "$datePart | $startPart - $endPart"

    binding.tvEnergyVolume.text = String.format("%.1f kW", res.energyAmountKW)
    binding.tvTradeType.text = res.tradeType

    binding.tvStatusBadge.text = res.status

    when (res.status.lowercase()) {
        "approved" -> {
            binding.tvStatusBadge.setBackgroundResource(R.drawable.bg_badge_approved)
            binding.tvStatusBadge.setTextColor(ContextCompat.getColor(this,
R.color.status_approved))

            binding.ivStatusIcon.setImageResource(android.R.drawable.checkbox_on_background)
            binding.ivStatusIcon.setColorFilter(ContextCompat.getColor(this,
R.color.status_approved))
            binding.btnViewQR.visibility = View.VISIBLE
        }
        "pending" -> {
            binding.tvStatusBadge.setBackgroundResource(R.drawable.bg_badge_pending)
            binding.tvStatusBadge.setTextColor(ContextCompat.getColor(this,
R.color.status_pending))
            binding.ivStatusIcon.setImageResource(android.R.drawable.ic_dialog_info)
            binding.ivStatusIcon.setColorFilter(ContextCompat.getColor(this,
R.color.status_pending))
            binding.btnViewQR.visibility = View.GONE
        }
        "cancelled" -> {
            binding.tvStatusBadge.setBackgroundResource(R.drawable.bg_badge_cancelled)

```

```

        binding.tvStatusBadge.setTextColor(ContextCompat.getColor(this,
R.color.status_cancelled))
        binding.ivStatusIcon.setImageResource(android.R.drawable.ic_delete)
        binding.ivStatusIcon.setColorFilter(ContextCompat.getColor(this,
R.color.status_cancelled))
        binding.tvSummaryTitle.text = "Booking Cancelled"
        binding.btnViewQR.visibility = View.GONE
    }
    else -> {
        binding.tvStatusBadge.setBackgroundResource(R.drawable.bg_badge_completed)
        binding.tvStatusBadge.setTextColor(ContextCompat.getColor(this,
R.color.status_completed))
        binding.btnViewQR.visibility = View.GONE
    }
}
}
}
}

```

CreateBookingActivity.kt

```

package com.solarix.mobile.ui.prosumer

import android.app.DatePickerDialog
import android.content.Intent
import android.os.Bundle
import android.view.View
import android.widget.AdapterView
import android.widget.AdapterView.OnItemClickListener
import android.widget.ArrayAdapter
import android.widget.Toast
import androidx.appcompat.app.AlertDialog
import androidx.appcompat.app.AppCompatActivity
import androidx.lifecycle.LifecycleScope
import com.solarix.mobile.data.remote.dtos.CreateReservationRequest
import com.solarix.mobile.data.remote.dtos.SlotResponse
import com.solarix.mobile.data.remote.dtos.StationResponse
import com.solarix.mobile.data.repository.ReservationRepository
import com.solarix.mobile.data.repository.StationRepository
import com.solarix.mobile.databinding.ActivityCreateBookingBinding
import com.solarix.mobile.utils.SessionManager
import kotlinx.coroutines.launch

```

```

import java.text.SimpleDateFormat
import java.util.*

class CreateBookingActivity : AppCompatActivity() {

    private lateinit var binding: ActivityCreateBookingBinding
    private lateinit var stationRepository: StationRepository
    private lateinit var reservationRepository: ReservationRepository
    private lateinit var sessionManager: SessionManager

    private var stationsList: List<StationResponse> = emptyList()
    private var slotsList: List<SlotResponse> = emptyList()
    private var selectedStationId: String? = null
    private var selectedSlotId: String? = null
    private var selectedDateStr: String = ""

    private val dateFormatter = SimpleDateFormat("yyyy-MM-dd", Locale.getDefault())

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityCreateBookingBinding.inflate(layoutInflater)
        setContentView(binding.root)

        stationRepository = StationRepository(this)
        reservationRepository = ReservationRepository(this)
        sessionManager = SessionManager.getInstance(this)

        // Default to today
        val calendar = Calendar.getInstance()
        selectedDateStr = dateFormatter.format(calendar.time)
        binding.etDate.setText(selectedDateStr)

        val preselectedStationId = intent.getStringExtra("STATION_ID")

        setupDatePicker()
        setupListeners()
        loadStations(preselectedStationId)
    }

    private fun setupDatePicker() {

```

```

val calendar = Calendar.getInstance()
val dateSetListener = DatePickerDialog.OnDateSetListener { _, year, month, dayOfMonth -
>
    calendar.set(Calendar.YEAR, year)
    calendar.set(Calendar.MONTH, month)
    calendar.set(Calendar.DAY_OF_MONTH, dayOfMonth)
    selectedDateStr = dateFormatter.format(calendar.time)
    binding.etDate.setText(selectedDateStr)
    loadSlots()
}

binding.etDate.setOnClickListener {
    DatePickerDialog(
        this,
        dateSetListener,
        calendar.get(Calendar.YEAR),
        calendar.get(Calendar.MONTH),
        calendar.get(Calendar.DAY_OF_MONTH)
    ).show()
}

private fun setupListeners() {
    binding.spinnerStations.onItemSelectedListener = object :
    AdapterView.OnItemSelectedListener {
        override fun onItemSelected(parent: AdapterView<*>?, view: View?, position: Int, id:
    Long) {
            if (position in stationsList.indices) {
                selectedStationId = stationsList[position].id
                loadSlots()
            }
        }

        override fun onNothingSelected(parent: AdapterView<*>?) {
            selectedStationId = null
        }
    }

    binding.spinnerSlots.onItemSelectedListener = object :
    AdapterView.OnItemSelectedListener {

```

```

        override fun onItemSelected(parent: AdapterView<*>?, view: View?, position: Int, id:
Long) {
            if (position in slotsList.indices) {
                selectedSlotId = slotsList[position].id
            }
        }

        override fun onNothingSelected(parent: AdapterView<*>?) {
            selectedSlotId = null
        }
    }

    binding.btnSubmitBooking.setOnClickListener {
        submitBooking()
    }
}

private fun loadStations(preselectedId: String?) {
    binding.progressBar.visibility = View.VISIBLE

    lifecycleScope.launch {
        val result = stationRepository.getStations(activeOnly = true)
        binding.progressBar.visibility = View.GONE

        result.onSuccess { stations ->
            stationsList = stations
            val displayNames = stations.map { "${it.stationName} (${it.locationName})" }
            val adapter = ArrayAdapter(this@CreateBookingActivity,
android.R.layout.simple_spinner_dropdown_item, displayNames)
            binding.spinnerStations.adapter = adapter

            if (!preselectedId.isNullOrEmpty()) {
                val index = stations.indexOfFirst { it.id == preselectedId }
                if (index >= 0) {
                    binding.spinnerStations.setSelection(index)
                    selectedStationId = preselectedId
                }
            } else if (stations.isNotEmpty()) {
                selectedStationId = stations[0].id
            }
        }
    }
}

```

```

        loadSlots()
    }.onFailure { error ->
        Toast.makeText(this@CreateBookingActivity, "Error loading stations:
${error.message}", Toast.LENGTH_LONG).show()
    }
}

private fun loadSlots() {
    val stId = selectedStationId ?: return
    if (selectedDateStr.isBlank()) return

    binding.progressBar.visibility = View.VISIBLE
    lifecycleScope.launch {
        val result = stationRepository.getSlotsForStation(stId, selectedDateStr)
        binding.progressBar.visibility = View.GONE

        result.onSuccess { slots ->
            slotsList = slots
            if (slots.isEmpty()) {
                val emptyAdapter = ArrayAdapter(
                    this@CreateBookingActivity,
                    android.R.layout.simple_spinner_dropdown_item,
                    listOf("No slots generated for this date")
                )
                binding.spinnerSlots.adapter = emptyAdapter
                selectedSlotId = null
            } else {
                val displaySlots = slots.map {
                    val start = it.startTime.split("T").lastOrNull()?.take(5) ?: it.startTime
                    val end = it.endTime.split("T").lastOrNull()?.take(5) ?: it.endTime
                    "$start - $end (Avail: ${it.availableCapacityKW} kW)"
                }
                val adapter = ArrayAdapter(this@CreateBookingActivity,
                    android.R.layout.simple_spinner_dropdown_item, displaySlots)
                binding.spinnerSlots.adapter = adapter
                selectedSlotId = slots[0].id
            }
        }.onFailure {

```

```

        val emptyAdapter = ArrayAdapter(
            this@CreateBookingActivity,
            android.R.layout.simple_spinner_dropdown_item,
            listOf("Failed to load slots")
        )
        binding.spinnerSlots.adapter = emptyAdapter
        selectedSlotId = null
    }
}

private fun submitBooking() {
    val nic = sessionManager.getUserNic()
    if (nic.isNullOrEmpty()) {
        Toast.makeText(this, "Session invalid. Please login again.",
            Toast.LENGTH_SHORT).show()
        return
    }

    val stationId = selectedStationId
    if (stationId.isNullOrEmpty()) {
        Toast.makeText(this, "Please select a solar station hub.",
            Toast.LENGTH_SHORT).show()
        return
    }

    val slotId = selectedSlotId
    if (slotId.isNullOrEmpty()) {
        Toast.makeText(this, "Please select an available energy slot.",
            Toast.LENGTH_SHORT).show()
        return
    }

    val energyStr = binding.etEnergyAmount.text.toString().trim()
    val energy = energyStr.toDoubleOrNull()
    if (energy == null || energy <= 0) {
        Toast.makeText(this, "Please enter a valid energy volume.",
            Toast.LENGTH_SHORT).show()
        return
    }
}

```

```

val tradeType = if (binding.rbDropOff.isChecked) "DropOff" else "Charge"

val request = CreateReservationRequest(
    prosumerNIC = nic,
    stationId = stationId,
    slotId = slotId,
    energyAmountKW = energy,
    tradeType = tradeType
)

binding.progressBar.visibility = View.VISIBLE
binding.btnSubmitBooking.isEnabled = false

lifecycleScope.launch {
    val result = reservationRepository.createReservation(request)
    binding.progressBar.visibility = View.GONE
    binding.btnSubmitBooking.isEnabled = true

    result.onSuccess { reservation ->
        Toast.makeText(this@CreateBookingActivity, "Reservation created successfully!",
            Toast.LENGTH_SHORT).show()
        val intent = Intent(this@CreateBookingActivity,
            BookingSummaryActivity::class.java).apply {
            putExtra("RESERVATION_DATA", reservation)
        }
        startActivity(intent)
        finish()
    }.onFailure { error ->
        // Render exact server error message (e.g. 7-day validation or station deactivation
guard)
        AlertDialog.Builder(this@CreateBookingActivity)
            .setTitle("Booking Rejected by Server")
            .setMessage(error.message ?: "The server rejected this reservation request.")
            .setPositiveButton("OK", null)
            .show()
    }
}
}
}
}

```

EditBookingActivity.kt

```
package com.solarix.mobile.ui.prosumer

import android.content.Intent
import android.os.Bundle
import android.view.View
import android.widget.AdapterView
import android.widget.Toast
import androidx.appcompat.app.AlertDialog
import androidx.appcompat.app.AppCompatActivity
import androidx.lifecycle.LifecycleScope
import com.solarix.mobile.data.remote.dtos.CancelReservationRequest
import com.solarix.mobile.data.remote.dtos.ReservationResponse
import com.solarix.mobile.data.remote.dtos.SlotResponse
import com.solarix.mobile.data.remote.dtos.UpdateReservationRequest
import com.solarix.mobile.data.repository.ReservationRepository
import com.solarix.mobile.data.repository.StationRepository
import com.solarix.mobile.databinding.ActivityEditBookingBinding
import com.solarix.mobile.utils.SessionManager
import kotlinx.coroutines.launch

class EditBookingActivity : AppCompatActivity() {

    private lateinit var binding: ActivityEditBookingBinding
    private lateinit var reservationRepository: ReservationRepository
    private lateinit var stationRepository: StationRepository
    private lateinit var sessionManager: SessionManager

    private var currentReservation: ReservationResponse? = null
    private var slotsList: List<SlotResponse> = emptyList()
    private var selectedSlotId: String? = null

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityEditBookingBinding.inflate(layoutInflater)
        setContentView(binding.root)

        reservationRepository = ReservationRepository(this)
```

```

stationRepository = StationRepository(this)
sessionManager = SessionManager.getInstance(this)

@Suppress("DEPRECATION")
currentReservation = intent.getSerializableExtra("RESERVATION_DATA") as?
ReservationResponse

if (currentReservation == null) {
    val resId = intent.getStringExtra("RESERVATION_ID")
    if (!resId.isNullOrBlank()) {
        loadReservationById(resId)
    } else {
        Toast.makeText(this, "No reservation data provided.",
Toast.LENGTH_SHORT).show()
        finish()
    }
} else {
    populateUI(currentReservation!!)
}

setupListeners()
}

private fun loadReservationById(id: String) {
    binding.progressBar.visibility = View.VISIBLE
    lifecycleScope.launch {
        val result = reservationRepository.getReservationById(id)
        binding.progressBar.visibility = View.GONE
        result.onSuccess {
            currentReservation = it
            populateUI(it)
        }.onFailure {
            Toast.makeText(this@EditBookingActivity, "Could not load booking: ${it.message}",
Toast.LENGTH_SHORT).show()
            finish()
        }
    }
}

private fun populateUI(res: ReservationResponse) {

```

```

        binding.tvBookingReference.text = "Reservation: ${res.reservationNumber.ifBlank {
"#RES-${res.id.takeLast(6).uppercase()}" }}"
        binding.tvStationInfo.text = "Station: ${res.stationName.ifBlank { res.stationId }}"
        binding.etEditEnergy.setText(res.energyAmountKW.toString())

        if (res.tradeType.equals("Charge", ignoreCase = true)) {
            binding.rbEditCharge.isChecked = true
        } else {
            binding.rbEditDropOff.isChecked = true
        }

        loadSlotsForStation(res.stationId, res.reservationDate.split("T").firstOrNull())
    }

    private fun loadSlotsForStation(stationId: String, date: String?) {
        lifecycleScope.launch {
            val result = stationRepository.getSlotsForStation(stationId, date)
            result.onSuccess { slots ->
                slotsList = slots
                val displaySlots = mutableListOf("Keep Current Slot")
                displaySlots.addAll(slots.map {
                    val start = it.startTime.split("T").lastOrNull()?.take(5) ?: it.startTime
                    val end = it.endTime.split("T").lastOrNull()?.take(5) ?: it.endTime
                    "$start - $end (Avail: ${it.availableCapacityKW} kW)"
                })

                val adapter = ArrayAdapter(this@EditBookingActivity,
                    android.R.layout.simple_spinner_dropdown_item, displaySlots)
                binding.spinnerEditSlots.adapter = adapter
            }
        }
    }

    private fun setupListeners() {
        binding.btnSaveUpdate.setOnClickListener {
            performUpdate()
        }

        binding.btnCancelReservation.setOnClickListener {
            showCancelConfirmation()
        }
    }

```

```

    }
}

private fun performUpdate() {
    val res = currentReservation ?: return
    val nic = sessionManager.getUserNic() ?: return

    val energyStr = binding.etEditEnergy.text.toString().trim()
    val energy = energyStr.toDoubleOrNull()
    if (energy == null || energy <= 0) {
        Toast.makeText(this, "Please enter a valid energy amount.",
Toast.LENGTH_SHORT).show()
        return
    }

    val slotPosition = binding.spinnerEditSlots.selectedItemPosition
    val newSlotId = if (slotPosition > 0 && (slotPosition - 1) in slotsList.indices) {
        slotsList[slotPosition - 1].id
    } else null

    val tradeType = if (binding.rbEditDropOff.isChecked) "DropOff" else "Charge"

    val request = UpdateReservationRequest(
        newSlotId = newSlotId,
        newEnergyAmountKW = energy,
        tradeType = tradeType
    )

    binding.progressBar.visibility = View.VISIBLE
    binding.btnSaveUpdate.isEnabled = false

    lifecycleScope.launch {
        val result = reservationRepository.updateReservation(res.id, nic, request)
        binding.progressBar.visibility = View.GONE
        binding.btnSaveUpdate.isEnabled = true

        result.onSuccess { updated ->
            Toast.makeText(this@EditBookingActivity, "Booking updated successfully!",
Toast.LENGTH_SHORT).show()

```

```

        val intent = Intent(this@EditBookingActivity,
BookingSummaryActivity::class.java).apply {
            putExtra("RESERVATION_DATA", updated)
        }
        startActivity(intent)
        finish()
    }.onFailure { error ->
        // Displays server error message (e.g. 12-hour threshold violation) in alert dialog
without crashing
        AlertDialog.Builder(this@EditBookingActivity)
            .setTitle("Update Denied by Server")
            .setMessage(error.message ?: "The server rejected this update. Modifications require
at least 12 hours notice prior to the slot.")
            .setPositiveButton("OK", null)
            .show()
    }
}

private fun showCancelConfirmation() {
    val res = currentReservation ?: return
    val nic = sessionManager.getUserNic() ?: return

    AlertDialog.Builder(this)
        .setTitle("Confirm Cancellation")
        .setMessage("Are you sure you want to cancel this booking? Reservations cannot be
cancelled within 12 hours of the slot start time.")
        .setPositiveButton("Yes, Cancel") { _, _ ->
            executeCancellation(res.id, nic)
        }
        .setNegativeButton("No", null)
        .show()
}

private fun executeCancellation(resId: String, nic: String) {
    binding.progressBar.visibility = View.VISIBLE
    binding.btnCancelReservation.isEnabled = false

    lifecycleScope.launch {
        val result = reservationRepository.cancelReservation(

```

```

        resId,
        CancelReservationRequest(prosumerNIC = nic, reason = "Cancelled via Mobile
Client")
    )
    binding.progressBar.visibility = View.GONE
    binding.btnCancelReservation.isEnabled = true

    result.onSuccess { cancelled ->
        Toast.makeText(this@EditBookingActivity, "Booking cancelled successfully.",
Toast.LENGTH_SHORT).show()
        val intent = Intent(this@EditBookingActivity,
BookingSummaryActivity::class.java).apply {
            putExtra("RESERVATION_DATA", cancelled)
        }
        startActivity(intent)
        finish()
    }.onFailure { error ->
        // Render exact server-side cancellation denial
        AlertDialog.Builder(this@EditBookingActivity)
            .setTitle("Cancellation Denied")
            .setMessage(error.message ?: "The server rejected cancellation because the 12-hour
minimum notice threshold has elapsed.")
            .setPositiveButton("OK", null)
            .show()
    }
}
}
}
}

```

ProsumerDashboardActivity.kt

```

package com.solarix.mobile.ui.prosumer

import android.content.Intent
import android.os.Bundle
import android.widget.Toast
import androidx.appcompat.app.AlertDialog
import androidx.appcompat.app.AppCompatActivity
import androidx.lifecycle.lifecycleScope
import com.solarix.mobile.data.repository.AuthRepository

```

```
import com.solarix.mobile.data.repository.ReservationRepository
import com.solarix.mobile.databinding.ActivityProsumerDashboardBinding
import com.solarix.mobile.ui.auth.LoginActivity
import com.solarix.mobile.ui.map.StationsMapActivity
import com.solarix.mobile.utils.SessionManager
import kotlinx.coroutines.launch
```

```
class ProsumerDashboardActivity : AppCompatActivity() {
```

```
    private lateinit var binding: ActivityProsumerDashboardBinding
    private lateinit var reservationRepository: ReservationRepository
    private lateinit var authRepository: AuthRepository
    private lateinit var sessionManager: SessionManager
```

```
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityProsumerDashboardBinding.inflate(layoutInflater)
        setContentView(binding.root)
```

```
        sessionManager = SessionManager.getInstance(this)
        reservationRepository = ReservationRepository(this)
        authRepository = AuthRepository(this)
```

```
        setupUserHeader()
        setupListeners()
    }
```

```
    override fun onResume() {
        super.onResume()
        loadDashboardMetrics()
    }
```

```
    private fun setupUserHeader() {
        val fullName = sessionManager.getUserFullName() ?: "Prosumer"
        val nic = sessionManager.getUserNic() ?: "Unknown"

        binding.tvWelcomeName.text = "Hello, $fullName"
        binding.tvUserNic.text = "NIC: $nic"
    }
```

```

private fun setupListeners() {
    binding.swipeRefresh.setOnRefreshListener {
        loadDashboardMetrics()
    }

    binding.cardActionBookSlot.setOnClickListener {
        startActivity(Intent(this, CreateBookingActivity::class.java))
    }

    binding.cardActionHistory.setOnClickListener {
        startActivity(Intent(this, BookingHistoryActivity::class.java))
    }

    binding.cardActionMap.setOnClickListener {
        startActivity(Intent(this, StationsMapActivity::class.java))
    }

    binding.cardActionProfile.setOnClickListener {
        startActivity(Intent(this, ProsumerProfileActivity::class.java))
    }

    binding.btnLogout.setOnClickListener {
        showLogoutConfirmation()
    }

    binding.cardPending.setOnClickListener {
        val intent = Intent(this, BookingHistoryActivity::class.java).apply {
            putExtra("FILTER_STATUS", "Pending")
        }
        startActivity(intent)
    }

    binding.cardApproved.setOnClickListener {
        val intent = Intent(this, BookingHistoryActivity::class.java).apply {
            putExtra("FILTER_STATUS", "Approved")
        }
        startActivity(intent)
    }
}

```

```

private fun loadDashboardMetrics() {
    val nic = sessionManager.getUserNic() ?: return
    binding.swipeRefresh.isRefreshing = true

    lifecycleScope.launch {
        val result = reservationRepository.getReservationsByProsumer(nic)
        binding.swipeRefresh.isRefreshing = false

        result.onSuccess { reservations ->
            val pendingCount = reservations.count { it.status.equals("Pending", ignoreCase = true)
        }
            val approvedCount = reservations.count { it.status.equals("Approved", ignoreCase =
true) }

            binding.tvPendingCount.text = pendingCount.toString()
            binding.tvApprovedCount.text = approvedCount.toString()
        }.onFailure { error ->
            Toast.makeText(this@ProsumerDashboardActivity, "Offline mode: Showing cached
data", Toast.LENGTH_SHORT).show()
        }
    }
}

private fun showLogoutConfirmation() {
    AlertDialog.Builder(this)
        .setTitle("Confirm Logout")
        .setMessage("Are you sure you want to end your SolariX session?")
        .setPositiveButton("Logout") { _, _ ->
            lifecycleScope.launch {
                authRepository.logout()
                val intent = Intent(this@ProsumerDashboardActivity, LoginActivity::class.java)
                intent.flags = Intent.FLAG_ACTIVITY_NEW_TASK or
Intent.FLAG_ACTIVITY_CLEAR_TASK
                startActivity(intent)
                finish()
            }
        }
        .setNegativeButton("Cancel", null)
        .show()
}

```

```
}
```

ProsumerProfileActivity.kt

```
package com.solarix.mobile.ui.prosumer

import android.content.Intent
import android.os.Bundle
import android.view.View
import android.widget.Toast
import androidx.appcompat.app.AlertDialog
import androidx.appcompat.app.AppCompatActivity
import androidx.lifecycle.LifecycleScope
import com.solarix.mobile.data.remote.dtos.UpdateProfileRequest
import com.solarix.mobile.data.repository.AuthRepository
import com.solarix.mobile.databinding.ActivityProsumerProfileBinding
import com.solarix.mobile.ui.auth.LoginActivity
import com.solarix.mobile.utils.SessionManager
import kotlinx.coroutines.launch

class ProsumerProfileActivity : AppCompatActivity() {

    private lateinit var binding: ActivityProsumerProfileBinding
    private lateinit var authRepository: AuthRepository
    private lateinit var sessionManager: SessionManager

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityProsumerProfileBinding.inflate(layoutInflater)
        setContentView(binding.root)

        authRepository = AuthRepository(this)
        sessionManager = SessionManager.getInstance(this)

        loadProfile()
        setupListeners()
    }

    private fun loadProfile() {
        val nic = sessionManager.getUserNic() ?: return
    }
}
```

```

binding.progressBar.visibility = View.VISIBLE

binding.etProfileNic.setText(nic)
binding.etProfileEmail.setText(sessionManager.getUserEmail() ?: "")
binding.etProfileFullName.setText(sessionManager.getUserFullName() ?: "")

lifecycleScope.launch {
    val result = authRepository.getProfile(nic)
    binding.progressBar.visibility = View.GONE

    result.onSuccess { user ->
        binding.etProfileFullName.setText(user.fullName)
        binding.etProfileEmail.setText(user.email)
        binding.etProfilePhone.setText(user.phoneNumber)
        binding.etProfileAddress.setText(user.address)
        binding.etProfileCapacity.setText(user.solarCapacityKW.toString())
    }.onFailure {
        // If offline, default values from session are already displayed
    }
}

private fun setupListeners() {
    binding.btnSaveProfile.setOnClickListener {
        saveProfileChanges()
    }

    binding.btnDeactivateAccount.setOnClickListener {
        showDeactivateConfirmation()
    }
}

private fun saveProfileChanges() {
    val nic = sessionManager.getUserNic() ?: return
    val fullName = binding.etProfileFullName.text.toString().trim()
    val phone = binding.etProfilePhone.text.toString().trim()
    val address = binding.etProfileAddress.text.toString().trim()
    val capacityStr = binding.etProfileCapacity.text.toString().trim()
    val capacity = capacityStr.toDoubleOrNull() ?: 5.0

```

```

if (fullName.isBlank()) {
    Toast.makeText(this, "Full name cannot be empty.", Toast.LENGTH_SHORT).show()
    return
}

val request = UpdateProfileRequest(
    fullName = fullName,
    phoneNumber = phone,
    address = address,
    solarCapacityKW = capacity
)

binding.progressBar.visibility = View.VISIBLE
binding.btnSaveProfile.isEnabled = false

lifecycleScope.launch {
    val result = authRepository.updateProfile(nic, request)
    binding.progressBar.visibility = View.GONE
    binding.btnSaveProfile.isEnabled = true

    result.onSuccess { updated ->
        Toast.makeText(this@ProsumerProfileActivity, "Profile updated successfully!",
Toast.LENGTH_SHORT).show()
    }.onFailure { error ->
        AlertDialog.Builder(this@ProsumerProfileActivity)
            .setTitle("Update Failed")
            .setMessage(error.message ?: "Failed to update profile.")
            .setPositiveButton("OK", null)
            .show()
    }
}
}

private fun showDeactivateConfirmation() {
    val nic = sessionManager.getUserNic() ?: return

    AlertDialog.Builder(this)
        .setTitle("Confirm Deactivation Request")
        .setMessage("Are you sure you want to deactivate your prosumer account? You will be
immediately logged out. Reactivation requires Backoffice administrative review.")

```

```

        .setPositiveButton("Yes, Deactivate") { _, _ ->
            executeDeactivation(nic)
        }
        .setNegativeButton("Cancel", null)
        .show()
    }

    private fun executeDeactivation(nic: String) {
        binding.progressBar.visibility = View.VISIBLE
        binding.btnDeactivateAccount.isEnabled = false

        lifecycleScope.launch {
            val result = authRepository.deactivateAccount(nic)
            binding.progressBar.visibility = View.GONE

            result.onSuccess { msg ->
                AlertDialog.Builder(this@ProsumerProfileActivity)
                    .setTitle("Account Deactivated")
                    .setMessage(msg)
                    .setPositiveButton("OK") { _, _ ->
                        val intent = Intent(this@ProsumerProfileActivity, LoginActivity::class.java)
                        intent.flags = Intent.FLAG_ACTIVITY_NEW_TASK or
Intent.FLAG_ACTIVITY_CLEAR_TASK
                        startActivity(intent)
                        finish()
                    }
                    .setCancelable(false)
                    .show()
            }.onFailure { error ->
                binding.btnDeactivateAccount.isEnabled = true
                AlertDialog.Builder(this@ProsumerProfileActivity)
                    .setTitle("Deactivation Failed")
                    .setMessage(error.message ?: "Could not complete account deactivation.")
                    .setPositiveButton("OK", null)
                    .show()
            }
        }
    }
}

```

QRDisplayActivity.kt

```
package com.solarix.mobile.ui.prosumer

import android.os.Bundle
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
import com.solarix.mobile.data.remote.dtos.ReservationResponse
import com.solarix.mobile.databinding.ActivityQrDisplayBinding
import com.solarix.mobile.utils.QRCodeGenerator

class QRDisplayActivity : AppCompatActivity() {

    private lateinit var binding: ActivityQrDisplayBinding
    private var reservation: ReservationResponse? = null

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityQrDisplayBinding.inflate(layoutInflater)
        setContentView(binding.root)

        @Suppress("DEPRECATION")
        reservation = intent.getSerializableExtra("RESERVATION_DATA") as?
        ReservationResponse

        if (reservation == null) {
            Toast.makeText(this, "No booking information found.",
                Toast.LENGTH_SHORT).show()
            finish()
            return
        }

        renderDetails(reservation!!)
        renderQRCode(reservation!!)

        binding.btnClose.setOnClickListener {
            finish()
        }
    }
}
```

```

private fun renderDetails(res: ReservationResponse) {
    val refNumber = if (res.reservationNumber.isNotBlank()) res.reservationNumber else
"#RES-${res.id.takeLast(6).uppercase()}"
    binding.tvBookingRef.text = refNumber

    val stName = if (res.stationName.isNotBlank()) res.stationName else "Station:
${res.stationId}"
    binding.tvQrStation.text = "Hub: $stName"

    val datePart = res.reservationDate.split("T").firstOrNull() ?: res.reservationDate
    val startPart = res.startTime.split("T").lastOrNull()?.take(5) ?: res.startTime
    val endPart = res.endTime.split("T").lastOrNull()?.take(5) ?: res.endTime
    binding.tvQrSlot.text = "Slot: $datePart | $startPart - $endPart"

    binding.tvQrEnergy.text = "Trade: ${String.format("%.1f", res.energyAmountKW)} kW
(${res.tradeType})"
}

private fun renderQRCode(res: ReservationResponse) {
    val token = res.qrCodeToken
    if (token.isBlank()) {
        Toast.makeText(this, "QR Code Token not generated yet for this booking.",
Toast.LENGTH_LONG).show()
        return
    }

    // Generate sharp QR bitmap using native ZXing writer
    val bitmap = QRCodeGenerator.generateQRCodeBitmap(token, 600, 600)
    if (bitmap != null) {
        binding.ivQRCode.setImageBitmap(bitmap)
    } else {
        Toast.makeText(this, "Could not render QR code bitmap.",
Toast.LENGTH_SHORT).show()
    }
}
}

```

BookingsAdapter.kt

```
package com.solarix.mobile.ui.prosumer.adapters
```

```

import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import androidx.core.content.ContextCompat
import androidx.recyclerview.widget.RecyclerView
import com.solarix.mobile.R
import com.solarix.mobile.data.remote.dtos.ReservationResponse
import com.solarix.mobile.databinding.ItemBookingCardBinding

class BookingsAdapter(
    private var bookings: List<ReservationResponse>,
    private val onViewQRClick: (ReservationResponse) -> Unit,
    private val onEditClick: (ReservationResponse) -> Unit
) : RecyclerView.Adapter<BookingsAdapter.BookingViewHolder>() {

    fun updateData(newBookings: List<ReservationResponse>) {
        this.bookings = newBookings
        notifyDataSetChanged()
    }

    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): BookingViewHolder {
        val binding = ItemBookingCardBinding.inflate(
            LayoutInflater.from(parent.context),
            parent,
            false
        )
        return BookingViewHolder(binding)
    }

    override fun onBindViewHolder(holder: BookingViewHolder, position: Int) {
        holder.bind(bookings[position])
    }

    override fun getItemCount(): Int = bookings.size

    inner class BookingViewHolder(private val binding: ItemBookingCardBinding) :
        RecyclerView.ViewHolder(binding.root) {

        fun bind(item: ReservationResponse) {

```

```

val context = binding.root.context

binding.tvCardResNumber.text = if (item.reservationNumber.isNotBlank()) {
    item.reservationNumber
} else {
    "#RES-${item.id.takeLast(6).uppercase()}"
}

binding.tvCardStationName.text = if (item.stationName.isNotBlank()) {
    item.stationName
} else {
    "Solar Hub: ${item.stationId}"
}

val datePart = item.reservationDate.split("T").firstOrNull() ?: item.reservationDate
val startPart = item.startTime.split("T").lastOrNull()?.take(5) ?: item.startTime
val endPart = item.endTime.split("T").lastOrNull()?.take(5) ?: item.endTime
binding.tvCardTiming.text = "$datePart | $startPart - $endPart"

binding.tvCardEnergy.text = String.format("%.1f kW", item.energyAmountKW)
binding.tvCardTradeType.text = item.tradeType

// Status Styling
binding.tvCardStatus.text = item.status
when (item.status.lowercase()) {
    "approved" -> {
        binding.tvCardStatus.setBackgroundResource(R.drawable.bg_badge_approved)
        binding.tvCardStatus.setTextColor(ContextCompat.getColor(context,
R.color.status_approved))
        binding.btnCardQR.visibility = View.VISIBLE
        binding.btnCardEdit.visibility = View.VISIBLE
    }
    "pending" -> {
        binding.tvCardStatus.setBackgroundResource(R.drawable.bg_badge_pending)
        binding.tvCardStatus.setTextColor(ContextCompat.getColor(context,
R.color.status_pending))
        binding.btnCardQR.visibility = View.GONE
        binding.btnCardEdit.visibility = View.VISIBLE
    }
    "completed" -> {

```

```

        binding.tvCardStatus.setBackgroundResource(R.drawable.bg_badge_completed)
        binding.tvCardStatus.setTextColor(ContextCompat.getColor(context,
R.color.status_completed))
        binding.btnCardQR.visibility = View.GONE
        binding.btnCardEdit.visibility = View.GONE
    }
    "cancelled" -> {
        binding.tvCardStatus.setBackgroundResource(R.drawable.bg_badge_cancelled)
        binding.tvCardStatus.setTextColor(ContextCompat.getColor(context,
R.color.status_cancelled))
        binding.btnCardQR.visibility = View.GONE
        binding.btnCardEdit.visibility = View.GONE
    }
    else -> {
        binding.tvCardStatus.setBackgroundResource(R.drawable.bg_badge_pending)
        binding.tvCardStatus.setTextColor(ContextCompat.getColor(context,
R.color.status_pending))
        binding.btnCardQR.visibility = View.GONE
        binding.btnCardEdit.visibility = View.VISIBLE
    }
}

binding.btnCardQR.setOnClickListener { onViewQRClick(item) }
binding.btnCardEdit.setOnClickListener { onEditClick(item) }
}
}
}

```

AppDatabase.kt

```
package com.solarix.mobile.data.local
```

```

import android.content.Context
import androidx.room.Database
import androidx.room.Room
import androidx.room.RoomDatabase
import com.solarix.mobile.data.local.entities.CachedBookingEntity
import com.solarix.mobile.data.local.entities.UserEntity

```

```

@Database(entities = [UserEntity::class, CachedBookingEntity::class], version = 1,
exportSchema = false)
abstract class AppDatabase : RoomDatabase() {
    abstract fun userDao(): UserDao
    abstract fun bookingDao(): BookingDao

    companion object {
        @Volatile
        private var INSTANCE: AppDatabase? = null

        fun getDatabase(context: Context): AppDatabase {
            return INSTANCE ?: synchronized(this) {
                val instance = Room.databaseBuilder(
                    context.applicationContext,
                    AppDatabase::class.java,
                    "solarix_database.db"
                ).fallbackToDestructiveMigration().build()
                INSTANCE = instance
                instance
            }
        }
    }
}

```

BookingDao.kt

```

package com.solarix.mobile.data.local

import androidx.room.Dao
import androidx.room.Insert
import androidx.room.OnConflictStrategy
import androidx.room.Query
import com.solarix.mobile.data.local.entities.CachedBookingEntity

@Dao
interface BookingDao {
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertOrUpdate(booking: CachedBookingEntity)

    @Insert(onConflict = OnConflictStrategy.REPLACE)

```

```

suspend fun insertAll(bookings: List<CachedBookingEntity>)

@Query("SELECT * FROM cached_bookings WHERE prosumerNIC = :nic ORDER BY
reservationDate DESC")
suspend fun getBookingsByProsumer(nic: String): List<CachedBookingEntity>

@Query("SELECT * FROM cached_bookings WHERE id = :id LIMIT 1")
suspend fun getBookingById(id: String): CachedBookingEntity?

@Query("DELETE FROM cached_bookings WHERE prosumerNIC = :nic")
suspend fun clearBookingsForProsumer(nic: String)

@Query("DELETE FROM cached_bookings")
suspend fun clearAll()
}

```

UserDao.kt

```

package com.solarix.mobile.data.local

import androidx.room.Dao
import androidx.room.Insert
import androidx.room.OnConflictStrategy
import androidx.room.Query
import androidx.room.Update
import com.solarix.mobile.data.local.entities.UserEntity

@Dao
interface UserDao {
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertOrUpdate(user: UserEntity)

    @Update
    suspend fun update(user: UserEntity)

    @Query("SELECT * FROM users WHERE nic = :nic LIMIT 1")
    suspend fun getUserByNic(nic: String): UserEntity?

    @Query("SELECT * FROM users ORDER BY lastLoginAt DESC LIMIT 1")
    suspend fun getActiveUser(): UserEntity?
}

```

```

    @Query("DELETE FROM users WHERE nic = :nic")
    suspend fun deleteUser(nic: String)

    @Query("DELETE FROM users")
    suspend fun clearAll()
}

```

CachedBookingEntity.kt

```

package com.solarix.mobile.data.local.entities

import androidx.room.Entity
import androidx.room.PrimaryKey

/**
 * Entity representing local cached reservations for offline viewing and quick access.
 */
@Entity(tableName = "cached_bookings")
data class CachedBookingEntity(
    @PrimaryKey
    val id: String,
    val reservationNumber: String,
    val prosumerNIC: String,
    val stationId: String,
    val stationName: String,
    val slotId: String,
    val reservationDate: String,
    val startTime: String,
    val endTime: String,
    val energyAmountKW: Double,
    val tradeType: String,
    val status: String,
    val qrCodeToken: String,
    val updatedAt: Long = System.currentTimeMillis()
)

```

UserEntity.kt

```

package com.solarix.mobile.data.local.entities

```

```

import androidx.room.Entity
import androidx.room.PrimaryKey

/**
 * Entity representing the local cached user session and profile.
 * NIC serves as the immutable primary key as mandated by course architecture.
 */
@Entity(tableName = "users")
data class UserEntity(
    @PrimaryKey
    val nic: String,
    val fullName: String,
    val email: String,
    val role: String,
    val status: String,
    val token: String,
    val phoneNumber: String? = null,
    val address: String? = null,
    val solarCapacityKW: Double = 0.0,
    val lastLoginAt: Long = System.currentTimeMillis()
)

```

AuthInterceptor.kt

```

package com.solarix.mobile.data.remote

import android.content.Context
import com.solarix.mobile.utils.SessionManager
import okhttp3.Interceptor
import okhttp3.Response

/**
 * OkHttp Interceptor that automatically attaches the JWT Bearer token from SQLite session.
 */
class AuthInterceptor(private val context: Context) : Interceptor {
    override fun intercept(chain: Interceptor.Chain): Response {
        val original = chain.request()
        val token = SessionManager.getInstance(context).getToken()
    }
}

```

```

        val requestBuilder = original.newBuilder()
            .header("Accept", "application/json")

        if (!token.isNullOrBlank()) {
            requestBuilder.header("Authorization", "Bearer $token")
        }

        return chain.proceed(requestBuilder.build())
    }
}

```

AuthDtos.kt

```
package com.solarix.mobile.data.remote.dtos
```

```
import com.google.gson.annotations.SerializedName
```

```

data class LoginRequest(
    @SerializedName("identifier") val identifier: String,
    @SerializedName("password") val password: String
)

```

```

data class LoginResponse(
    @SerializedName("nic") val nic: String,
    @SerializedName("fullName") val fullName: String,
    @SerializedName("email") val email: String,
    @SerializedName("role") val role: String,
    @SerializedName("status") val status: String,
    @SerializedName("token") val token: String
)

```

```

data class RegisterProsumerRequest(
    @SerializedName("nic") val nic: String,
    @SerializedName("fullName") val fullName: String,
    @SerializedName("email") val email: String,
    @SerializedName("phoneNumber") val phoneNumber: String,
    @SerializedName("password") val password: String,
    @SerializedName("address") val address: String,
    @SerializedName("solarCapacityKW") val solarCapacityKW: Double = 5.0
)

```

```
data class UpdateProfileRequest(
    @SerializedName("fullName") val fullName: String,
    @SerializedName("phoneNumber") val phoneNumber: String,
    @SerializedName("address") val address: String,
    @SerializedName("solarCapacityKW") val solarCapacityKW: Double
)
```

```
data class UserResponse(
    @SerializedName("id") val id: String,
    @SerializedName("nic") val nic: String,
    @SerializedName("fullName") val fullName: String,
    @SerializedName("email") val email: String,
    @SerializedName("phoneNumber") val phoneNumber: String,
    @SerializedName("role") val role: String,
    @SerializedName("status") val status: String,
    @SerializedName("address") val address: String,
    @SerializedName("solarCapacityKW") val solarCapacityKW: Double,
    @SerializedName("createdAt") val createdAt: String?
)
```

```
data class ApiResponse<T>(
    @SerializedName("message") val message: String? = null,
    @SerializedName("error") val error: String? = null,
    @SerializedName("data") val data: T? = null
)
```

ReservationDtos.kt

```
package com.solarix.mobile.data.remote.dtos
```

```
import com.google.gson.annotations.SerializedName
import java.io.Serializable
```

```
data class CreateReservationRequest(
    @SerializedName("prosumerNIC") val prosumerNIC: String,
    @SerializedName("stationId") val stationId: String,
    @SerializedName("slotId") val slotId: String,
    @SerializedName("energyAmountKW") val energyAmountKW: Double,
    @SerializedName("tradeType") val tradeType: String = "DropOff"
)
```

)

```
data class UpdateReservationRequest(  
    @SerializedName("newSlotId") val newSlotId: String?,  
    @SerializedName("newEnergyAmountKW") val newEnergyAmountKW: Double?,  
    @SerializedName("tradeType") val tradeType: String?  
)
```

```
data class CancelReservationRequest(  
    @SerializedName("prosumerNIC") val prosumerNIC: String,  
    @SerializedName("reason") val reason: String = "Cancelled by user"  
)
```

```
data class VerifyAndCompleteRequest(  
    @SerializedName("qrToken") val qrToken: String,  
    @SerializedName("operatorId") val operatorId: String,  
    @SerializedName("energyDeliveredKWh") val energyDeliveredKWh: Double,  
    @SerializedName("notes") val notes: String?  
)
```

```
data class ReservationResponse(  
    @SerializedName("id") val id: String,  
    @SerializedName("reservationNumber") val reservationNumber: String,  
    @SerializedName("prosumerNIC") val prosumerNIC: String,  
    @SerializedName("stationId") val stationId: String,  
    @SerializedName("stationName") val stationName: String,  
    @SerializedName("slotId") val slotId: String,  
    @SerializedName("reservationDate") val reservationDate: String,  
    @SerializedName("startTime") val startTime: String,  
    @SerializedName("endTime") val endTime: String,  
    @SerializedName("energyAmountKW") val energyAmountKW: Double,  
    @SerializedName("tradeType") val tradeType: String,  
    @SerializedName("status") val status: String,  
    @SerializedName("qrCodeToken") val qrCodeToken: String,  
    @SerializedName("cancellationReason") val cancellationReason: String?,  
    @SerializedName("cancelledAt") val cancelledAt: String?,  
    @SerializedName("finalizedByOperatorNIC") val finalizedByOperatorNIC: String?,  
    @SerializedName("finalizedAt") val finalizedAt: String?,  
    @SerializedName("operatorNotes") val operatorNotes: String?,  
    @SerializedName("createdAt") val createdAt: String?
```

```
) : Serializable
```

```
data class DashboardSummaryResponse(  
    @SerializedName("totalStations") val totalStations: Int,  
    @SerializedName("activeStations") val activeStations: Int,  
    @SerializedName("totalProsumers") val totalProsumers: Int,  
    @SerializedName("pendingProsumerApprovals") val pendingProsumerApprovals: Int,  
    @SerializedName("pendingReservationsCount") val pendingReservationsCount: Int,  
    @SerializedName("approvedFutureReservationsCount") val  
approvedFutureReservationsCount: Int,  
    @SerializedName("completedReservationsCount") val completedReservationsCount: Int,  
    @SerializedName("cancelledReservationsCount") val cancelledReservationsCount: Int,  
    @SerializedName("totalEnergyTradedKW") val totalEnergyTradedKW: Double  
)
```

StationDtos.kt

```
package com.solarix.mobile.data.remote.dtos
```

```
import com.google.gson.annotations.SerializedName
```

```
data class StationResponse(  
    @SerializedName("id") val id: String,  
    @SerializedName("stationCode") val stationCode: String,  
    @SerializedName("stationName") val stationName: String,  
    @SerializedName("locationName") val locationName: String,  
    @SerializedName("latitude") val latitude: Double,  
    @SerializedName("longitude") val longitude: Double,  
    @SerializedName("capacityKWh") val capacityKWh: Double,  
    @SerializedName("totalBatterySlots") val totalBatterySlots: Int,  
    @SerializedName("availableBatterySlots") val availableBatterySlots: Int,  
    @SerializedName("isActive") val isActive: Boolean,  
    @SerializedName("operationalHours") val operationalHours: String?,  
    @SerializedName("contactNumber") val contactNumber: String?,  
    @SerializedName("createdAt") val createdAt: String?  
)
```

```
data class SlotResponse(  
    @SerializedName("id") val id: String,  
    @SerializedName("stationId") val stationId: String,
```

```

@SerializedName("slotDate") val slotDate: String,
@SerializedName("startTime") val startTime: String,
@SerializedName("endTime") val endTime: String,
@SerializedName("maxCapacityKW") val maxCapacityKW: Double,
@SerializedName("availableCapacityKW") val availableCapacityKW: Double,
@SerializedName("status") val status: String
)

```

ReservationRepository.kt

```

package com.solarix.mobile.data.repository

import android.content.Context
import com.solarix.mobile.data.local.AppDatabase
import com.solarix.mobile.data.local.entities.CachedBookingEntity
import com.solarix.mobile.data.remote.RetrofitClient
import com.solarix.mobile.data.remote.dtos.*
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.withContext

class ReservationRepository(context: Context) {
    private val api = RetrofitClient.getService(context)
    private val db = AppDatabase.getDatabase(context)

    suspend fun createReservation(request: CreateReservationRequest):
    Result<ReservationResponse> =
        withContext(Dispatchers.IO) {
            try {
                val response = api.createReservation(request)
                if (response.isSuccessful && response.body() != null) {
                    val res = response.body()!!
                    cacheReservation(res)
                    Result.success(res)
                } else {
                    val msg = RetrofitClient.parseErrorMessage(response)
                    Result.failure(Exception(msg))
                }
            } catch (e: Exception) {
                Result.failure(e)
            }
        }
}

```

```

    }

suspend fun updateReservation(
    id: String,
    prosumerNic: String,
    request: UpdateReservationRequest
): Result<ReservationResponse> = withContext(Dispatchers.IO) {
    try {
        val response = api.updateReservation(id, prosumerNic, request)
        if (response.isSuccessful && response.body() != null) {
            val res = response.body()!!
            cacheReservation(res)
            Result.success(res)
        } else {
            val msg = RetrofitClient.parseErrorMessage(response)
            Result.failure(Exception(msg))
        }
    } catch (e: Exception) {
        Result.failure(e)
    }
}

```

```

suspend fun cancelReservation(
    id: String,
    request: CancelReservationRequest
): Result<ReservationResponse> = withContext(Dispatchers.IO) {
    try {
        val response = api.cancelReservation(id, request)
        if (response.isSuccessful && response.body() != null) {
            val res = response.body()!!
            cacheReservation(res)
            Result.success(res)
        } else {
            val msg = RetrofitClient.parseErrorMessage(response)
            Result.failure(Exception(msg))
        }
    } catch (e: Exception) {
        Result.failure(e)
    }
}

```

```

suspend fun getReservationById(id: String): Result<ReservationResponse> =
    withContext(Dispatchers.IO) {
        try {
            val response = api.getReservationById(id)
            if (response.isSuccessful && response.body() != null) {
                val res = response.body()!!
                cacheReservation(res)
                Result.success(res)
            } else {
                val msg = RetrofitClient.parseErrorMessage(response)
                Result.failure(Exception(msg))
            }
        } catch (e: Exception) {
            Result.failure(e)
        }
    }
}

```

```

suspend fun getReservationsByProsumer(nic: String): Result<List<ReservationResponse>> =
    withContext(Dispatchers.IO) {
        try {
            val response = api.getReservationsByProsumer(nic)
            if (response.isSuccessful && response.body() != null) {
                val list = response.body()!!
                // Cache in Room SQLite
                val entities = list.map { it.toCachedEntity() }
                db.bookingDao().clearBookingsForProsumer(nic)
                db.bookingDao().insertAll(entities)
                Result.success(list)
            } else {
                val cached = db.bookingDao().getBookingsByProsumer(nic)
                if (cached.isNotEmpty()) {
                    Result.success(cached.map { it.toResponse() })
                } else {
                    val msg = RetrofitClient.parseErrorMessage(response)
                    Result.failure(Exception(msg))
                }
            }
        } catch (e: Exception) {
            // Fallback to local Room SQLite cache if offline
        }
    }
}

```

```

        val cached = db.bookingDao().getBookingsByProsumer(nic)
        if (cached.isNotEmpty()) {
            Result.success(cached.map { it.toResponse() })
        } else {
            Result.failure(e)
        }
    }
}

```

```

suspend fun getDashboardSummary(): Result<DashboardSummaryResponse> =
    withContext(Dispatchers.IO) {
        try {
            val response = api.getDashboardSummary()
            if (response.isSuccessful && response.body() != null) {
                Result.success(response.body()!!)
            } else {
                val msg = RetrofitClient.parseErrorMessage(response)
                Result.failure(Exception(msg))
            }
        } catch (e: Exception) {
            Result.failure(e)
        }
    }
}

```

```

suspend fun verifyAndComplete(request: VerifyAndCompleteRequest):
Result<ReservationResponse> =
    withContext(Dispatchers.IO) {
        try {
            val response = api.verifyAndComplete(request)
            if (response.isSuccessful && response.body() != null) {
                val res = response.body()!!
                cacheReservation(res)
                Result.success(res)
            } else {
                val msg = RetrofitClient.parseErrorMessage(response)
                Result.failure(Exception(msg))
            }
        } catch (e: Exception) {
            Result.failure(e)
        }
    }
}

```

```

    }

private suspend fun cacheReservation(res: ReservationResponse) {
    db.bookingDao().insertOrUpdate(res.toCachedEntity())
}

private fun ReservationResponse.toCachedEntity(): CachedBookingEntity {
    return CachedBookingEntity(
        id = this.id,
        reservationNumber = this.reservationNumber,
        prosumerNIC = this.prosumerNIC,
        stationId = this.stationId,
        stationName = this.stationName,
        slotId = this.slotId,
        reservationDate = this.reservationDate,
        startTime = this.startTime,
        endTime = this.endTime,
        energyAmountKW = this.energyAmountKW,
        tradeType = this.tradeType,
        status = this.status,
        qrCodeToken = this.qrCodeToken
    )
}

private fun CachedBookingEntity.toResponse(): ReservationResponse {
    return ReservationResponse(
        id = this.id,
        reservationNumber = this.reservationNumber,
        prosumerNIC = this.prosumerNIC,
        stationId = this.stationId,
        stationName = this.stationName,
        slotId = this.slotId,
        reservationDate = this.reservationDate,
        startTime = this.startTime,
        endTime = this.endTime,
        energyAmountKW = this.energyAmountKW,
        tradeType = this.tradeType,
        status = this.status,
        qrCodeToken = this.qrCodeToken,
        cancellationReason = null,
    )
}

```

```

        cancelledAt = null,
        finalizedByOperatorNIC = null,
        finalizedAt = null,
        operatorNotes = null,
        createdAt = null
    )
}
}

```

StationRepository.kt

```
package com.solarix.mobile.data.repository
```

```

import android.content.Context
import com.solarix.mobile.data.remote.RetrofitClient
import com.solarix.mobile.data.remote.dtos.SlotResponse
import com.solarix.mobile.data.remote.dtos.StationResponse
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.withContext

```

```

class StationRepository(context: Context) {
    private val api = RetrofitClient.getService(context)

    suspend fun getStations(activeOnly: Boolean? = true): Result<List<StationResponse>> =
        withContext(Dispatchers.IO) {
            try {
                val response = api.getStations(activeOnly)
                if (response.isSuccessful && response.body() != null) {
                    Result.success(response.body()!!)
                } else {
                    val msg = RetrofitClient.parseErrorMessage(response)
                    Result.failure(Exception(msg))
                }
            } catch (e: Exception) {
                Result.failure(e)
            }
        }

    suspend fun getStationById(id: String): Result<StationResponse> =
        withContext(Dispatchers.IO) {

```

```

    try {
        val response = api.getStationById(id)
        if (response.isSuccessful && response.body() != null) {
            Result.success(response.body()!!)
        } else {
            val msg = RetrofitClient.parseErrorMessage(response)
            Result.failure(Exception(msg))
        }
    } catch (e: Exception) {
        Result.failure(e)
    }
}

suspend fun getSlotsForStation(stationId: String, date: String? = null):
Result<List<SlotResponse>> =
    withContext(Dispatchers.IO) {
        try {
            val response = api.getSlotsForStation(stationId, date)
            if (response.isSuccessful && response.body() != null) {
                Result.success(response.body()!!)
            } else {
                val msg = RetrofitClient.parseErrorMessage(response)
                Result.failure(Exception(msg))
            }
        } catch (e: Exception) {
            Result.failure(e)
        }
    }
}

```

NetworkUtils.kt

```

package com.solarix.mobile.utils

import android.content.Context
import android.net.ConnectivityManager
import android.net.NetworkCapabilities
import android.os.Build

/**

```

```

* Utility to check active network connectivity across Android versions.
*/
object NetworkUtils {

    fun isNetworkAvailable(context: Context): Boolean {
        val connectivityManager =
            context.getSystemService(Context.CONNECTIVITY_SERVICE) as?
ConnectivityManager
            ?: return false

        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.M) {
            val activeNetwork = connectivityManager.activeNetwork ?: return false
            val capabilities =
                connectivityManager.getNetworkCapabilities(activeNetwork) ?: return false
            return capabilities.hasCapability(NetworkCapabilities.NET_CAPABILITY_INTERNET)
&&
capabilities.hasCapability(NetworkCapabilities.NET_CAPABILITY_VALIDATED)
        } else {
            @Suppress("DEPRECATION")
            val networkInfo = connectivityManager.activeNetworkInfo
            @Suppress("DEPRECATION")
            return networkInfo != null && networkInfo.isConnected
        }
    }
}

```

QRCodeGenerator.kt

```

package com.solarix.mobile.utils

import android.graphics.Bitmap
import android.graphics.Color
import com.google.zxing.BarcodeFormat
import com.google.zxing.EncodeHintType
import com.google.zxing.qrcode.QRCodeWriter
import com.google.zxing.qrcode.decoder.ErrorCorrectionLevel
import java.util.EnumMap

/**

```

```
* Pure native ZXing bitmap generator for rendering high-contrast, tamper-proof QR codes.  
*/
```

```
object QRCodeGenerator {
```

```
    fun generateQRCodeBitmap(  
        content: String,
```

```
        width: Int = 600,
```

```
        height: Int = 600
```

```
    ): Bitmap? {
```

```
        if (content.isBlank()) return null
```

```
        return try {
```

```
            val hints = EnumMap<EncodeHintType, Any>(EncodeHintType::class.java).apply {
```

```
                put(EncodeHintType.CHARACTER_SET, "UTF-8")
```

```
                put(EncodeHintType.ERROR_CORRECTION, ErrorCorrectionLevel.H)
```

```
                put(EncodeHintType.MARGIN, 2)
```

```
            }
```

```
            val bitMatrix = QRCodeWriter().encode(  
                content,
```

```
                BarcodeFormat.QR_CODE,
```

```
                width,
```

```
                height,
```

```
                hints
```

```
            )
```

```
            val matrixWidth = bitMatrix.width
```

```
            val matrixHeight = bitMatrix.height
```

```
            val pixels = IntArray(matrixWidth * matrixHeight)
```

```
            for (y in 0 until matrixHeight) {
```

```
                val offset = y * matrixWidth
```

```
                for (x in 0 until matrixWidth) {
```

```
                    pixels[offset + x] = if (bitMatrix.get(x, y)) Color.BLACK else Color.WHITE
```

```
                }
```

```
            }
```

```
            val bitmap = Bitmap.createBitmap(matrixWidth, matrixHeight,  
                Bitmap.Config.ARGB_8888)
```

```
            bitmap.setPixels(pixels, 0, matrixWidth, 0, 0, matrixWidth, matrixHeight)
```

```

        bitmap
    } catch (e: Exception) {
        e.printStackTrace()
        null
    }
}
}
}

```

SessionManager.kt

```
package com.solarix.mobile.utils
```

```

import android.content.Context
import android.content.SharedPreferences
import com.solarix.mobile.data.local.AppDatabase
import com.solarix.mobile.data.local.entities.UserEntity
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.withContext

```

```
/**
```

```
 * Manages the current user session state, bridging local SQLite persistence (Room)
```

```
 * and lightweight credentials for fast interceptor access.
```

```
*/
```

```
class SessionManager(private val context: Context) {
```

```
    private val prefs: SharedPreferences =
```

```
        context.getSharedPreferences("solarix_session_prefs", Context.MODE_PRIVATE)
```

```
    companion object {
```

```
        private const val KEY_TOKEN = "jwt_token"
```

```
        private const val KEY_NIC = "user_nic"
```

```
        private const val KEY_ROLE = "user_role"
```

```
        private const val KEY_NAME = "user_full_name"
```

```
        private const val KEY_STATUS = "user_status"
```

```
        private const val KEY_EMAIL = "user_email"
```

```
        @Volatile
```

```
        private var INSTANCE: SessionManager? = null
```

```
        fun getInstance(context: Context): SessionManager {
```

```
            return INSTANCE ?: synchronized(this) {
```

```

        INSTANCE ?: SessionManager(context.applicationContext).also { INSTANCE = it }
    }
}

```

```

fun saveSession(
    token: String,
    nic: String,
    role: String,
    fullName: String,
    email: String,
    status: String
) {
    prefs.edit()
        .putString(KEY_TOKEN, token)
        .putString(KEY_NIC, nic)
        .putString(KEY_ROLE, role)
        .putString(KEY_NAME, fullName)
        .putString(KEY_EMAIL, email)
        .putString(KEY_STATUS, status)
        .apply()
}

```

```

suspend fun saveFullUserSession(user: UserEntity) {
    saveSession(
        token = user.token,
        nic = user.nic,
        role = user.role,
        fullName = user.fullName,
        email = user.email,
        status = user.status
    )
    withContext(Dispatchers.IO) {
        AppDatabase.getDatabase(context).userDao().insertOrUpdate(user)
    }
}

```

```

fun getToken(): String? = prefs.getString(KEY_TOKEN, null)

```

```

fun getUserNic(): String? = prefs.getString(KEY_NIC, null)

```

```

fun getUserRole(): String? = prefs.getString(KEY_ROLE, null)

fun getUserFullName(): String? = prefs.getString(KEY_NAME, null)

fun getUserEmail(): String? = prefs.getString(KEY_EMAIL, null)

fun getUserStatus(): String? = prefs.getString(KEY_STATUS, null)

fun isLoggedIn(): Boolean = !getToken().isNullOrBlank()

suspend fun clearSession() {
    val currentNic = getUserNic()
    prefs.edit().clear().apply()
    withContext(Dispatchers.IO) {
        if (!currentNic.isNullOrBlank()) {
            AppDatabase.getDatabase(context).bookingDao().clearBookingsForProsumer(currentNic)
        }
        AppDatabase.getDatabase(context).userDao().clearAll()
    }
}

```

Program.cs

```

//
=====

// File: Program.cs
// Project: SolariX - Smart Solar Microgrid Trading System
// Description: Application entry point configuring dependency injection, MongoDB services,
// CORS, Swagger UI, and IIS hosting.
// Module: SE4040 Enterprise Application Development
//
=====

using System.Text.Json.Serialization;
using Microsoft.OpenApi.Models;

```

```

using SolariX.Api.Data;
using SolariX.Api.Services;

var builder = WebApplication.CreateBuilder(args);

// Configure strongly-typed MongoDB settings
builder.Services.Configure<MongoDbSettings>(
    builder.Configuration.GetSection(MongoDbSettings.SectionName));

// Register MongoDB context and domain services
builder.Services.AddSingleton<IMongoDbContext, MongoDbContext>();
builder.Services.AddSingleton<IQRService, QRService>();
builder.Services.AddScoped<IUserService, UserService>();
builder.Services.AddScoped<IStationService, StationService>();
builder.Services.AddScoped<IBookingService, BookingService>();

// Configure controllers with JSON StringEnumConverter
builder.Services.AddControllers()
    .AddJsonOptions(options =>
    {
        options.JsonSerializerOptions.Converters.Add(new JsonStringEnumConverter());
        options.JsonSerializerOptions.DefaultIgnoreCondition =
JsonIgnoreCondition.WhenWritingNull;
    });

// Configure CORS for web portal and mobile client access
builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowAll", policy =>
    {
        policy.AllowAnyOrigin()
            .AllowAnyMethod()
            .AllowAnyHeader();
    });
});

// Configure Swagger / OpenAPI documentation
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen(c =>
{

```

```

c.SwaggerDoc("v1", new OpenApiInfo
{
    Title = "SolariX Smart Solar Microgrid Web API (The FAT Service)",
    Version = "v1",
    Description = "Centralized FAT service API for Backoffice, Grid Operators, and Prosumers
managing solar trading, 7-day future booking windows, and 12-hour cancellation thresholds.",
    Contact = new OpenApiContact
    {
        Name = "SolariX Engineering Team",
        Email = "support@solarix.com"
    }
});
});

var app = builder.Build();

// Enable Swagger UI across all environments for verification and testing
app.UseSwagger();
app.UseSwaggerUI(c =>
{
    c.SwaggerEndpoint("/swagger/v1/swagger.json", "SolariX FAT Service API v1");
    c.RoutePrefix = string.Empty; // Serves Swagger UI at root URL (e.g. http://localhost:5000/)
});

app.UseCors("AllowAll");
app.UseAuthorization();
app.MapControllers();

// Execute automated initial seed for users, stations, and slots
using (var scope = app.Services.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<IMongoDbContext>();
    await DbSeeder.SeedAsync(context);
}

app.Run();

```

ReservationsController.cs

```
//
=====

// File: ReservationsController.cs
// Project: SolariX - Smart Solar Microgrid Trading System
// Description: API controller for energy reservations, enforcing the 7-day booking window, 12-
// hour cancellation/update thresholds, and operator QR completion.
// Module: SE4040 Enterprise Application Development
//
=====

using Microsoft.AspNetCore.Mvc;
using SolariX.Api.DTOs;
using SolariX.Api.Services;

namespace SolariX.Api.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    [Produces("application/json")]
    public class ReservationsController : ControllerBase
    {
        private readonly IBookingService _bookingService;

        public ReservationsController(IBookingService bookingService)
        {
            // Inline: Injects the central FAT booking service to coordinate reservations and enforce
            validation rules.
            _bookingService = bookingService;
        }

        /// <summary>
        /// Creates a new energy reservation. Enforces 7-day future window validation.
        /// </summary>
        [HttpPost]
        [ProducesResponseType(typeof(ReservationResponse), StatusCodes.Status201Created)]
        [ProducesResponseType(StatusCodes.Status400BadRequest)]
        [ProducesResponseType(StatusCodes.Status404NotFound)]
        public async Task<ActionResult> Create([FromBody] CreateReservationRequest request)
```

```

    {
        // Inline: Calls the FAT service to validate the 7-day scheduling window, confirm
        // prosumer status and slot availability, and persist reservation with secure QR payload.
        if (!ModelState.IsValid)
        {
            return BadRequest(ModelState);
        }

        try
        {
            var response = await _bookingService.CreateReservationAsync(request);
            return CreatedAtAction(nameof(GetById), new { id = response.Id }, response);
        }
        catch (ArgumentOutOfRangeException ex)
        {
            return BadRequest(new { error = "ValidationError", message = ex.Message });
        }
        catch (InvalidOperationException ex)
        {
            return BadRequest(new { error = "InvalidOperation", message = ex.Message });
        }
        catch (KeyNotFoundException ex)
        {
            return NotFound(new { error = "NotFound", message = ex.Message });
        }
        catch (Exception ex)
        {
            return StatusCode(StatusCodes.Status500InternalServerError, new { message =
ex.Message });
        }
    }

    /// <summary>
    /// Modifies/updates an existing reservation. Enforces the 12-hour notice threshold.
    /// </summary>
    [HttpPut("{id}")]
    [ProducesResponseType(typeof(ReservationResponse), StatusCodes.Status200OK)]
    [ProducesResponseType(StatusCodes.Status400BadRequest)]
    [ProducesResponseType(StatusCodes.Status404NotFound)]

```

```

    public async Task<IActionResult> Update(string id, [FromQuery] string prosumerNic,
[FromBody] UpdateReservationRequest request)
    {
        // Inline: Enforces at least 12 hours' notice prior to slot start time before allowing
modification.
        if (string.IsNullOrEmpty(prosumerNic))
        {
            return BadRequest(new { message = "prosumerNic query parameter is required." });
        }

        try
        {
            var updated = await _bookingService.UpdateReservationAsync(id, prosumerNic,
request);
            return Ok(updated);
        }
        catch (InvalidOperationException ex)
        {
            return BadRequest(new { error = "ThresholdError", message = ex.Message });
        }
        catch (ArgumentOutOfRangeException ex)
        {
            return BadRequest(new { error = "ValidationError", message = ex.Message });
        }
        catch (KeyNotFoundException ex)
        {
            return NotFound(new { error = "NotFound", message = ex.Message });
        }
    }

    /// <summary>
    /// Cancels an existing reservation. Enforces the minimum 12-hour notice threshold.
    /// </summary>
    [HttpPost("{id}/cancel")]
    [ProducesResponseType(typeof(ReservationResponse), StatusCodes.Status200OK)]
    [ProducesResponseType(StatusCodes.Status400BadRequest)]
    [ProducesResponseType(StatusCodes.Status404NotFound)]
    public async Task<IActionResult> Cancel(string id, [FromBody]
CancelReservationRequest request)
    {

```

```

// Inline: Validates that cancellation request satisfies the mandatory 12-hour threshold.
if (!ModelState.IsValid)
{
    return BadRequest(ModelState);
}

try
{
    var cancelled = await _bookingService.CancelReservationAsync(id, request);
    return Ok(cancelled);
}
catch (InvalidOperationException ex)
{
    return BadRequest(new { error = "CancellationWindowExpired", message =
ex.Message });
}
catch (KeyNotFoundException ex)
{
    return NotFound(new { error = "NotFound", message = ex.Message });
}
}

```

```

/// <summary>
/// Grid Operator mode: Verifies QR token, confirms Approved status, validates
GridOperator role, and completes trade.
/// </summary>
[HttpPost("verify-and-complete")]
[ProducesResponseType(typeof(ReservationResponse), StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status400BadRequest)]
[ProducesResponseType(StatusCodes.Status401Unauthorized)]
[ProducesResponseType(StatusCodes.Status403Forbidden)]
[ProducesResponseType(StatusCodes.Status404NotFound)]
public async Task<IActionResult> VerifyAndComplete([FromBody]
VerifyAndCompleteRequest request)
{
    // Inline: Verifies QR token signature, confirms reservation is currently in Approved
status, validates GridOperator role, sets status to Completed, and timestamps completion.
    if (!ModelState.IsValid)
    {
        return BadRequest(ModelState);
    }
}

```

```

    }

    try
    {
        var completed = await _bookingService.VerifyAndCompleteAsync(request);
        return Ok(completed);
    }
    catch (ArgumentException ex)
    {
        return BadRequest(new { error = "InvalidQRCode", message = ex.Message });
    }
    catch (UnauthorizedAccessException ex)
    {
        return StatusCode(StatusCodes.Status403Forbidden, new { error = "Forbidden",
message = ex.Message });
    }
    catch (InvalidOperationException ex)
    {
        return BadRequest(new { error = "InvalidOperation", message = ex.Message });
    }
    catch (KeyNotFoundException ex)
    {
        return NotFound(new { error = "NotFound", message = ex.Message });
    }
}

/// <summary>
/// Grid Operator mode: Scans and verifies transaction QR token and finalizes energy
transfer.
/// </summary>
[HttpPost("finalize-transfer")]
[ProducesResponseType(typeof(ReservationResponse), StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status400BadRequest)]
[ProducesResponseType(StatusCodes.Status404NotFound)]
public async Task<IActionResult> FinalizeTransfer([FromBody] FinalizeTransferRequest
request)
{
    // Inline: Authenticates the cryptographic QR token, logs the Grid Operator, and
transitions reservation status to Completed.
    if (!ModelState.IsValid)

```

```

    {
        return BadRequest(ModelState);
    }

    try
    {
        var completed = await _bookingService.FinalizeTransferAsync(request);
        return Ok(completed);
    }
    catch (ArgumentException ex)
    {
        return BadRequest(new { error = "InvalidQRCode", message = ex.Message });
    }
    catch (UnauthorizedAccessException ex)
    {
        return StatusCode(StatusCode.Status403Forbidden, new { error = "Forbidden",
message = ex.Message });
    }
    catch (InvalidOperationException ex)
    {
        return BadRequest(new { error = "InvalidOperation", message = ex.Message });
    }
    catch (KeyNotFoundException ex)
    {
        return NotFound(new { error = "NotFound", message = ex.Message });
    }
}

/// <summary>
/// Retrieves an individual reservation by ID.
/// </summary>
[HttpGet("{id}")]
[ProducesResponseType(typeof(ReservationResponse), StatusCode.Status200OK)]
[ProducesResponseType(StatusCode.Status404NotFound)]
public async Task<IActionResult> GetById(string id)
{
    // Inline: Fetches reservation details by MongoDB document ID.
    var reservation = await _bookingService.GetReservationByIdAsync(id);
    if (reservation == null)
    {

```

```

        return NotFound(new { message = $"Reservation with ID '{id}' was not found." });
    }

    return Ok(reservation);
}

/// <summary>
/// Retrieves reservations with optional filters (prosumer NIC, station ID, status, date range,
search).
/// </summary>
[HttpGet]
[ProducesResponseType(typeof(List<ReservationResponse>), StatusCodes.Status200OK)]
public async Task<IActionResult> GetList([FromQuery] ReservationFilterRequest filter)
{
    // Inline: Queries reservation records matching provided search and status filters.
    var list = await _bookingService.GetReservationsAsync(filter);
    return Ok(list);
}

/// <summary>
/// Retrieves all bookings for a specific prosumer by NIC.
/// </summary>
[HttpGet("prosumer/{nic}")]
[ProducesResponseType(typeof(List<ReservationResponse>), StatusCodes.Status200OK)]
public async Task<IActionResult> GetByProsumer(string nic)
{
    // Inline: Returns the personal booking history and pending reservations for a prosumer.
    var list = await _bookingService.GetReservationsByProsumerAsync(nic);
    return Ok(list);
}

/// <summary>
/// Returns system-wide operational metrics and reservation counts for web/mobile
dashboards.
/// </summary>
[HttpGet("dashboard-summary")]
[ProducesResponseType(typeof(DashboardSummaryResponse),
StatusCodes.Status200OK)]
public async Task<IActionResult> GetDashboardSummary()
{

```

```

        // Inline: Computes count of active stations, total prosumers, pending reservations, and
        count of approved future reservations.
        var summary = await _bookingService.GetDashboardSummaryAsync();
        return Ok(summary);
    }
}

```

SlotsController.cs

```

//
=====

// File: SlotsController.cs
// Project: SolariX - Smart Solar Microgrid Trading System
// Description: API controller for querying available time slots and scheduling battery storage
intervals.
// Module: SE4040 Enterprise Application Development
//
=====

using Microsoft.AspNetCore.Mvc;
using SolariX.Api.DTOs;
using SolariX.Api.Services;

namespace SolariX.Api.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    [Produces("application/json")]
    public class SlotsController : ControllerBase
    {
        private readonly IBookingService _bookingService;

        public SlotsController(IBookingService bookingService)
        {
            // Inline: Injects central booking service to manage and retrieve battery storage slots.
            _bookingService = bookingService;
        }
    }
}

```

```

/// <summary>
/// Retrieves available energy booking slots for a station, optionally filtered by date.
/// </summary>
[HttpGet("station/{stationId}")]
[ProducesResponseType(typeof(List<SlotResponse>), StatusCodes.Status200OK)]
public async Task<IActionResult> GetSlots(string stationId, [FromQuery] DateTime? date)
{
    // Inline: Queries available slots with capacity for the target station.
    var slots = await _bookingService.GetAvailableSlotsAsync(stationId, date);
    return Ok(slots);
}

/// <summary>
/// Creates an individual energy booking slot.
/// </summary>
[HttpPost]
[ProducesResponseType(typeof(SlotResponse), StatusCodes.Status201Created)]
[ProducesResponseType(StatusCodes.Status400BadRequest)]
public async Task<IActionResult> CreateSlot([FromBody] CreateSlotRequest request)
{
    // Inline: Adds a new booking slot to the station schedule.
    if (!ModelState.IsValid)
    {
        return BadRequest(ModelState);
    }

    var created = await _bookingService.CreateSlotAsync(request);
    return CreatedAtAction(nameof(GetSlots), new { stationId = created.StationId },
created);
}

/// <summary>
/// Batch-generates hourly slots for a station for an entire operational day.
/// </summary>
[HttpPost("generate-daily")]
[ProducesResponseType(typeof(List<SlotResponse>), StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status400BadRequest)]
[ProducesResponseType(StatusCodes.Status404NotFound)]

```

```

        public async Task<IActionResult> GenerateDaily([FromBody] GenerateDailySlotsRequest
request)
        {
            // Inline: Automatically populates daily recurring slots across operational hours for a
station.
            if (!ModelState.IsValid)
            {
                return BadRequest(ModelState);
            }

            try
            {
                var slots = await _bookingService.GenerateDailySlotsAsync(request);
                return Ok(slots);
            }
            catch (KeyNotFoundException ex)
            {
                return NotFound(new { message = ex.Message });
            }
        }
    }
}

```

StationsController.cs

```

//
=====

=====
// File: StationsController.cs
// Project: SolariX - Smart Solar Microgrid Trading System
// Description: API controller for solar microgrid hub management, location mapping, and
reservation-guarded node deactivation.
// Module: SE4040 Enterprise Application Development
//
=====

=====

using Microsoft.AspNetCore.Mvc;
using SolariX.Api.DTOs;
using SolariX.Api.Services;

```

```

namespace SolariX.Api.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    [Produces("application/json")]
    public class StationsController : ControllerBase
    {
        private readonly IStationService _stationService;

        public StationsController(IStationService stationService)
        {
            // Inline: Injects station management service for hub operations and deactivation
            validation.
            _stationService = stationService;
        }

        /// <summary>
        /// Retrieves all solar stations, optionally filtered to only active hubs.
        /// </summary>
        [HttpGet]
        [ProducesResponseType(typeof(List<StationResponse>), StatusCodes.Status200OK)]
        public async Task<IActionResult> GetAll([FromQuery] bool? activeOnly)
        {
            // Inline: Returns solar stations including coordinates, capacity, and slot metrics.
            var stations = await _stationService.GetAllStationsAsync(activeOnly);
            return Ok(stations);
        }

        /// <summary>
        /// Retrieves a single station by its unique MongoDB identifier.
        /// </summary>
        [HttpGet("{id}")]
        [ProducesResponseType(typeof(StationResponse), StatusCodes.Status200OK)]
        [ProducesResponseType(StatusCodes.Status404NotFound)]
        public async Task<IActionResult> GetById(string id)
        {
            // Inline: Fetches detailed specifications and battery slot information for a single hub.
            var station = await _stationService.GetStationByIdAsync(id);
            if (station == null)

```

```

    {
        return NotFound(new { message = $"Station with ID '{id}' was not found." });
    }

    return Ok(station);
}

/// <summary>
/// Creates a new solar microgrid hub.
/// </summary>
[HttpPost]
[ProducesResponseType(typeof(StationResponse), StatusCodes.Status201Created)]
[ProducesResponseType(StatusCodes.Status400BadRequest)]
public async Task<IActionResult> Create([FromBody] CreateStationRequest request)
{
    // Inline: Validates and persists a new solar station hub.
    if (!ModelState.IsValid)
    {
        return BadRequest(ModelState);
    }

    try
    {
        var created = await _stationService.CreateStationAsync(request);
        return CreatedAtAction(nameof(GetById), new { id = created.Id }, created);
    }
    catch (ArgumentException ex)
    {
        return BadRequest(new { message = ex.Message });
    }
}

/// <summary>
/// Updates solar station specifications, operational hours, or battery slot counts.
/// </summary>
[HttpPut("{id}")]
[ProducesResponseType(typeof(StationResponse), StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status400BadRequest)]
[ProducesResponseType(StatusCodes.Status404NotFound)]

```

```

    public async Task<IActionResult> Update(string id, [FromBody] UpdateStationRequest
request)
    {
        // Inline: Modifies station attributes; guards against deactivation if active reservations
exist.
        try
        {
            var updated = await _stationService.UpdateStationAsync(id, request);
            return Ok(updated);
        }
        catch (KeyNotFoundException ex)
        {
            return NotFound(new { message = ex.Message });
        }
        catch (InvalidOperationException ex)
        {
            return BadRequest(new { message = ex.Message });
        }
    }

    /// <summary>
    /// Deactivates a solar station hub. Checks for active reservations before proceeding.
    /// </summary>
    [HttpPost("{id}/deactivate")]
    [ProducesResponseType(StatusCodes.Status200OK)]
    [ProducesResponseType(StatusCodes.Status400BadRequest)]
    [ProducesResponseType(StatusCodes.Status404NotFound)]
    public async Task<IActionResult> Deactivate(string id)
    {
        // Inline: Enforces the FAT service rule: deactivation is blocked if active reservations
exist.
        try
        {
            var success = await _stationService.DeactivateStationAsync(id);
            if (!success)
            {
                return BadRequest(new { message = "Unable to deactivate station." });
            }
        }
    }

```

```

        return Ok(new { message = $"Solar station with ID '{id}' was successfully
deactivated." });
    }
    catch (KeyNotFoundException ex)
    {
        return NotFound(new { message = ex.Message });
    }
    catch (InvalidOperationException ex)
    {
        return BadRequest(new { message = ex.Message });
    }
}

/// <summary>
/// Reactivates an inactive solar station hub.
/// </summary>
[HttpPost("{id}/reactivate")]
[ProducesResponseType(StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status404NotFound)]
public async Task<IActionResult> Reactivate(string id)
{
    // Inline: Re-enables operational status for a station.
    try
    {
        var success = await _stationService.ReactivateStationAsync(id);
        if (!success)
        {
            return BadRequest(new { message = "Unable to reactivate station." });
        }

        return Ok(new { message = $"Solar station with ID '{id}' was reactivated." });
    }
    catch (KeyNotFoundException ex)
    {
        return NotFound(new { message = ex.Message });
    }
}
}

```

UsersController.cs

```
//  
=====
```

// File: UsersController.cs
// Project: SolariX - Smart Solar Microgrid Trading System
// Description: API controller for managing user profiles, role-based administration,
deactivations, and Backoffice-only reactivations.
// Module: SE4040 Enterprise Application Development
//

```
=====
```

using Microsoft.AspNetCore.Mvc;
using SolariX.Api.DTOs;
using SolariX.Api.Models;
using SolariX.Api.Services;

namespace SolariX.Api.Controllers
{
 [ApiController]
 [Route("api/[controller]")]
 [Produces("application/json")]
 public class UsersController : ControllerBase
 {
 private readonly IUserService _userService;
 public UsersController(IUserService userService)
 {
 // Inline: Injects user service for lifecycle, updates, and authorization validations.
 _userService = userService;
 }

 /// <summary>
 /// Retrieves all users filtered optionally by role and status.
 /// </summary>
 [HttpGet]
 [ProducesResponseType(typeof(List<UserResponse>), StatusCodes.Status200OK)]

```

    public async Task<IActionResult> GetAll([FromQuery] UserRole? role, [FromQuery]
AccountStatus? status)
    {
        // Inline: Retrieves a list of platform users based on requested role and status filters.
        var users = await _userService.GetUsersAsync(role, status);
        return Ok(users);
    }

    /// <summary>
    /// Retrieves pending prosumer registrations requiring Backoffice verification.
    /// </summary>
    [HttpGet("pending-prosumers")]
    [ProducesResponseType(typeof(List<UserResponse>), StatusCodes.Status200OK)]
    public async Task<IActionResult> GetPendingProsumers()
    {
        // Inline: Queries all prosumer profiles currently awaiting administrator approval.
        var list = await _userService.GetPendingProsumersAsync();
        return Ok(list);
    }

    /// <summary>
    /// Creates a new Backoffice or Grid Operator staff user.
    /// </summary>
    [HttpPost("staff")]
    [ProducesResponseType(typeof(UserResponse), StatusCodes.Status201Created)]
    [ProducesResponseType(StatusCodes.Status400BadRequest)]
    public async Task<IActionResult> CreateStaff([FromBody] CreateOperatorRequest
request)
    {
        // Inline: Enforces creation of operational roles with role restrictions and unique identity
keys.
        if (!ModelState.IsValid)
        {
            return BadRequest(ModelState);
        }

        try
        {
            var created = await _userService.CreateStaffUserAsync(request);
            return CreatedAtAction("GetAll", new { id = created.Id }, created);
        }
    }

```

```

    }
    catch (ArgumentException ex)
    {
        return BadRequest(new { message = ex.Message });
    }
    catch (Exception ex)
    {
        return StatusCode(StatusCodes.Status500InternalServerError, new { message =
ex.Message });
    }
}

```

```

/// <summary>
/// Updates a prosumer profile (name, phone, address, solar capacity).
/// </summary>
[HttpPut("prosumer/{nic}")]
[ProducesResponseType(typeof(UserResponse), StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status404NotFound)]
public async Task<IActionResult> UpdateProfile(string nic, [FromBody]
UpdateProfileRequest request)
{
    // Inline: Updates profile information for the specified prosumer NIC.
    try
    {
        var updated = await _userService.UpdateProsumerProfileAsync(nic, request);
        return Ok(updated);
    }
    catch (KeyNotFoundException ex)
    {
        return NotFound(new { message = ex.Message });
    }
}

```

```

/// <summary>
/// Deactivates a user account.
/// </summary>
[HttpPost("{nic}/deactivate")]
[ProducesResponseType(StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status400BadRequest)]

```

```

    public async Task<IActionResult> Deactivate(string nic, [FromQuery] string
requestedByNic)
    {
        // Inline: Updates account status to Deactivated and logs the requesting identity.
        if (string.IsNullOrEmpty(requestedByNic))
        {
            return BadRequest(new { message = "requestedByNic parameter is required." });
        }

        var success = await _userService.DeactivateAccountAsync(nic, requestedByNic);
        if (!success)
        {
            return BadRequest(new { message = $"Could not deactivate user with NIC '{nic}'." });
        }

        return Ok(new { message = $"Account '{nic}' has been successfully deactivated." });
    }

    /// <summary>
    /// Reactivates a deactivated account. Strictly requires Backoffice authorization.
    /// </summary>
    [HttpPost("{nic}/reactivate")]
    [ProducesResponseType(StatusCodes.Status200OK)]
    [ProducesResponseType(StatusCodes.Status403Forbidden)]
    [ProducesResponseType(StatusCodes.Status400BadRequest)]
    public async Task<IActionResult> Reactivate(string nic, [FromQuery] string
backofficeNic)
    {
        // Inline: Enforces that only Backoffice personnel can reactivate deactivated accounts as
per assignment requirements.
        if (string.IsNullOrEmpty(backofficeNic))
        {
            return BadRequest(new { message = "backofficeNic parameter is required." });
        }

        try
        {
            var success = await _userService.ReactivateAccountByBackofficeAsync(nic,
backofficeNic);
            if (!success)

```

```

        {
            return BadRequest(new { message = $"Could not reactivate user with NIC '{nic}'."
});
        }

        return Ok(new { message = $"Account '{nic}' has been successfully reactivated by
Backoffice officer '{backofficeNic}'." });
    }
    catch (UnauthorizedAccessException ex)
    {
        return StatusCode(StatusCodes.Status403Forbidden, new { message = ex.Message });
    }
    catch (Exception ex)
    {
        return BadRequest(new { message = ex.Message });
    }
}

/// <summary>
/// Approves a pending prosumer registration. Requires Backoffice authorization.
/// </summary>
[HttpPost("{nic}/approve")]
[ProducesResponseType(StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status403Forbidden)]
public async Task<IActionResult> Approve(string nic, [FromQuery] string backofficeNic)
{
    // Inline: Approves prosumer registration by a Backoffice user.
    try
    {
        var success = await _userService.ApproveProsumerRegistrationAsync(nic,
backofficeNic);
        if (!success)
        {
            return BadRequest(new { message = $"Could not approve user with NIC '{nic}'." });
        }

        return Ok(new { message = $"Prosumer account '{nic}' approved." });
    }
    catch (UnauthorizedAccessException ex)
    {

```

```

        return StatusCode(StatusCodes.Status403Forbidden, new { message = ex.Message });
    }
}
}
}

```

User.cs

```

//
=====

// File: User.cs
// Project: SolariX - Smart Solar Microgrid Trading System
// Description: User entity mapped to the MongoDB 'Users' collection. Supports Prosumers, Grid
// Operators, and Backoffice staff.
// Module: SE4040 Enterprise Application Development
//
=====

using MongoDB.Bson;
using MongoDB.Bson.Serialization.Attributes;

namespace SolariX.Api.Models
{
    /// <summary>
    /// Represents an authenticated system user or prosumer registered on the microgrid platform.
    /// </summary>
    [BsonIgnoreExtraElements]
    public class User
    {
        [BsonId]
        [BsonRepresentation(BsonType.ObjectId)]
        public string? Id { get; set; }

        /// <summary>
        /// National Identity Card (NIC) number acting as primary identity key for Prosumers.
        /// </summary>
        [BsonElement("nic")]
        public string NIC { get; set; } = string.Empty;
    }
}

```

```

[BsonElement("fullName")]
public string FullName { get; set; } = string.Empty;

[BsonElement("email")]
public string Email { get; set; } = string.Empty;

[BsonElement("phoneNumber")]
public string PhoneNumber { get; set; } = string.Empty;

[BsonElement("passwordHash")]
public string PasswordHash { get; set; } = string.Empty;

[BsonElement("role")]
[BsonRepresentation(BsonType.String)]
public UserRole Role { get; set; } = UserRole.Prosumer;

[BsonElement("status")]
[BsonRepresentation(BsonType.String)]
public AccountStatus Status { get; set; } = AccountStatus.Active;

[BsonElement("address")]
public string Address { get; set; } = string.Empty;

[BsonElement("solarCapacityKW")]
public double SolarCapacityKW { get; set; } = 0.0;

[BsonElement("deactivatedAt")]
public DateTime? DeactivatedAt { get; set; }

[BsonElement("deactivatedBy")]
public string? DeactivatedBy { get; set; }

[BsonElement("createdAt")]
public DateTime CreatedAt { get; set; } = DateTime.UtcNow;

[BsonElement("updatedAt")]
public DateTime UpdatedAt { get; set; } = DateTime.UtcNow;

/// <summary>

```

```

    /// Factory helper to build a new Prosumer record with initial timestamps.
    /// </summary>
    public static User CreateProsumer(string nic, string fullName, string email, string phone,
    string passwordHash, string address, double solarCapacityKW)
    {
        // Inline: Instantiates and returns a new Prosumer user model with active status and
        default timestamps.
        return new User
        {
            NIC = nic.Trim().ToUpperInvariant(),
            FullName = fullName.Trim(),
            Email = email.Trim().ToLowerInvariant(),
            PhoneNumber = phone.Trim(),
            PasswordHash = passwordHash,
            Role = UserRole.Prosumer,
            Status = AccountStatus.Active,
            Address = address.Trim(),
            SolarCapacityKW = solarCapacityKW,
            CreatedAt = DateTime.UtcNow,
            UpdatedAt = DateTime.UtcNow
        };
    }
}

```

SolarStationInfo.cs

```

//
=====

// File: SolarStationInfo.cs
// Project: SolariX - Smart Solar Microgrid Trading System
// Description: Solar station hub entity mapped to the MongoDB 'SolarStationInfo' collection.
// Module: SE4040 Enterprise Application Development
//
=====

using MongoDB.Bson;
using MongoDB.Bson.Serialization.Attributes;

```

```

namespace SolariX.Api.Models
{
    /// <summary>
    /// Represents a physical solar microgrid hub/station that hosts battery storage slots and
    handles energy trades.
    /// </summary>
    [BsonIgnoreExtraElements]
    public class SolarStationInfo
    {
        [BsonId]
        [BsonRepresentation(BsonType.ObjectId)]
        public string? Id { get; set; }

        [BsonElement("stationCode")]
        public string StationCode { get; set; } = string.Empty;

        [BsonElement("stationName")]
        public string StationName { get; set; } = string.Empty;

        [BsonElement("locationName")]
        public string LocationName { get; set; } = string.Empty;

        [BsonElement("latitude")]
        public double Latitude { get; set; }

        [BsonElement("longitude")]
        public double Longitude { get; set; }

        /// <summary>
        /// Total station capacity specs in kW/h.
        /// </summary>
        [BsonElement("capacityKWh")]
        public double CapacityKWh { get; set; }

        /// <summary>
        /// Total physical battery storage slots present at this hub.
        /// </summary>
        [BsonElement("totalBatterySlots")]
        public int TotalBatterySlots { get; set; }
    }
}

```

```

    /// <summary>
    /// Currently available battery storage slots ready for booking.
    /// </summary>
    [BsonElement("availableBatterySlots")]
    public int AvailableBatterySlots { get; set; }

    /// <summary>
    /// Operational status of the solar grid hub.
    /// </summary>
    [BsonElement("isActive")]
    public bool IsActive { get; set; } = true;

    [BsonElement("operationalHours")]
    public string OperationalHours { get; set; } = "06:00 - 20:00";

    [BsonElement("contactNumber")]
    public string ContactNumber { get; set; } = string.Empty;

    [BsonElement("createdAt")]
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;

    [BsonElement("updatedAt")]
    public DateTime UpdatedAt { get; set; } = DateTime.UtcNow;
}

```

EnergyBookingSlot.cs

```

//
=====

// File: EnergyBookingSlot.cs
// Project: SolariX - Smart Solar Microgrid Trading System
// Description: Battery storage slot entity mapped to the MongoDB 'EnergyBookingSlots'
collection.
// Module: SE4040 Enterprise Application Development
//
=====
=====

```

```

using MongoDB.Bson;
using MongoDB.Bson.Serialization.Attributes;

namespace SolariX.Api.Models
{
    /// <summary>
    /// Represents a discrete schedulable time slot for energy charging or drop-off at a specific
    microgrid station.
    /// </summary>
    [BsonIgnoreExtraElements]
    public class EnergyBookingSlot
    {
        [BsonId]
        [BsonRepresentation(BsonType.ObjectId)]
        public string? Id { get; set; }

        [BsonElement("stationId")]
        [BsonRepresentation(BsonType.ObjectId)]
        public string StationId { get; set; } = string.Empty;

        [BsonElement("slotDate")]
        public DateTime SlotDate { get; set; }

        [BsonElement("startTime")]
        public DateTime StartTime { get; set; }

        [BsonElement("endTime")]
        public DateTime EndTime { get; set; }

        [BsonElement("maxCapacityKW")]
        public double MaxCapacityKW { get; set; }

        [BsonElement("availableCapacityKW")]
        public double AvailableCapacityKW { get; set; }

        [BsonElement("status")]
        [BsonRepresentation(BsonType.String)]
        public SlotStatus Status { get; set; } = SlotStatus.Available;
    }
}

```

```

[BsonElement("createdAt")]
public DateTime CreatedAt { get; set; } = DateTime.UtcNow;

[BsonElement("updatedAt")]
public DateTime UpdatedAt { get; set; } = DateTime.UtcNow;
}
}

```

Enums.cs

```

//
=====

// File: Enums.cs
// Project: SolariX - Smart Solar Microgrid Trading System
// Description: Domain enumerations for roles, account statuses, reservation statuses, and slot
// statuses.
// Module: SE4040 Enterprise Application Development
//
=====

namespace SolariX.Api.Models
{
    /// <summary>
    /// User roles recognized by the SolariX platform.
    /// </summary>
    public enum UserRole
    {
        Backoffice,
        GridOperator,
        Prosumer
    }

    /// <summary>
    /// Lifecycle status of a user profile.
    /// </summary>
    public enum AccountStatus
    {
        PendingApproval,

```

```

        Active,
        Deactivated
    }

    /// <summary>
    /// Lifecycle status of an energy trading reservation.
    /// </summary>
    public enum ReservationStatus
    {
        Pending,
        Approved,
        Completed,
        Cancelled
    }

    /// <summary>
    /// Operational status of an energy booking slot.
    /// </summary>
    public enum SlotStatus
    {
        Available,
        Booked,
        Blocked
    }
}

```

BookingService.cs

```

//
=====

// File: BookingService.cs
// Project: SolariX - Smart Solar Microgrid Trading System
// Description: Core FAT service implementation enforcing business rules: 7-day future booking
// window, 12-hour cancellation/update notice thresholds, QR dispatching, and slot capacities.
// Module: SE4040 Enterprise Application Development
//
=====
=====

```

```

using MongoDB.Driver;
using SolariX.Api.Data;
using SolariX.Api.DTOs;
using SolariX.Api.Models;

namespace SolariX.Api.Services
{
    public class BookingService : IBookingService
    {
        private readonly IMongoDbContext _context;
        private readonly IQRService _qrService;

        public BookingService(IMongoDbContext context, IQRService qrService)
        {
            // Inline: Injects the central MongoDB context and QR signature generator for reservation
            lifecycle management.
            _context = context;
            _qrService = qrService;
        }

        public async Task<ReservationResponse>
        CreateReservationAsync(CreateReservationRequest request)
        {
            // Inline: Validates prosumer status, checks station state, enforces the 7-day future
            window rule against combined UTC slot time, verifies slot capacity, generates secure QR token,
            and creates the reservation.
            var cleanNic = request.ProsumerNIC.Trim().ToUpperInvariant();

            // 1. Verify Prosumer Profile
            var prosumer = await _context.Users.Find(u => u.NIC ==
            cleanNic).FirstOrDefaultAsync();
            if (prosumer == null)
            {
                throw new KeyNotFoundException($"Prosumer with NIC '{cleanNic}' was not
            found.");
            }

            if (prosumer.Status != AccountStatus.Active)
            {

```

```
        throw new InvalidOperationException($"Cannot create booking. Prosumer account is currently '{prosumer.Status}'. Only Active prosumers can book energy slots.");
    }
```

```
    // 2. Verify Solar Station Hub
    var station = await _context.SolarStationInfo.Find(s => s.Id == request.StationId).FirstOrDefaultAsync();
    if (station == null)
    {
        throw new KeyNotFoundException($"Solar Station with ID '{request.StationId}' was not found.");
    }
```

```
    if (!station.IsActive)
    {
        throw new InvalidOperationException($"Solar Station '{station.StationName}' is currently inactive or undergoing maintenance.");
    }
```

```
    // 3. Verify Energy Booking Slot
    var slot = await _context.EnergyBookingSlots.Find(s => s.Id == request.SlotId && s.StationId == request.StationId).FirstOrDefaultAsync();
    if (slot == null)
    {
        throw new KeyNotFoundException($"Energy Booking Slot with ID '{request.SlotId}' was not found for station '{station.StationName}'.");
    }
```

```
    if (slot.Status != SlotStatus.Available)
    {
        throw new InvalidOperationException("The requested energy booking slot is no longer available.");
    }
```

```
    if (slot.AvailableCapacityKW < request.EnergyAmountKW)
    {
        throw new InvalidOperationException($"Insufficient slot capacity. Requested: {request.EnergyAmountKW} kW, Available: {slot.AvailableCapacityKW} kW.");
    }
```

```

// 4. TIMEZONE-SAFE COMBINATION: Combine SlotDate + StartTime into UTC
DateTime
    DateTime slotDateTimeUtc =
DateTime.SpecifyKind(slot.SlotDate.Date.Add(slot.StartTime.TimeOfDay),
DateTimeKind.Utc);
    var nowUtc = DateTime.UtcNow;
    var maxFutureWindow = nowUtc.AddDays(7);

    if (slotDateTimeUtc < nowUtc)
    {
        throw new ArgumentOutOfRangeException(nameof(request.SlotId), "Cannot book a
time slot in the past.");
    }

    if (slotDateTimeUtc > maxFutureWindow)
    {
        throw new ArgumentOutOfRangeException(nameof(request.SlotId), $"Power trading
reservations must be scheduled within 7 days from now (allowed up to
{maxFutureWindow:yyyy-MM-dd HH:mm UTC}). The selected slot is beyond this window.");
    }

// 5. Generate Human-Readable Unique Reservation Number
var timestampCode = DateTime.UtcNow.ToString("yyMMddHHmmss");
var randomSuffix = Random.Shared.Next(100, 999);
var reservationNumber = $"RES-{timestampCode}-{randomSuffix}";

// 6. Generate Cryptographic QR Payload: { ReservationId, ProsumerNIC, StationId,
SlotId, Timestamp, SignatureHash }
var reservationId = MongoDB.Bson.ObjectId.GenerateNewId().ToString();
var qrToken = _qrService.GenerateQrPayload(reservationId, cleanNic, station.Id!,
slot.Id!, slotDateTimeUtc);

// 7. Persist Reservation Record
var reservation = new EnergyReservation
{
    Id = reservationId,
    ReservationNumber = reservationNumber,
    ProsumerNIC = cleanNic,
    StationId = station.Id!,
    SlotId = slot.Id!,

```

```

        ReservationDate = DateTime.SpecifyKind(slot.SlotDate.Date, DateTimeKind.Utc),
        StartTime = slotDateTimeUtc,
        EndTime =
DateTime.SpecifyKind(slot.SlotDate.Date.Add(slot.EndTime.TimeOfDay), DateTimeKind.Utc),
        EnergyAmountKW = request.EnergyAmountKW,
        TradeType = request.TradeType ?? "DropOff",
        Status = ReservationStatus.Approved,
        QrCodeToken = qrToken,
        CreatedAt = DateTime.UtcNow,
        UpdatedAt = DateTime.UtcNow
    };

    await _context.EnergyReservations.InsertOneAsync(reservation);

    // 8. Deduct Slot Capacity
    var updatedCapacity = slot.AvailableCapacityKW - request.EnergyAmountKW;
    var slotStatus = updatedCapacity <= 0 ? SlotStatus.Booked : SlotStatus.Available;

    var slotUpdate = Builders<EnergyBookingSlot>.Update
        .Set(s => s.AvailableCapacityKW, Math.Max(0, updatedCapacity))
        .Set(s => s.Status, slotStatus)
        .Set(s => s.UpdatedAt, DateTime.UtcNow);

    await _context.EnergyBookingSlots.UpdateOneAsync(s => s.Id == slot.Id, slotUpdate);

    return MapToResponse(reservation, station.StationName);
}

public async Task<ReservationResponse> UpdateReservationAsync(string reservationId,
string prosumerNic, UpdateReservationRequest request)
{
    // Inline: Evaluates the 12-hour cancellation/update rule against combined UTC SlotDate
    + StartTime, updates slot or energy amount, and recalculates QR payload.
    var cleanNic = prosumerNic.Trim().ToUpperInvariant();
    var reservation = await _context.EnergyReservations.Find(r => r.Id == reservationId &&
r.ProsumerNIC == cleanNic).FirstOrDefaultAsync();
    if (reservation == null)
    {
        throw new KeyNotFoundException($"Reservation with ID '{reservationId}' for
prosumer '{cleanNic}' was not found.");
    }
}

```

```

    }

    if (reservation.Status != ReservationStatus.Approved && reservation.Status !=
ReservationStatus.Pending)
    {
        throw new InvalidOperationException($"Cannot modify a reservation with status
'{reservation.Status}'.");
    }

    // Combine SlotDate + StartTime safely into UTC
    DateTime slotDateTimeUtc =
DateTime.SpecifyKind(reservation.ReservationDate.Date.Add(reservation.StartTime.TimeOfDa
y), DateTimeKind.Utc);
    var nowUtc = DateTime.UtcNow;
    var timeRemaining = slotDateTimeUtc - nowUtc;

    if (timeRemaining < TimeSpan.FromHours(12))
    {
        throw new InvalidOperationException($"Updates and cancellations require at least 12
hours' advance notice. Only {timeRemaining.TotalHours:F1} hour(s) remain before this
reservation.");
    }

    // If changing slot, validate new slot and the 7-day rule
    if (!string.IsNullOrEmpty(request.NewSlotId) && request.NewSlotId !=
reservation.SlotId)
    {
        var newSlot = await _context.EnergyBookingSlots.Find(s => s.Id ==
request.NewSlotId).FirstOrDefaultAsync();
        if (newSlot == null)
        {
            throw new KeyNotFoundException($"Target booking slot '{request.NewSlotId}' not
found.");
        }

        DateTime newSlotDateTimeUtc =
DateTime.SpecifyKind(newSlot.SlotDate.Date.Add(newSlot.StartTime.TimeOfDay),
DateTimeKind.Utc);
        if (newSlotDateTimeUtc > nowUtc.AddDays(7))
        {

```

```

        throw new ArgumentOutOfRangeException(nameof(request.NewSlotId), "New slot
must also fall within the 7-day booking window.");
    }

    var requiredKW = request.NewEnergyAmountKW ?? reservation.EnergyAmountKW;
    if (newSlot.AvailableCapacityKW < requiredKW)
    {
        throw new InvalidOperationException($"Target slot does not have sufficient
capacity. Available: {newSlot.AvailableCapacityKW} kW.");
    }

    // Restore previous slot capacity
    var prevSlotUpdate = Builders<EnergyBookingSlot>.Update
        .Inc(s => s.AvailableCapacityKW, reservation.EnergyAmountKW)
        .Set(s => s.Status, SlotStatus.Available)
        .Set(s => s.UpdatedAt, DateTime.UtcNow);
    await _context.EnergyBookingSlots.UpdateOneAsync(s => s.Id == reservation.SlotId,
prevSlotUpdate);

    // Deduct from new slot
    var newCapacity = newSlot.AvailableCapacityKW - requiredKW;
    var newSlotUpdate = Builders<EnergyBookingSlot>.Update
        .Set(s => s.AvailableCapacityKW, Math.Max(0, newCapacity))
        .Set(s => s.Status, newCapacity <= 0 ? SlotStatus.Booked : SlotStatus.Available)
        .Set(s => s.UpdatedAt, DateTime.UtcNow);
    await _context.EnergyBookingSlots.UpdateOneAsync(s => s.Id == newSlot.Id,
newSlotUpdate);

    reservation.SlotId = newSlot.Id!;
    reservation.StationId = newSlot.StationId;
    reservation.ReservationDate = DateTime.SpecifyKind(newSlot.SlotDate.Date,
DateTimeKind.Utc);
    reservation.StartTime = newSlotDateTimeUtc;
    reservation.EndTime =
DateTime.SpecifyKind(newSlot.SlotDate.Date.Add(newSlot.EndTime.TimeOfDay),
DateTimeKind.Utc);
    reservation.QrCodeToken = _qrService.GenerateQrPayload(reservation.Id!,
reservation.ProsumerNIC, reservation.StationId, reservation.SlotId, reservation.StartTime);
}

```

```

    if (request.NewEnergyAmountKW.HasValue && request.NewEnergyAmountKW.Value
> 0)
    {
        reservation.EnergyAmountKW = request.NewEnergyAmountKW.Value;
    }

    if (!string.IsNullOrEmpty(request.TradeType))
    {
        reservation.TradeType = request.TradeType;
    }

    reservation.UpdatedAt = DateTime.UtcNow;

    await _context.EnergyReservations.ReplaceOneAsync(r => r.Id == reservation.Id,
reservation);

    var station = await _context.SolarStationInfo.Find(s => s.Id ==
reservation.StationId).FirstOrDefaultAsync();
    return MapToResponse(reservation, station?.StationName ?? "Solar Station");
}

public async Task<ReservationResponse> CancelReservationAsync(string reservationId,
CancelReservationRequest request)
{
    // Inline: Validates that cancellation request satisfies the mandatory 12-hour threshold
against combined UTC SlotDate + StartTime; releases slot capacity if valid.
    var cleanNic = request.ProsumerNIC.Trim().ToUpperInvariant();
    var reservation = await _context.EnergyReservations.Find(r => r.Id == reservationId &&
r.ProsumerNIC == cleanNic).FirstOrDefaultAsync();
    if (reservation == null)
    {
        throw new KeyNotFoundException($"Reservation with ID '{reservationId}' for
prosumer '{cleanNic}' was not found.");
    }

    if (reservation.Status == ReservationStatus.Cancelled)
    {
        throw new InvalidOperationException("This reservation has already been cancelled.");
    }
}

```

```

        if (reservation.Status == ReservationStatus.Completed)
        {
            throw new InvalidOperationException("Cannot cancel an already completed energy
transfer reservation.");
        }

        // Combine SlotDate + StartTime safely into UTC
        DateTime slotDateTimeUtc =
DateTime.SpecifyKind(reservation.ReservationDate.Date.Add(reservation.StartTime.TimeOfDa
y), DateTimeKind.Utc);
        var nowUtc = DateTime.UtcNow;
        var timeRemaining = slotDateTimeUtc - nowUtc;

        if (timeRemaining < TimeSpan.FromHours(12))
        {
            throw new InvalidOperationException($"Updates and cancellations require at least 12
hours' advance notice. Only {timeRemaining.TotalHours:F1} hour(s) remain before this
reservation.");
        }

        // Restore slot capacity
        var slotUpdate = Builders<EnergyBookingSlot>.Update
            .Inc(s => s.AvailableCapacityKW, reservation.EnergyAmountKW)
            .Set(s => s.Status, SlotStatus.Available)
            .Set(s => s.UpdatedAt, DateTime.UtcNow);
        await _context.EnergyBookingSlots.UpdateOneAsync(s => s.Id == reservation.SlotId,
slotUpdate);

        // Update reservation status to Cancelled
        reservation.Status = ReservationStatus.Cancelled;
        reservation.CancellationReason = request.Reason ?? "Cancelled by prosumer";
        reservation.CancelledAt = DateTime.UtcNow;
        reservation.UpdatedAt = DateTime.UtcNow;

        await _context.EnergyReservations.ReplaceOneAsync(r => r.Id == reservation.Id,
reservation);

        var station = await _context.SolarStationInfo.Find(s => s.Id ==
reservation.StationId).FirstOrDefaultAsync();
        return MapToResponse(reservation, station?.StationName ?? "Solar Station");

```

```

    }

    public async Task<ReservationResponse>
VerifyAndCompleteAsync(VerifyAndCompleteRequest request)
    {
        // Inline: Verifies QR token signature, confirms reservation is currently in Approved
status, checks calling user has GridOperator role, sets status to Completed, and timestamps
completion.
        bool isValidQr = _qrService.ValidateQrPayload(request.QrToken, out var reservationId,
out var prosumerNic, out _, out _, out _);
        if (!isValidQr)
        {
            throw new ArgumentException("Invalid or forged QR code payload. Signature
verification failed.");
        }

        // Verify operator identity and role
        var cleanOperatorId = request.OperatorId.Trim().ToUpperInvariant();
        bool isOpObjectId = MongoDB.Bson.ObjectId.TryParse(request.OperatorId, out _);

        var operatorUser = isOpObjectId
            ? await _context.Users.Find(u => u.NIC == cleanOperatorId || u.Id ==
request.OperatorId).FirstOrDefaultAsync()
            : await _context.Users.Find(u => u.NIC == cleanOperatorId).FirstOrDefaultAsync();

        if (operatorUser == null || operatorUser.Role != UserRole.GridOperator)
        {
            throw new UnauthorizedAccessException("Forbidden: Only users with the
GridOperator role are authorized to verify and finalize energy transfers.");
        }

        // Find reservation and verify Approved status
        bool isResObjectId = MongoDB.Bson.ObjectId.TryParse(reservationId, out _);
        var reservation = isResObjectId
            ? await _context.EnergyReservations.Find(r => (r.Id == reservationId ||
r.ReservationNumber == reservationId) && r.ProsumerNIC ==
prosumerNic).FirstOrDefaultAsync()
            : await _context.EnergyReservations.Find(r => r.ReservationNumber == reservationId
&& r.ProsumerNIC == prosumerNic).FirstOrDefaultAsync();

```

```

        if (reservation == null)
        {
            throw new KeyNotFoundException($"No active reservation found matching identifier
'{reservationId}'.");
        }

        if (reservation.Status != ReservationStatus.Approved)
        {
            throw new InvalidOperationException($"Cannot complete reservation. Current status
is '{reservation.Status}', expected 'Approved'.");
        }

        reservation.Status = ReservationStatus.Completed;
        reservation.FinalizedByOperatorNIC = operatorUser.NIC;
        reservation.FinalizedAt = DateTime.UtcNow;
        reservation.EnergyAmountKW = request.EnergyDeliveredKWh > 0 ?
request.EnergyDeliveredKWh : reservation.EnergyAmountKW;
        reservation.OperatorNotes = request.Notes;
        reservation.UpdatedAt = DateTime.UtcNow;

        await _context.EnergyReservations.ReplaceOneAsync(r => r.Id == reservation.Id,
reservation);

        var station = await _context.SolarStationInfo.Find(s => s.Id ==
reservation.StationId).FirstOrDefaultAsync();
        return MapToResponse(reservation, station?.StationName ?? "Solar Station");
    }

    public async Task<ReservationResponse> FinalizeTransferAsync(FinalizeTransferRequest
request)
    {
        // Inline: Adapter delegating to VerifyAndCompleteAsync with operator NIC and notes.
        return await VerifyAndCompleteAsync(new VerifyAndCompleteRequest
        {
            QrToken = request.QrToken,
            OperatorId = request.OperatorNIC,
            EnergyDeliveredKWh = 0,
            Notes = request.Notes
        });
    }

```

```

public async Task<ReservationResponse?> GetReservationByIdAsync(string id)
{
    // Inline: Queries an individual reservation by MongoDB document ObjectId.
    var reservation = await _context.EnergyReservations.Find(r => r.Id ==
id).FirstOrDefaultAsync();
    if (reservation == null) return null;

    var station = await _context.SolarStationInfo.Find(s => s.Id ==
reservation.StationId).FirstOrDefaultAsync();
    return MapToResponse(reservation, station?.StationName ?? "Solar Station");
}

public async Task<ReservationResponse?> GetReservationByNumberAsync(string
reservationNumber)
{
    // Inline: Queries an individual reservation by its unique human-readable reservation
number.
    var reservation = await _context.EnergyReservations.Find(r => r.ReservationNumber ==
reservationNumber).FirstOrDefaultAsync();
    if (reservation == null) return null;

    var station = await _context.SolarStationInfo.Find(s => s.Id ==
reservation.StationId).FirstOrDefaultAsync();
    return MapToResponse(reservation, station?.StationName ?? "Solar Station");
}

public async Task<List<ReservationResponse>>
GetReservationsAsync(ReservationFilterRequest filter)
{
    // Inline: Filters reservations by prosumer NIC, station ID, reservation status, date range,
or free-text search.
    var builder = Builders<EnergyReservation>.Filter;
    var query = builder.Empty;

    if (!string.IsNullOrEmpty(filter.ProsumerNIC))
    {
        query &= builder.Eq(r => r.ProsumerNIC,
filter.ProsumerNIC.Trim().ToUpperInvariant());
    }
}

```

```

if (!string.IsNullOrEmpty(filter.StationId))
{
    query &= builder.Eq(r => r.StationId, filter.StationId.Trim());
}

if (filter.Status.HasValue)
{
    query &= builder.Eq(r => r.Status, filter.Status.Value);
}

if (filter.FromDate.HasValue)
{
    query &= builder.Gte(r => r.ReservationDate, filter.FromDate.Value.Date);
}

if (filter.ToDate.HasValue)
{
    query &= builder.Lte(r => r.ReservationDate,
filter.ToDate.Value.Date.AddDays(1).AddTicks(-1));
}

if (!string.IsNullOrEmpty(filter.SearchQuery))
{
    var q = filter.SearchQuery.Trim();
    query &= builder.Or(
        builder.Regex(r => r.ReservationNumber, new
MongoDB.Bson.BsonRegularExpression(q, "i")),
        builder.Regex(r => r.ProsumerNIC, new
MongoDB.Bson.BsonRegularExpression(q, "i"))
    );
}

var reservations = await _context.EnergyReservations.Find(query).SortByDescending(r
=> r.CreatedAt).ToListAsync();

var stationIds = reservations.Select(r => r.StationId).Distinct().ToList();
var stations = await _context.SolarStationInfo.Find(s =>
stationIds.Contains(s.Id!)).ToListAsync();
var stationMap = stations.ToDictionary(s => s.Id!, s => s.StationName);

```

```

        return reservations.Select(r => MapToResponse(r,
stationMap.GetValueOrDefault(r.StationId, "Solar Station"))).ToList();
    }

    public async Task<List<ReservationResponse>> GetReservationsByProsumerAsync(string
prosumerNic)
    {
        // Inline: Retrieves the full reservation history for a specific prosumer identified by NIC.
        return await GetReservationsAsync(new ReservationFilterRequest { ProsumerNIC =
prosumerNic });
    }

    public async Task<DashboardSummaryResponse> GetDashboardSummaryAsync()
    {
        // Inline: Computes system-wide operational metrics including station health, prosumer
counts, and active vs approved future reservations.
        var totalStations = (int)await _context.SolarStationInfo.CountDocumentsAsync(_ =>
true);
        var activeStations = (int)await _context.SolarStationInfo.CountDocumentsAsync(s =>
s.IsActive);

        var totalProsumers = (int)await _context.Users.CountDocumentsAsync(u => u.Role ==
UserRole.Prosumer);
        var pendingProsumers = (int)await _context.Users.CountDocumentsAsync(u => u.Role
== UserRole.Prosumer && u.Status == AccountStatus.PendingApproval);

        var pendingReservations = (int)await
_context.EnergyReservations.CountDocumentsAsync(r => r.Status ==
ReservationStatus.Pending);

        // Approved future reservations (start time in future)
        var nowUtc = DateTime.UtcNow;
        var approvedFuture = (int)await _context.EnergyReservations.CountDocumentsAsync(r
=>
            r.Status == ReservationStatus.Approved && r.StartTime >= nowUtc);

        var completedReservations = (int)await
_context.EnergyReservations.CountDocumentsAsync(r => r.Status ==
ReservationStatus.Completed);

```

```

        var cancelledReservations = (int)await
_context.EnergyReservations.CountDocumentsAsync(r => r.Status ==
ReservationStatus.Cancelled);

        // Total energy traded for completed reservations
        var completedList = await _context.EnergyReservations.Find(r => r.Status ==
ReservationStatus.Completed).ToListAsync();
        var totalEnergyTraded = completedList.Sum(r => r.EnergyAmountKW);

        return new DashboardSummaryResponse
        {
            TotalStations = totalStations,
            ActiveStations = activeStations,
            TotalProsumers = totalProsumers,
            PendingProsumerApprovals = pendingProsumers,
            PendingReservationsCount = pendingReservations,
            ApprovedFutureReservationsCount = approvedFuture,
            CompletedReservationsCount = completedReservations,
            CancelledReservationsCount = cancelledReservations,
            TotalEnergyTradedKW = totalEnergyTraded
        };
    }

    public async Task<List<SlotResponse>> GetAvailableSlotsAsync(string stationId,
DateTime? date = null)
    {
        // Inline: Queries available slots with capacity for the target solar station, optionally
scoped to a particular date.
        var builder = Builders<EnergyBookingSlot>.Filter;
        var filter = builder.Eq(s => s.StationId, stationId) & builder.Eq(s => s.Status,
SlotStatus.Available);

        if (date.HasValue)
        {
            var dayStart = date.Value.Date;
            var dayEnd = dayStart.AddDays(1);
            filter &= builder.Gte(s => s.SlotDate, dayStart) & builder.Lt(s => s.SlotDate, dayEnd);
        }
    }

```

```

        var slots = await _context.EnergyBookingSlots.Find(filter).SortBy(s =>
s.StartTime).ToListAsync();
        return slots.Select(MapSlotToResponse).ToList();
    }

    public async Task<SlotResponse> CreateSlotAsync(CreateSlotRequest request)
    {
        // Inline: Adds a single booking slot to a microgrid station with UTC timestamps.
        var slot = new EnergyBookingSlot
        {
            StationId = request.StationId,
            SlotDate = DateTime.SpecifyKind(request.SlotDate.Date, DateTimeKind.Utc),
            StartTime = DateTime.SpecifyKind(request.StartTime, DateTimeKind.Utc),
            EndTime = DateTime.SpecifyKind(request.EndTime, DateTimeKind.Utc),
            MaxCapacityKW = request.MaxCapacityKW,
            AvailableCapacityKW = request.MaxCapacityKW,
            Status = SlotStatus.Available,
            CreatedAt = DateTime.UtcNow,
            UpdatedAt = DateTime.UtcNow
        };

        await _context.EnergyBookingSlots.InsertOneAsync(slot);
        return MapSlotToResponse(slot);
    }

    public async Task<List<SlotResponse>>
GenerateDailySlotsAsync(GenerateDailySlotsRequest request)
    {
        // Inline: Automates batch slot creation across operational hours in UTC for a target
        microgrid station and date.
        var station = await _context.SolarStationInfo.Find(s => s.Id ==
request.StationId).FirstOrDefaultAsync();
        if (station == null)
        {
            throw new KeyNotFoundException($"Station with ID '{request.StationId}' not
found.");
        }

        var targetDate = DateTime.SpecifyKind(request.TargetDate.Date, DateTimeKind.Utc);
        var startHour = 8; // 08:00 AM

```

```

        var endHour = 18; // 06:00 PM
        var durationMinutes = request.SlotDurationMinutes > 0 ? request.SlotDurationMinutes :
60;
        var capacityKW = request.SlotCapacityKW > 0 ? request.SlotCapacityKW : 25.0;

        var newSlots = new List<EnergyBookingSlot>();
        var current = targetDate.AddHours(startHour);
        var dayEnd = targetDate.AddHours(endHour);

        while (current < dayEnd)
        {
            var slotEnd = current.AddMinutes(durationMinutes);
            if (slotEnd > dayEnd) break;

            newSlots.Add(new EnergyBookingSlot
            {
                StationId = request.StationId,
                SlotDate = targetDate,
                StartTime = current,
                EndTime = slotEnd,
                MaxCapacityKW = capacityKW,
                AvailableCapacityKW = capacityKW,
                Status = SlotStatus.Available,
                CreatedAt = DateTime.UtcNow,
                UpdatedAt = DateTime.UtcNow
            });

            current = slotEnd;
        }

        if (newSlots.Count > 0)
        {
            await _context.EnergyBookingSlots.InsertManyAsync(newSlots);
        }

        return newSlots.Select(MapSlotToResponse).ToList();
    }

    private static ReservationResponse MapToResponse(EnergyReservation res, string
stationName)

```

```

    {
    // Inline: Transforms internal EnergyReservation entity into client ReservationResponse
    DTO.
    return new ReservationResponse
    {
        Id = res.Id ?? string.Empty,
        ReservationNumber = res.ReservationNumber,
        ProsumerNIC = res.ProsumerNIC,
        StationId = res.StationId,
        StationName = stationName,
        SlotId = res.SlotId,
        ReservationDate = res.ReservationDate,
        StartTime = res.StartTime,
        EndTime = res.EndTime,
        EnergyAmountKW = res.EnergyAmountKW,
        TradeType = res.TradeType,
        Status = res.Status.ToString(),
        QrCodeToken = res.QrCodeToken,
        CancellationReason = res.CancellationReason,
        CancelledAt = res.CancelledAt,
        FinalizedByOperatorNIC = res.FinalizedByOperatorNIC,
        FinalizedAt = res.FinalizedAt,
        OperatorNotes = res.OperatorNotes,
        CreatedAt = res.CreatedAt
    };
    }

    private static SlotResponse MapSlotToResponse(EnergyBookingSlot slot)
    {
    // Inline: Transforms internal EnergyBookingSlot entity into SlotResponse DTO.
    return new SlotResponse
    {
        Id = slot.Id ?? string.Empty,
        StationId = slot.StationId,
        SlotDate = slot.SlotDate,
        StartTime = slot.StartTime,
        EndTime = slot.EndTime,
        MaxCapacityKW = slot.MaxCapacityKW,
        AvailableCapacityKW = slot.AvailableCapacityKW,
        Status = slot.Status.ToString()
    }
    }

```

```

    };
  }
}

```

MongoDbContext.cs

```

//
=====

// File: MongoDbContext.cs
// Project: SolariX - Smart Solar Microgrid Trading System
// Description: Implementation of IMongoDbContext managing MongoDB database connections,
// collections, and index configurations.
// Module: SE4040 Enterprise Application Development
//
=====

using Microsoft.Extensions.Options;
using MongoDB.Driver;
using SolariX.Api.Models;

namespace SolariX.Api.Data
{
    /// <summary>
    /// Encapsulates database connectivity and provides typed collections for the 4 core data
    models.
    /// </summary>
    public class MongoDbContext : IMongoDbContext
    {
        private readonly IMongoDatabase _database;
        private readonly MongoClientSettings _settings;

        public MongoDbContext(IOptions<MongoDbSettings> settings)
        {
            // Inline: Initializes the MongoDB client and database instance based on configured
            connection settings, then ensures required indexes are provisioned.
            _settings = settings.Value;
            var client = new MongoClient(_settings.ConnectionString);

```

```

        _database = client.GetDatabase(_settings.DatabaseName);

        ConfigureIndexes();
    }

    public IMongoCollection<User> Users =>
        _database.GetCollection<User>(_settings.UsersCollectionName);

    public IMongoCollection<SolarStationInfo> SolarStationInfo =>
        _database.GetCollection<SolarStationInfo>(_settings.StationsCollectionName);

    public IMongoCollection<EnergyBookingSlot> EnergyBookingSlots =>
        _database.GetCollection<EnergyBookingSlot>(_settings.SlotsCollectionName);

    public IMongoCollection<EnergyReservation> EnergyReservations =>
        _database.GetCollection<EnergyReservation>(_settings.ReservationsCollectionName);

    /// <summary>
    /// Proactively builds indexes on primary business keys and query filters.
    /// </summary>
    private void ConfigureIndexes()
    {
        // Inline: Creates unique and compound indexes on NIC, email, reservation numbers, and
        // station associations to optimize lookup performance and enforce business uniqueness.
        try
        {
            // Unique index on User NIC
            var userNicIndex = new CreateIndexModel<User>(
                Builders<User>.IndexKeys.Ascending(u => u.NIC),
                new CreateIndexOptions { Unique = true, Sparse = false }
            );
            Users.Indexes.CreateOne(userNicIndex);

            // Index on ReservationNumber
            var resNumberIndex = new CreateIndexModel<EnergyReservation>(
                Builders<EnergyReservation>.IndexKeys.Ascending(r => r.ReservationNumber),
                new CreateIndexOptions { Unique = true }
            );
            EnergyReservations.Indexes.CreateOne(resNumberIndex);
        }
        catch { }
    }

```

```

        // Compound Index on StationId and SlotDate for fast slot availability lookups
        var slotIndex = new CreateIndexModel<EnergyBookingSlot>(
            Builders<EnergyBookingSlot>.IndexKeys.Ascending(s => s.StationId).Ascending(s
=> s.SlotDate)
        );
        EnergyBookingSlots.Indexes.CreateOne(slotIndex);
    }
    catch
    {
        // In scenarios where MongoDB is initializing or offline during build-time tests,
        suppress index exception
    }
}
}
}
}

```

ReservationDtos.cs

```

//
=====

// File: ReservationDtos.cs
// Project: SolariX - Smart Solar Microgrid Trading System
// Description: Data transfer objects for energy reservation lifecycle, cancellations,
modifications, QR verification, and operational dashboards.
// Module: SE4040 Enterprise Application Development
//
=====

using System.ComponentModel.DataAnnotations;
using SolariX.Api.Models;

namespace SolariX.Api.DTOs
{
    public class CreateReservationRequest
    {
        [Required]
        public string ProsumerNIC { get; set; } = string.Empty;
    }
}

```

```

    [Required]
    public string StationId { get; set; } = string.Empty;

    [Required]
    public string SlotId { get; set; } = string.Empty;

    [Range(0.1, 1000.0)]
    public double EnergyAmountKW { get; set; }

    public string TradeType { get; set; } = "DropOff"; // "DropOff" or "Charge"
}

public class UpdateReservationRequest
{
    public string? NewSlotId { get; set; }

    [Range(0.1, 1000.0)]
    public double? NewEnergyAmountKW { get; set; }

    public string? TradeType { get; set; }
}

public class CancelReservationRequest
{
    [Required]
    public string ProsumerNIC { get; set; } = string.Empty;

    public string Reason { get; set; } = "Cancelled by user";
}

/// <summary>
/// Request payload for Grid Operator scanning prosumer QR code and completing trade.
/// </summary>
public class VerifyAndCompleteRequest
{
    [Required]
    public string QrToken { get; set; } = string.Empty;

    [Required]
    public string OperatorId { get; set; } = string.Empty;
}

```

```

    [Range(0.1, 10000.0)]
    public double EnergyDeliveredKWh { get; set; }

    public string? Notes { get; set; }
}

public class FinalizeTransferRequest
{
    [Required]
    public string QrToken { get; set; } = string.Empty;

    [Required]
    public string OperatorNIC { get; set; } = string.Empty;

    public string? Notes { get; set; }
}

public class ReservationResponse
{
    public string Id { get; set; } = string.Empty;
    public string ReservationNumber { get; set; } = string.Empty;
    public string ProsumerNIC { get; set; } = string.Empty;
    public string StationId { get; set; } = string.Empty;
    public string StationName { get; set; } = string.Empty;
    public string SlotId { get; set; } = string.Empty;
    public DateTime ReservationDate { get; set; }
    public DateTime StartTime { get; set; }
    public DateTime EndTime { get; set; }
    public double EnergyAmountKW { get; set; }
    public string TradeType { get; set; } = string.Empty;
    public string Status { get; set; } = string.Empty;
    public string QrCodeToken { get; set; } = string.Empty;
    public string? CancellationReason { get; set; }
    public DateTime? CancelledAt { get; set; }
    public string? FinalizedByOperatorNIC { get; set; }
    public DateTime? FinalizedAt { get; set; }
    public string? OperatorNotes { get; set; }
    public DateTime CreatedAt { get; set; }
}

```

```

public class DashboardSummaryResponse
{
    public int TotalStations { get; set; }
    public int ActiveStations { get; set; }
    public int TotalProsumers { get; set; }
    public int PendingProsumerApprovals { get; set; }
    public int PendingReservationsCount { get; set; }
    public int ApprovedFutureReservationsCount { get; set; }
    public int CompletedReservationsCount { get; set; }
    public int CancelledReservationsCount { get; set; }
    public double TotalEnergyTradedKW { get; set; }
}

```

```

public class ReservationFilterRequest
{
    public string? ProsumerNIC { get; set; }
    public string? StationId { get; set; }
    public ReservationStatus? Status { get; set; }
    public DateTime? FromDate { get; set; }
    public DateTime? ToDate { get; set; }
    public string? SearchQuery { get; set; }
}
}

```

BookingBusinessRulesTests.cs

```

//
=====
=====
// File: BookingBusinessRulesTests.cs
// Project: SolariX - Smart Solar Microgrid Trading System
// Description: Unit tests validating the FAT service business rules: 7-day future window, 12-
hour cancellation/update thresholds, node deactivation conflicts, and QR validation.
// Module: SE4040 Enterprise Application Development
//
=====
=====

```

```

using System.Threading;

```

```

using System.Threading.Tasks;
using MongoDB.Driver;
using Moq;
using SolariX.Api.Data;
using SolariX.Api.DTOs;
using SolariX.Api.Models;
using SolariX.Api.Services;
using Xunit;

namespace SolariX.Tests
{
    public class BookingBusinessRulesTests
    {
        private readonly Mock<IMongoDbContext> _mockContext;
        private readonly Mock<IQRService> _mockQrService;
        private readonly Mock<IMongoCollection<User>> _mockUsers;
        private readonly Mock<IMongoCollection<SolarStationInfo>> _mockStations;
        private readonly Mock<IMongoCollection<EnergyBookingSlot>> _mockSlots;
        private readonly Mock<IMongoCollection<EnergyReservation>> _mockReservations;

        public BookingBusinessRulesTests()
        {
            // Inline: Prepares mock MongoDB context and collections for isolated business rule
            // verification.
            _mockContext = new Mock<IMongoDbContext>();
            _mockQrService = new Mock<IQRService>();

            _mockUsers = new Mock<IMongoCollection<User>>();
            _mockStations = new Mock<IMongoCollection<SolarStationInfo>>();
            _mockSlots = new Mock<IMongoCollection<EnergyBookingSlot>>();
            _mockReservations = new Mock<IMongoCollection<EnergyReservation>>();

            _mockContext.Setup(c => c.Users).Returns(_mockUsers.Object);
            _mockContext.Setup(c => c.SolarStationInfo).Returns(_mockStations.Object);
            _mockContext.Setup(c => c.EnergyBookingSlots).Returns(_mockSlots.Object);
            _mockContext.Setup(c => c.EnergyReservations).Returns(_mockReservations.Object);
        }

        [Fact]
        public void QRService_GeneratesAndValidates_TamperProofPayload()

```

```

    {
        // Inline: Verifies that QRService generates a valid signed payload with { ReservationId,
        ProsumerNIC, StationId, SlotId, Timestamp, SignatureHash } and detects tampering.
        var qrService = new QRService();
        var reservationId = "66ea1234567890abcdef1234";
        var prosumerNic = "200012345678";
        var stationId = "station123";
        var slotId = "slot456";
        var slotStartTime = DateTime.UtcNow.AddDays(2);

        var payload = qrService.GenerateQrPayload(reservationId, prosumerNic, stationId,
        slotId, slotStartTime);
        Assert.NotNull(payload);
        Assert.Contains("#", payload);

        // Validate authentic payload
        bool isValid = qrService.ValidateQrPayload(payload, out var extractedResId, out var
        extractedNic, out var extractedStationId, out var extractedSlotId, out _);
        Assert.True(isValid);
        Assert.Equal(reservationId, extractedResId);
        Assert.Equal(prosumerNic, extractedNic);
        Assert.Equal(stationId, extractedStationId);
        Assert.Equal(slotId, extractedSlotId);

        // Validate tampered payload fails
        var tamperedPayload = payload.Replace(prosumerNic, "999999999999");
        bool isTamperedValid = qrService.ValidateQrPayload(tamperedPayload, out _, out _, out
        _, out _, out _);
        Assert.False(isTamperedValid);
    }

    [Fact]
    public async Task CancelReservation_Fails_WhenLessThan12HoursRemaining()
    {
        // Inline: Enforces the 12-hour rule: attempting to cancel a slot scheduled in 5 hours must
        throw an InvalidOperationException.
        var reservationId = "res_001";
        var prosumerNic = "200012345678";
        var now = DateTime.UtcNow;

```

```

// Reservation scheduled only 5 hours from now (< 12 hours)
var reservation = new EnergyReservation
{
    Id = reservationId,
    ReservationNumber = "RES-001",
    ProsumerNIC = prosumerNic,
    StationId = "station_001",
    SlotId = "slot_001",
    ReservationDate = now.Date,
    StartTime = now.AddHours(5),
    EndTime = now.AddHours(6),
    EnergyAmountKW = 10,
    Status = ReservationStatus.Approved
};

var mockCursor = new Mock<IAsyncCursor<EnergyReservation>>();
mockCursor.SetupSequence(c =>
c.MoveNext(It.IsAny<CancellationToken>())).Returns(true).Returns(false);
mockCursor.SetupSequence(c =>
c.MoveNextAsync(It.IsAny<CancellationToken>())).ReturnsAsync(true).ReturnsAsync(false);
mockCursor.Setup(c => c.Current).Returns(new List<EnergyReservation> { reservation
});

_mockReservations
    .Setup(r => r.FindAsync(
        It.IsAny<FilterDefinition<EnergyReservation>>(),
        It.IsAny<FindOptions<EnergyReservation, EnergyReservation>>(),
        It.IsAny<CancellationToken>()))
    .ReturnsAsync(mockCursor.Object);

var bookingService = new BookingService(_mockContext.Object,
_mockQrService.Object);

var cancelRequest = new CancelReservationRequest
{
    ProsumerNIC = prosumerNic,
    Reason = "Need to cancel"
};

var exception = await Assert.ThrowsAsync<InvalidOperationException>(() =>

```

```

        bookingService.CancelReservationAsync(reservationId, cancelRequest));

    Assert.Contains("at least 12 hours' advance notice", exception.Message);
}

[Fact]
public async Task UpdateReservation_Fails_WhenLessThan12HoursRemaining()
{
    // Inline: Enforces the 12-hour rule: attempting to modify a slot scheduled in 3 hours must
    throw an InvalidOperationException.
    var reservationId = "res_002";
    var prosumerNic = "200012345678";
    var now = DateTime.UtcNow;

    var reservation = new EnergyReservation
    {
        Id = reservationId,
        ReservationNumber = "RES-002",
        ProsumerNIC = prosumerNic,
        StationId = "station_001",
        SlotId = "slot_001",
        ReservationDate = now.Date,
        StartTime = now.AddHours(3),
        EndTime = now.AddHours(4),
        EnergyAmountKW = 10,
        Status = ReservationStatus.Approved
    };

    var mockCursor = new Mock<IAsyncCursor<EnergyReservation>>();
    mockCursor.SetupSequence(c =>
    c.MoveNext(It.IsAny<CancellationToken>())).Returns(true).Returns(false);
    mockCursor.SetupSequence(c =>
    c.MoveNextAsync(It.IsAny<CancellationToken>())).ReturnsAsync(true).ReturnsAsync(false);
    mockCursor.Setup(c => c.Current).Returns(new List<EnergyReservation> { reservation
});

    _mockReservations
        .Setup(r => r.FindAsync(
            It.IsAny<FilterDefinition<EnergyReservation>>()),
            It.IsAny<FindOptions<EnergyReservation, EnergyReservation>>()),

```

```

        It.IsAny<CancellationToken>()))
        .ReturnsAsync(mockCursor.Object);

        var bookingService = new BookingService(_mockContext.Object,
        _mockQrService.Object);

        var updateRequest = new UpdateReservationRequest
        {
            NewEnergyAmountKW = 15.0
        };

        var exception = await Assert.ThrowsAsync<InvalidOperationException>(() =>
            bookingService.UpdateReservationAsync(reservationId, prosumerNic,
            updateRequest));

        Assert.Contains("at least 12 hours' advance notice", exception.Message);
    }

    [Fact]
    public async Task CreateReservation_Fails_WhenBookingBeyond7DayFutureWindow()
    {
        // Inline: Enforces the 7-day rule: attempting to schedule a booking 10 days in the future
        must throw an ArgumentOutOfRangeException.
        var prosumerNic = "200012345678";
        var stationId = "station_001";
        var slotId = "slot_001";
        var now = DateTime.UtcNow;

        // User active
        var prosumer = new User { NIC = prosumerNic, Status = AccountStatus.Active, Role =
        UserRole.Prosumer };
        var mockUserCursor = new Mock<IAsyncCursor<User>>();
        mockUserCursor.SetupSequence(c =>
        c.MoveNext(It.IsAny<CancellationToken>())).Returns(true).Returns(false);
        mockUserCursor.SetupSequence(c =>
        c.MoveNextAsync(It.IsAny<CancellationToken>())).ReturnsAsync(true).ReturnsAsync(false);
        mockUserCursor.Setup(c => c.Current).Returns(new List<User> { prosumer });
        _mockUsers.Setup(u => u.FindAsync(It.IsAny<FilterDefinition<User>>(),
        It.IsAny<FindOptions<User, User>>(), It.IsAny<CancellationToken>()))
        .ReturnsAsync(mockUserCursor.Object);
    }

```

```

        // Station active
        var station = new SolarStationInfo { Id = stationId, StationName = "Main Hub", IsActive
= true };
        var mockStationCursor = new Mock<IAsyncCursor<SolarStationInfo>>();
        mockStationCursor.SetupSequence(c =>
c.MoveNext(It.IsAny<CancellationToken>())).Returns(true).Returns(false);
        mockStationCursor.SetupSequence(c =>
c.MoveNextAsync(It.IsAny<CancellationToken>())).ReturnsAsync(true).ReturnsAsync(false);
        mockStationCursor.Setup(c => c.Current).Returns(new List<SolarStationInfo> { station
});
        _mockStations.Setup(s => s.FindAsync(It.IsAny<FilterDefinition<SolarStationInfo>>(),
It.IsAny<FindOptions<SolarStationInfo, SolarStationInfo>>(), It.IsAny<CancellationToken>()))
.ReturnsAsync(mockStationCursor.Object);

        // Slot scheduled 10 days in future (> 7 days)
        var futureSlot = new EnergyBookingSlot
        {
            Id = slotId,
            StationId = stationId,
            SlotDate = now.Date.AddDays(10),
            StartTime = now.Date.AddDays(10).AddHours(10),
            EndTime = now.Date.AddDays(10).AddHours(11),
            AvailableCapacityKW = 50,
            Status = SlotStatus.Available
        };
        var mockSlotCursor = new Mock<IAsyncCursor<EnergyBookingSlot>>();
        mockSlotCursor.SetupSequence(c =>
c.MoveNext(It.IsAny<CancellationToken>())).Returns(true).Returns(false);
        mockSlotCursor.SetupSequence(c =>
c.MoveNextAsync(It.IsAny<CancellationToken>())).ReturnsAsync(true).ReturnsAsync(false);
        mockSlotCursor.Setup(c => c.Current).Returns(new List<EnergyBookingSlot> {
futureSlot });
        _mockSlots.Setup(s => s.FindAsync(It.IsAny<FilterDefinition<EnergyBookingSlot>>(),
It.IsAny<FindOptions<EnergyBookingSlot, EnergyBookingSlot>>(),
It.IsAny<CancellationToken>()))
.ReturnsAsync(mockSlotCursor.Object);

        var bookingService = new BookingService(_mockContext.Object,
_mockQrService.Object);

```

```

var request = new CreateReservationRequest
{
    ProsumerNIC = prosumerNic,
    StationId = stationId,
    SlotId = slotId,
    EnergyAmountKW = 10,
    TradeType = "DropOff"
};

var exception = await Assert.ThrowsAsync<ArgumentOutOfRangeException>(() =>
    bookingService.CreateReservationAsync(request));

    Assert.Contains("Power trading reservations must be scheduled within 7 days",
exception.Message);
}

[Fact]
public async Task DeactivateStation_Fails_WhenActiveReservationsExist()
{
    // Inline: Enforces node deactivation rule: if active reservations exist for the solar hub,
deactivation must be rejected with an InvalidOperationException.
    var stationId = "station_active_001";
    var station = new SolarStationInfo
    {
        Id = stationId,
        StationName = "Kaduwela Hub",
        IsActive = true
    };

    var mockStationCursor = new Mock<IAsyncCursor<SolarStationInfo>>();
    mockStationCursor.SetupSequence(c =>
c.MoveNext(It.IsAny<CancellationToken>())).Returns(true).Returns(false);
    mockStationCursor.SetupSequence(c =>
c.MoveNextAsync(It.IsAny<CancellationToken>())).ReturnsAsync(true).ReturnsAsync(false);
    mockStationCursor.Setup(c => c.Current).Returns(new List<SolarStationInfo> { station
});

    _mockStations.Setup(s => s.FindAsync(It.IsAny<FilterDefinition<SolarStationInfo>>(),
It.IsAny<FindOptions<SolarStationInfo, SolarStationInfo>>(), It.IsAny<CancellationToken>()))
        .ReturnsAsync(mockStationCursor.Object);

```

```

        _mockReservations
            .Setup(r =>
r.CountDocumentsAsync(It.IsAny<FilterDefinition<EnergyReservation>>()),
It.IsAny<CountOptions>(), It.IsAny<CancellationToken>()))
            .ReturnsAsync(3);

var stationService = new StationService(_mockContext.Object);

var exception = await Assert.ThrowsAsync<InvalidOperationException>(() =>
    stationService.DeactivateStationAsync(stationId));

Assert.Contains("Deactivation blocked", exception.Message);
Assert.Contains("3 active or approved reservation(s)", exception.Message);
}

[Fact]
public async Task ReactivateAccount_Fails_WhenNonBackofficeUserAttempts()
{
    // Inline: Enforces prosumer lifecycle rule: only Backoffice officers can reactivate
    accounts; non-backoffice attempts must throw UnauthorizedAccessException.
    var operatorNic = "GO2000000002";
    var targetProsumerNic = "200012345678";

    var gridOperator = new User
    {
        NIC = operatorNic,
        FullName = "Field Operator",
        Role = UserRole.GridOperator,
        Status = AccountStatus.Active
    };

    var mockUserCursor = new Mock<IAsyncCursor<User>>();
    mockUserCursor.SetupSequence(c =>
c.MoveNext(It.IsAny<CancellationToken>())).Returns(true).Returns(false);
    mockUserCursor.SetupSequence(c =>
c.MoveNextAsync(It.IsAny<CancellationToken>())).ReturnsAsync(true).ReturnsAsync(false);
    mockUserCursor.Setup(c => c.Current).Returns(new List<User> { gridOperator });
    _mockUsers.Setup(u => u.FindAsync(It.IsAny<FilterDefinition<User>>(),
It.IsAny<FindOptions<User, User>>(), It.IsAny<CancellationToken>()))

```

```

        .ReturnsAsync(mockUserCursor.Object);

var userService = new UserService(_mockContext.Object);

var exception = await Assert.ThrowsAsync<UnauthorizedAccessException>(() =>
    userService.ReactivateAccountByBackofficeAsync(targetProsumerNic, operatorNic));

    Assert.Contains("Only Backoffice officers are authorized to reactivate",
exception.Message);
    }
    }
}

```

App.jsx

```

import React from 'react';
import { BrowserRouter as Router, Routes, Route, Navigate } from 'react-router-dom';
import { AuthProvider } from './context/AuthContext';
import Navbar from './components/common/Navbar';
import ProtectedRoute from './components/common/ProtectedRoute';

// Pages
import Home from './pages/Home';
import Login from './pages/Login';
import Unauthorized from './pages/Unauthorized';
import BackofficeDashboard from './pages/backoffice/BackofficeDashboard';
import ProsumerManagement from './pages/backoffice/ProsumerManagement';
import NodeManagement from './pages/backoffice/NodeManagement';
import OperatorDashboard from './pages/operator/OperatorDashboard';
import SlotManagement from './pages/operator/SlotManagement';

function App() {
  return (
    <Router>
      <AuthProvider>
        <div className="min-h-screen bg-slate-950 text-slate-100 flex flex-col">
          <Navbar />
          <main className="flex-1">
            <Routes>
              {/* Public Routes */}

```

```

<Route path="/" element={<Home />} />
<Route path="/login" element={<Login />} />
<Route path="/unauthorized" element={<Unauthorized />} />

```

```

{/* Backoffice Admin Routes */}

```

```

<Route
  path="/backoffice/dashboard"
  element={
    <ProtectedRoute allowedRoles={['Backoffice']}>
      <BackofficeDashboard />
    </ProtectedRoute>
  }
/>

```

```

<Route
  path="/backoffice/prosumers"
  element={
    <ProtectedRoute allowedRoles={['Backoffice']}>
      <ProsumerManagement />
    </ProtectedRoute>
  }
/>

```

```

<Route
  path="/backoffice/nodes"
  element={
    <ProtectedRoute allowedRoles={['Backoffice']}>
      <NodeManagement />
    </ProtectedRoute>
  }
/>

```

```

{/* Grid Operator Routes */}

```

```

<Route
  path="/operator/dashboard"
  element={
    <ProtectedRoute allowedRoles={['GridOperator']}>
      <OperatorDashboard />
    </ProtectedRoute>
  }
/>

```

```

<Route

```

```

        path="/operator/slots"
        element={
          <ProtectedRoute allowedRoles={['GridOperator']}>
            <SlotManagement />
          </ProtectedRoute>
        }
      />

      { /* Fallback Catch-All */}
      <Route path="*" element={<Navigate to="/" replace />} />
    </Routes>
  </main>
</div>
</AuthProvider>
</Router>
);
}

```

```
export default App;
```

main.jsx

```

import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App.jsx';
import './index.css';

```

```

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
);

```

index.css

```

@tailwind base;
@tailwind components;
@tailwind utilities;

```

```
@layer base {
```

```

body {
  @apply bg-slate-950 text-slate-100 antialiased overflow-x-hidden;
}

/* Custom glassmorphism and glow utilities */
.glass-panel {
  background: rgba(15, 23, 42, 0.75);
  backdrop-filter: blur(12px);
  -webkit-backdrop-filter: blur(12px);
  border: 1px solid rgba(255, 255, 255, 0.08);
}

.glass-panel-hover {
  transition: all 0.2s ease-in-out;
}

.glass-panel-hover:hover {
  background: rgba(30, 41, 59, 0.85);
  border-color: rgba(245, 158, 11, 0.3);
  box-shadow: 0 10px 25px -5px rgba(245, 158, 11, 0.1);
}

.solar-gradient {
  background: linear-gradient(135deg, #f59e0b 0%, #d97706 50%, #b45309 100%);
}

.solar-glow {
  box-shadow: 0 0 25px -5px rgba(245, 158, 11, 0.4);
}

.grid-glow {
  box-shadow: 0 0 25px -5px rgba(34, 197, 94, 0.3);
}

/* Custom modern scrollbar */
::-webkit-scrollbar {
  width: 8px;
  height: 8px;
}

```

```
::-webkit-scrollbar-track {  
  background: #020617;  
}  
  
::-webkit-scrollbar-thumb {  
  background: #334155;  
  border-radius: 4px;  
}  
  
::-webkit-scrollbar-thumb:hover {  
  background: #475569;  
}
```

axiosClient.js

```
import axios from 'axios';  
  
const API_BASE_URL = import.meta.env.VITE_API_BASE_URL || 'http://localhost:5000/api';  
  
const axiosClient = axios.create({  
  baseURL: API_BASE_URL,  
  headers: {  
    'Content-Type': 'application/json',  
  },  
  timeout: 15000,  
});  
  
// Request interceptor: attach token from local storage if present  
axiosClient.interceptors.request.use(  
  (config) => {  
    const token = localStorage.getItem('solarix_token');  
    if (token) {  
      config.headers.Authorization = `Bearer ${token}`;  
    }  
    return config;  
  },  
  (error) => Promise.reject(error)  
);
```

```
// Response interceptor: extract clean error message from FAT service responses
axiosClient.interceptors.response.use(
  (response) => response.data,
  (error) => {
    let message = 'An unexpected network error occurred.';
    if (error.response?.data) {
      if (typeof error.response.data === 'string') {
        message = error.response.data;
      } else if (error.response.data.message) {
        message = error.response.data.message;
      } else if (error.response.data.error) {
        message = `${error.response.data.error}: ${error.response.data.message || ''}`;
      } else if (error.response.data.title) {
        message = error.response.data.title;
      }
    } else if (error.message) {
      message = error.message;
    }
    return Promise.reject(new Error(message));
  }
);

export default axiosClient;
```

BackofficeDashboard.jsx

```
import React, { useEffect, useState } from 'react';
import { Link } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';
import { reservationsApi } from '../api/reservationsApi';
import { stationsApi } from '../api/stationsApi';
import { usersApi } from '../api/usersApi';
import StatCard from '../components/common/StatCard';
import Toast from '../components/common/Toast';
import {
  Shield,
  Cpu,
  Users,
  Battery,
  AlertTriangle,
}
```

```

    ArrowUpRight,
    RefreshCw,
    Zap,
    Radio,
    CheckCircle2,
  } from 'lucide-react';

const BackofficeDashboard = () => {
  const { user } = useAuth();
  const [summary, setSummary] = useState(null);
  const [recentStations, setRecentStations] = useState([]);
  const [pendingProsumers, setPendingProsumers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [toast, setToast] = useState(null);

  const fetchDashboardData = async () => {
    setLoading(true);
    try {
      const [sumData, stationsData, pendingData] = await Promise.all([
        reservationsApi.getDashboardSummary(),
        stationsApi.getStations(),
        usersApi.getPendingProsumers(),
      ]);

      setSummary(sumData);
      setRecentStations(stationsData.slice(0, 4));
      setPendingProsumers(pendingData.slice(0, 5));
    } catch (err) {
      setToast({ type: 'error', message: err.message || 'Failed to load dashboard metrics.' });
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => {
    fetchDashboardData();
  }, []);

  return (
    <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-8">

```

```

    {toast && <Toast type={toast.type} message={toast.message} onClose={() =>
setToast(null)} />}

    {/* Greeting Banner */}
    <div className="glass-panel rounded-3xl p-6 sm:p-8 border border-slate-800 shadow-xl
relative overflow-hidden mb-8">
      <div className="absolute top-0 right-0 w-80 h-80 bg-amber-500/10 blur-3xl rounded-full
pointer-events-none" />

      <div className="flex flex-col md:flex-row md:items-center justify-between gap-4">
        <div>
          <div className="inline-flex items-center space-x-2 px-3 py-1 rounded-full text-xs font-
semibold bg-amber-500/15 text-amber-400 border border-amber-500/30 mb-2">
            <Shield className="w-3.5 h-3.5" />
            <span>Backoffice Administrator Portal</span>
          </div>
          <h1 className="text-2xl sm:text-3xl font-black text-white tracking-tight">
            Welcome back, {user?.fullName || 'Administrator'}
          </h1>
          <p className="text-xs text-slate-400 mt-1">
            Centralized administration: manage prosumer lifecycle, configure microgrid hubs, and
monitor active trading.
          </p>
        </div>

        <div className="flex items-center space-x-3">
          <button
            onClick={fetchDashboardData}
            className="flex items-center space-x-1.5 px-3 py-2 rounded-xl text-xs font-semibold
bg-slate-900 text-slate-300 hover:text-white border border-slate-800 hover:border-slate-700
transition-colors"
            >
            <RefreshCw className={`w-3.5 h-3.5 ${loading ? 'animate-spin' : ''}`} />
            <span>Refresh Metrics</span>
          </button>

          <Link
            to="/backoffice/nodes"
            className="flex items-center space-x-1.5 px-4 py-2 rounded-xl text-xs font-bold text-
slate-950 solar-gradient hover:opacity-95 transition-opacity shadow-md"

```

```

    >
    <Cpu className="w-3.5 h-3.5" />
    <span>Manage Nodes</span>
  </Link>
</div>
</div>
</div>

{/* KPI Metric Cards Grid */}
<div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4 gap-6 mb-8">
  <StatCard
    title="Active Microgrid Hubs"
    value={summary ? `${summary.activeStations} / ${summary.totalStations}` : '--'}
    icon={Cpu}
    color="amber"
    subtitle="Operational solar nodes"
    badge="Hubs Online"
  />

  <StatCard
    title="Total Prosumers"
    value={summary ? summary.totalProsumers : '--'}
    icon={Users}
    color="green"
    subtitle={
      summary?.pendingProsumerApprovals > 0
        ? `${summary.pendingProsumerApprovals} awaiting approval`
        : 'All accounts verified'
    }
    badge={summary?.pendingProsumerApprovals > 0 ? 'Action Needed' : 'Normal'}
  />

  <StatCard
    title="Approved Future Bookings"
    value={summary ? summary.approvedFutureReservationsCount : '--'}
    icon={Battery}
    color="blue"
    subtitle="Scheduled within 7-day window"
    badge="Active Window"
  />

```

```

<StatCard
  title="Total Power Traded"
  value={summary ? `${summary.totalEnergyTradedKW.toFixed(1)} kW` : '--'}
  icon={Zap}
  color="purple"
  subtitle="From finalized transfers"
  badge="Delivered"
/>
</div>

{/* Two Columns: Actionable Lists */}
<div className="grid grid-cols-1 lg:grid-cols-2 gap-8">
  {/* Pending Prosumers Approvals Panel */}
  <div className="glass-panel rounded-2xl p-6 border border-slate-800 shadow-xl">
    <div className="flex items-center justify-between pb-4 border-b border-slate-800">
      <div>
        <h2 className="text-base font-bold text-white flex items-center gap-2">
          <Users className="w-4 h-4 text-amber-400" />
          Pending Prosumer Approvals
        </h2>
        <p className="text-xs text-slate-400 mt-0.5">
          New accounts requiring Backoffice identity verification.
        </p>
      </div>
      <Link
        to="/backoffice/prosumers"
        className="text-xs text-amber-400 hover:text-amber-300 font-semibold flex items-
center gap-1 "
      >
        <span>View All</span>
        <ArrowUpRight className="w-3.5 h-3.5" />
      </Link>
    </div>

    <div className="mt-4 space-y-3">
      {pendingProsumers.length === 0 ? (
        <div className="text-center py-8 text-xs text-slate-500 flex flex-col items-center">
          <CheckCircle2 className="w-8 h-8 text-emerald-400/60 mb-2" />
          <span>No pending prosumer registrations. All profiles are processed.</span>
        </div>
      ) : (

```

```

    </div>
  ) : (
    pendingProsumers.map((p) => (
      <div
        key={p.nic}
        className="bg-slate-900/80 border border-slate-800 rounded-xl p-3.5 flex items-
center justify-between hover:border-slate-700 transition-colors"
      >
        <div>
          <div className="text-xs font-bold text-white">{p.fullName}</div>
          <div className="text-[11px] font-mono text-amber-400 mt-0.5">NIC:
{p.nic}</div>
          <div className="text-[11px] text-slate-400 mt-0.5">{p.email}</div>
        </div>
        <Link
          to="/backoffice/prosumers"
          className="px-3 py-1.5 rounded-lg text-xs font-bold bg-amber-500/20 text-amber-
300 hover:bg-amber-500/30 border border-amber-500/40 transition-colors"
        >
          Review
        </Link>
      </div>
    ))
  )}
</div>
</div>

```

```

{ /* Microgrid Hubs Quick Overview */ }
<div className="glass-panel rounded-2xl p-6 border border-slate-800 shadow-xl">
  <div className="flex items-center justify-between pb-4 border-b border-slate-800">
    <div>
      <h2 className="text-base font-bold text-white flex items-center gap-2">
        <Cpu className="w-4 h-4 text-emerald-400" />
        Microgrid Hub Status
      </h2>
      <p className="text-xs text-slate-400 mt-0.5">
        Solar stations with active battery bay capacity.
      </p>
    </div>
    <Link

```

```

      to="/backoffice/nodes"
      className="text-xs text-emerald-400 hover:text-emerald-300 font-semibold flex items-
center gap-1"
    >
      <span>Manage Nodes</span>
      <ArrowUpRight className="w-3.5 h-3.5" />
    </Link>
  </div>

  <div className="mt-4 space-y-3">
    {recentStations.length === 0 ? (
      <div className="text-center py-8 text-xs text-slate-500">
        No microgrid stations currently registered.
      </div>
    ) : (
      recentStations.map((s) => (
        <div
          key={s.id}
          className="bg-slate-900/80 border border-slate-800 rounded-xl p-3.5 flex items-
center justify-between hover:border-slate-700 transition-colors"
        >
          <div>
            <div className="flex items-center space-x-2">
              <span className="text-xs font-bold text-white">{s.stationName}</span>
              <span className="text-[10px] font-mono px-1.5 py-0.5 rounded bg-slate-800 text-
slate-400">
                {s.stationCode}
              </span>
            </div>
            <div className="text-[11px] text-slate-400 mt-0.5">{s.locationName}</div>
          </div>

          <div className="text-right">
            <div className="text-xs font-bold text-emerald-400">
              {s.availableBatterySlots} / {s.totalBatterySlots} Slots
            </div>
            <div className="text-[10px] text-slate-500 mt-0.5">
              {s.isActive ? (
                <span className="text-emerald-400">● Online</span>
              ) : (

```

```

        <span className="text-rose-400">● Deactivated</span>
      )}
    </div>
  </div>
</div>
))
)}
</div>
</div>
</div>
</div>
);
};

```

```
export default BackofficeDashboard;
```

NodeManagement.jsx

```

import React, { useEffect, useState } from 'react';
import { stationsApi } from '../api/stationsApi';
import StationModal from '../components/backoffice/StationModal';
import Toast from '../components/common/Toast';
import {
  Cpu,
  Plus,
  MapPin,
  Battery,
  Clock,
  Ban,
  RefreshCw,
  Zap,
  Edit,
  AlertTriangle,
  CheckCircle,
} from 'lucide-react';

const NodeManagement = () => {
  const [stations, setStations] = useState([]);
  const [loading, setLoading] = useState(false);
  const [toast, setToast] = useState(null);

```

```

// Modal State
const [isModalOpen, setIsModalOpen] = useState(false);
const [editingStation, setEditingStation] = useState(null);
const [saving, setSaving] = useState(false);

const fetchStations = async () => {
  setLoading(true);
  try {
    const data = await stationsApi.getStations();
    setStations(data);
  } catch (err) {
    setToast({ type: 'error', message: err.message || 'Failed to fetch microgrid hubs.' });
  } finally {
    setLoading(false);
  }
};

useEffect(() => {
  fetchStations();
}, []);

const handleCreateOrUpdate = async (formData) => {
  setSaving(true);
  try {
    if (editingStation) {
      await stationsApi.updateStation(editingStation.id, formData);
      setToast({ type: 'success', message: `Station '${formData.stationName}' updated successfully.` });
    } else {
      await stationsApi.createStation(formData);
      setToast({ type: 'success', message: `New solar hub '${formData.stationName}' registered successfully.` });
    }
    setIsModalOpen(false);
    setEditingStation(null);
    fetchStations();
  } catch (err) {
    setToast({ type: 'error', message: err.message || 'Operation failed.' });
  } finally {

```

```

    setSaving(false);
  }
};

const handleDeactivate = async (station) => {
  const confirmed = window.confirm(
    `Are you sure you want to deactivate station node '${station.stationName}'
    (${station.stationCode})? This action will check for active reservations first.`
  );
  if (!confirmed) return;

  try {
    // Calls POST /api/stations/{id}/deactivate
    const res = await stationsApi.deactivateStation(station.id);
    setToast({
      type: 'success',
      message: res.message || `Station '${station.stationName}' was successfully deactivated.`,
    });
    fetchStations();
  } catch (err) {
    // FAT service compliance: Catch 400 Bad Request if active reservations exist and display
    exact server message in toast!
    setToast({
      type: 'error',
      message: err.message || 'Deactivation blocked by FAT service business rule.',
    });
  }
};

const handleReactivate = async (station) => {
  try {
    const res = await stationsApi.reactivateStation(station.id);
    setToast({
      type: 'success',
      message: res.message || `Station '${station.stationName}' reactivated successfully.`,
    });
    fetchStations();
  } catch (err) {
    setToast({ type: 'error', message: err.message || 'Reactivation failed.' });
  }
}

```

```

    };

    const openCreateModal = () => {
      setEditingStation(null);
      setIsModalOpen(true);
    };

    const openEditModal = (station) => {
      setEditingStation(station);
      setIsModalOpen(true);
    };

    return (
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-8">
        {toast && <Toast type={toast.type} message={toast.message} onClose={() =>
setToast(null)} />}
        </div>

        <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-4 mb-8">
          <div>
            <h1 className="text-2xl font-black text-white tracking-tight flex items-center gap-2.5">
              <Cpu className="w-6 h-6 text-amber-400" />
              Solar Microgrid Node Management
            </h1>
            <p className="text-xs text-slate-400 mt-1">
              Configure regional microgrid hubs, GPS positioning, battery slot capacity specs, and
              enforce active-reservation deactivation guards.
            </p>
          </div>

          <button
            onClick={openCreateModal}
            className="flex items-center space-x-2 px-4 py-2.5 rounded-xl text-xs font-bold text-
            slate-950 solar-gradient hover:opacity-95 transition-all shadow-lg self-start sm:self-auto
            hover:scale-105"
            >
            <Plus className="w-4 h-4" />
            <span>Add New Grid Hub</span>
          </button>
        </div>

```

```

    { /* Stations List Grid */
    { loading ? (
      <div className="flex items-center justify-center py-20 text-slate-400 text-xs">
        <RefreshCw className="w-6 h-6 animate-spin text-amber-400 mr-2" />
        <span>Loading solar microgrid nodes from FAT service...</span>
      </div>
    ) : stations.length === 0 ? (
      <div className="glass-panel rounded-3xl p-12 text-center text-slate-400">
        <Cpu className="w-12 h-12 text-slate-600 mx-auto mb-3" />
        <h3 className="text-base font-bold text-white">No Microgrid Nodes Found</h3>
        <p className="text-xs text-slate-500 mt-1">Click "Add New Grid Hub" to register the
first station.</p>
      </div>
    ) : (
      <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
        {stations.map((s) => (
          <div
            key={s.id}
            className={`glass-panel rounded-2xl p-6 border transition-all duration-300 shadow-xl
$ {
      s.isActive
        ? 'border-slate-800 hover:border-amber-500/40'
        : 'border-rose-950/60 bg-rose-950/10'
      }`}
          >
            { /* Card Top */
            <div className="flex items-start justify-between">
              <div>
                <span className="text-[10px] font-mono font-bold px-2 py-0.5 rounded bg-slate-
800 text-amber-400 border border-slate-700">
                  {s.stationCode}
                </span>
              </div>
              <h3 className="text-base font-bold text-white mt-1.5">{s.stationName}</h3>
            </div>

            {s.isActive ? (
              <span className="inline-flex items-center px-2.5 py-0.5 rounded-full text-[11px]
font-semibold bg-emerald-500/15 text-emerald-400 border border-emerald-500/30">

```

```

    <span className="w-1.5 h-1.5 rounded-full bg-emerald-400 mr-1.5 animate-
pulse"></span>
    Active
  </span>
): (
  <span className="inline-flex items-center px-2.5 py-0.5 rounded-full text-[11px]
font-semibold bg-rose-500/15 text-rose-400 border border-rose-500/30">
    Deactivated
  </span>
)}
</div>

```

```

{/* Location & GPS */}
<div className="mt-4 text-xs space-y-1.5 text-slate-300">
  <div className="flex items-center space-x-2 text-slate-400">
    <MapPin className="w-3.5 h-3.5 text-amber-400 shrink-0" />
    <span className="truncate">{s.locationName}</span>
  </div>
  <div className="text-[11px] font-mono text-slate-400 pl-5">
    GPS: {s.latitude.toFixed(4)}, {s.longitude.toFixed(4)}
  </div>
</div>

```

```

{/* Specs Ribbon */}
<div className="mt-5 pt-4 border-t border-slate-800/80 grid grid-cols-2 gap-3 text-
xs">
  <div>
    <span className="text-[11px] text-slate-400 block">Hub Capacity:</span>
    <span className="font-bold text-amber-300 flex items-center gap-1 mt-0.5">
      <Zap className="w-3.5 h-3.5" />
      {s.capacityKWh} kW/h
    </span>
  </div>
  <div>
    <span className="text-[11px] text-slate-400 block">Battery Storage:</span>
    <span className="font-bold text-emerald-400 flex items-center gap-1 mt-0.5">
      <Battery className="w-3.5 h-3.5" />
      {s.availableBatterySlots} / {s.totalBatterySlots} Slots
    </span>
  </div>
</div>

```

```

</div>

{/* Operational Hours */}
<div className="mt-3 flex items-center space-x-1.5 text-[11px] text-slate-400">
  <Clock className="w-3.5 h-3.5 text-slate-500" />
  <span>Hours: {s.operationalHours}</span>
</div>

{/* Action Buttons */}
<div className="mt-6 pt-4 border-t border-slate-800 flex items-center justify-
between">
  <button
    onClick={() => openEditModal(s)}
    className="flex items-center space-x-1 px-3 py-1.5 rounded-lg text-xs font-
semibold bg-slate-900 text-slate-300 hover:text-white border border-slate-800 hover:border-
slate-700 transition-colors"
  >
    <Edit className="w-3.5 h-3.5" />
    <span>Edit Hub</span>
  </button>

  {s.isActive ? (
    <button
      onClick={() => handleDeactivate(s)}
      title="Queries active reservations before proceeding; blocked if bookings exist"
      className="flex items-center space-x-1 px-3 py-1.5 rounded-lg text-xs font-
semibold bg-rose-500/15 text-rose-300 hover:bg-rose-500/25 border border-rose-500/30
transition-colors"
    >
      <Ban className="w-3.5 h-3.5" />
      <span>Deactivate</span>
    </button>
  ) : (
    <button
      onClick={() => handleReactivate(s)}
      className="flex items-center space-x-1 px-3 py-1.5 rounded-lg text-xs font-
semibold bg-emerald-500/15 text-emerald-300 hover:bg-emerald-500/25 border border-emerald-
500/30 transition-colors"
    >
      <CheckCircle className="w-3.5 h-3.5" />

```

```

        <span>Reactivate</span>
      </button>
    )}
  </div>
</div>
)})
</div>
)}

{/* Station Modal */}
<StationModal
  isOpen={isModalOpen}
  onClose={() => setIsModalOpen(false)}
  onSubmit={handleCreateOrUpdate}
  initialData={editingStation}
  loading={saving}
/>
</div>
);
};

```

```
export default NodeManagement;
```

ProsumerManagement.jsx

```

import React, { useEffect, useState } from 'react';
import { useAuth } from '../context/AuthContext';
import { usersApi } from '../api/usersApi';
import { authApi } from '../api/authApi';
import ProsumerTable from '../components/backoffice/ProsumerTable';
import Toast from '../components/common/Toast';
import Modal from '../components/common/Modal';
import { UserPlus, Shield, Zap } from 'lucide-react';

const ProsumerManagement = () => {
  const { user } = useAuth();
  const [prosumers, setProsumers] = useState([]);
  const [loading, setLoading] = useState(false);
  const [toast, setToast] = useState(null);

```

```

// New Prosumer Modal State
const [isRegisterOpen, setIsRegisterOpen] = useState(false);
const [registerForm, setRegisterForm] = useState({
  nic: "",
  fullName: "",
  email: "",
  phoneNumber: "",
  password: 'Password@123',
  address: "",
  solarCapacityKW: 5.0,
});
const [registering, setRegistering] = useState(false);

const fetchProsumers = async () => {
  setLoading(true);
  try {
    const data = await usersApi.getUsers('Prosumer');
    setProsumers(data);
  } catch (err) {
    setToast({ type: 'error', message: err.message || 'Failed to fetch prosumers from API.' });
  } finally {
    setLoading(false);
  }
};

useEffect(() => {
  fetchProsumers();
}, []);

const handleApprove = async (prosumerNic) => {
  try {
    await usersApi.approveProsumer(prosumerNic, user.nic);
    setToast({
      type: 'success',
      message: `Prosumer '${prosumerNic}' successfully approved and activated.`,
    });
    fetchProsumers();
  } catch (err) {
    setToast({ type: 'error', message: err.message || 'Failed to approve prosumer.' });
  }
}

```

```
};
```

```
const handleDeactivate = async (prosumerNic) => {  
  try {  
    await usersApi.deactivateAccount(prosumerNic, user.nic);  
    setToast({  
      type: 'warning',  
      message: `Prosumer '${prosumerNic}' has been deactivated.`,  
    });  
    fetchProsumers();  
  } catch (err) {  
    setToast({ type: 'error', message: err.message || 'Failed to deactivate account.' });  
  }  
};
```

```
const handleReactivate = async (prosumerNic) => {  
  try {  
    await usersApi.reactivateAccount(prosumerNic, user.nic);  
    setToast({  
      type: 'success',  
      message: `Account '${prosumerNic}' successfully reactivated by Backoffice officer  
'${user.nic}'.`,  
    });  
    fetchProsumers();  
  } catch (err) {  
    setToast({ type: 'error', message: err.message || 'Reactivation failed.' });  
  }  
};
```

```
const handleRegisterSubmit = async (e) => {  
  e.preventDefault();  
  setRegistering(true);  
  try {  
    await authApi.registerProsumer(registerForm);  
  
    setToast({  
      type: 'success',  
      message: `Prosumer '${registerForm.nic}' registered successfully.`,  
    });  
    setIsRegisterOpen(false);  
  }  
};
```

```

setRegisterForm({
  nic: "",
  fullName: "",
  email: "",
  phoneNumber: "",
  password: 'Password@123',
  address: "",
  solarCapacityKW: 5.0,
});
fetchProsumers();
} catch (err) {
  setToast({ type: 'error', message: err.message || 'Registration failed.' });
} finally {
  setRegistering(false);
}
};

return (
  <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-8">
    {toast && <Toast type={toast.type} message={toast.message} onClose={() =>
setToast(null)} />}
    </div>

    { /* Page Header */ }
    <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-4 mb-6">
      <div>
        <h1 className="text-2xl font-black text-white tracking-tight flex items-center gap-2.5">
          <Shield className="w-6 h-6 text-amber-400" />
          Prosumer Account Management
        </h1>
        <p className="text-xs text-slate-400 mt-1">
          Backoffice control center for verifying identities (NIC), handling approvals, and
enforcing exclusive account reactivations.
        </p>
      </div>

      <button
        onClick={() => setIsRegisterOpen(true)}
        className="flex items-center space-x-2 px-4 py-2 rounded-xl text-xs font-bold text-slate-
950 solar-gradient hover:opacity-95 transition-opacity shadow-md self-start sm:self-auto"
      >

```

```

      <UserPlus className="w-4 h-4" />
      <span>Onboard New Prosumer</span>
    </button>
  </div>

  { /* Prosumer Table Component */}
  <ProsumerTable
    prosumers={prosumers}
    loading={loading}
    onApprove={handleApprove}
    onDeactivate={handleDeactivate}
    onReactivate={handleReactivate}
  />

  { /* Manual Prosumer Onboarding Modal */}
  <Modal
    isOpen={isRegisterOpen}
    onClose={() => setIsRegisterOpen(false)}
    title="Onboard Solar Prosumer"
    subtitle="Register prosumer using National Identity Card (NIC) as primary identity key."
  >
    <form onSubmit={handleRegisterSubmit} className="space-y-4 text-xs">
      <div className="grid grid-cols-2 gap-4">
        <div>
          <label className="block text-slate-300 font-semibold mb-1">
            NIC Number (Primary Key)
          </label>
          <input
            type="text"
            required
            placeholder="e.g. 200012345678"
            value={registerForm.nic}
            onChange={(e) => setRegisterForm({ ...registerForm, nic: e.target.value })}
            className="w-full bg-slate-950 border border-slate-700 rounded-xl px-3 py-2 text-white font-mono placeholder-slate-500 focus:outline-none focus:border-amber-500/70"
          />
        </div>
        <div>
          <label className="block text-slate-300 font-semibold mb-1">
            Full Name

```

```

    </label>
    <input
      type="text"
      required
      placeholder="e.g. Sunimal Perera"
      value={registerForm.fullName}
      onChange={(e) => setRegisterForm({ ...registerForm, fullName: e.target.value })}
      className="w-full bg-slate-950 border border-slate-700 rounded-xl px-3 py-2 text-
white placeholder-slate-500 focus:outline-none focus:border-amber-500/70"
    />
  </div>
</div>

<div className="grid grid-cols-2 gap-4">
  <div>
    <label className="block text-slate-300 font-semibold mb-1">
      Email Address
    </label>
    <input
      type="email"
      required
      placeholder="prosumer@domain.com"
      value={registerForm.email}
      onChange={(e) => setRegisterForm({ ...registerForm, email: e.target.value })}
      className="w-full bg-slate-950 border border-slate-700 rounded-xl px-3 py-2 text-
white placeholder-slate-500 focus:outline-none focus:border-amber-500/70"
    />
  </div>
  <div>
    <label className="block text-slate-300 font-semibold mb-1">
      Phone Number
    </label>
    <input
      type="text"
      required
      placeholder="+94 77 123 4567"
      value={registerForm.phoneNumber}
      onChange={(e) => setRegisterForm({ ...registerForm, phoneNumber: e.target.value
    }}}

```

```

        className="w-full bg-slate-950 border border-slate-700 rounded-xl px-3 py-2 text-
white placeholder-slate-500 focus:outline-none focus:border-amber-500/70"
    />
</div>
</div>

<div className="grid grid-cols-2 gap-4">
    <div>
        <label className="block text-slate-300 font-semibold mb-1">
            Residential / Array Address
        </label>
        <input
            type="text"
            placeholder="12 Solar Way, Colombo"
            value={registerForm.address}
            onChange={(e) => setRegisterForm({ ...registerForm, address: e.target.value })}
            className="w-full bg-slate-950 border border-slate-700 rounded-xl px-3 py-2 text-
white placeholder-slate-500 focus:outline-none focus:border-amber-500/70"
        />
    </div>
    <div>
        <label className="block text-slate-300 font-semibold mb-1 flex items-center gap-1">
            <Zap className="w-3.5 h-3.5 text-amber-400" />
            Solar Array Capacity (kW)
        </label>
        <input
            type="number"
            step="0.1"
            min="0.5"
            required
            value={registerForm.solarCapacityKW}
            onChange={(e) =>
                setRegisterForm({ ...registerForm, solarCapacityKW: parseFloat(e.target.value) || 0 })
            }
            className="w-full bg-slate-950 border border-slate-700 rounded-xl px-3 py-2 text-
white placeholder-slate-500 focus:outline-none focus:border-amber-500/70"
        />
    </div>
</div>

```

```

    <div className="flex items-center justify-end space-x-3 pt-4 border-t border-slate-800">
      <button
        type="button"
        onClick={() => setIsRegisterOpen(false)}
        className="px-4 py-2 rounded-xl text-slate-400 hover:text-white hover:bg-slate-800
transition-colors"
      >
        Cancel
      </button>
      <button
        type="submit"
        disabled={registering}
        className="px-5 py-2 rounded-xl font-bold text-slate-950 solar-gradient hover:opacity-
95 transition-opacity disabled:opacity-50 shadow-lg"
      >
        {registering ? 'Registering...' : 'Complete Registration'}
      </button>
    </div>
  </form>
</Modal>
</div>
);
};

```

```
export default ProsumerManagement;
```

OperatorDashboard.jsx

```

import React, { useEffect, useState } from 'react';
import { Link } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';
import { reservationsApi } from '../api/reservationsApi';
import { stationsApi } from '../api/stationsApi';
import StatCard from '../components/common/StatCard';
import Toast from '../components/common/Toast';
import {
  Zap,
  Radio,
  Cpu,
  Battery,

```

```

Clock,
ArrowUpRight,
RefreshCw,
QrCode,
CheckCircle2,
} from 'lucide-react';

const OperatorDashboard = () => {
  const { user } = useAuth();
  const [summary, setSummary] = useState(null);
  const [stations, setStations] = useState([]);
  const [recentReservations, setRecentReservations] = useState([]);
  const [loading, setLoading] = useState(true);
  const [toast, setToast] = useState(null);

  const fetchOperatorData = async () => {
    setLoading(true);
    try {
      const [sumData, stationsData, reservationsData] = await Promise.all([
        reservationsApi.getDashboardSummary(),
        stationsApi.getStations(true), // activeOnly = true
        reservationsApi.getReservations(),
      ]);

      setSummary(sumData);
      setStations(stationsData);
      setRecentReservations(reservationsData.slice(0, 5));
    } catch (err) {
      setToast({ type: 'error', message: err.message || 'Failed to load operator metrics.' });
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => {
    fetchOperatorData();
  }, []);

  return (
    <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-8">

```

```

    {toast && <Toast type={toast.type} message={toast.message} onClose={() =>
setToast(null)} />}

    {/* Operator Banner */}
    <div className="glass-panel rounded-3xl p-6 sm:p-8 border border-slate-800 shadow-xl
relative overflow-hidden mb-8">
      <div className="absolute top-0 right-0 w-80 h-80 bg-emerald-500/10 blur-3xl rounded-
full pointer-events-none" />

      <div className="flex flex-col md:flex-row md:items-center justify-between gap-4">
        <div>
          <div className="inline-flex items-center space-x-2 px-3 py-1 rounded-full text-xs font-
semibold bg-emerald-500/15 text-emerald-400 border border-emerald-500/30 mb-2">
            <Zap className="w-3.5 h-3.5" />
            <span>Grid Operator Console</span>
          </div>
          <h1 className="text-2xl sm:text-3xl font-black text-white tracking-tight">
            Operational Station Feed &bull; {user?.fullName || 'Operator'}
          </h1>
          <p className="text-xs text-slate-400 mt-1">
            Monitor active microgrid nodes, inspect scheduled battery storage slots, and dispatch
prosumer power trades.
          </p>
        </div>

        <div className="flex items-center space-x-3">
          <button
            onClick={fetchOperatorData}
            className="flex items-center space-x-1.5 px-3 py-2 rounded-xl text-xs font-semibold
bg-slate-900 text-slate-300 hover:text-white border border-slate-800 hover:border-slate-700
transition-colors"
          >
            <RefreshCw className={`w-3.5 h-3.5 ${loading ? 'animate-spin' : ''}`} />
            <span>Refresh</span>
          </button>

          <Link
            to="/operator/slots"

```

```
        className="flex items-center space-x-1.5 px-4 py-2 rounded-xl text-xs font-bold text-slate-950 bg-gradient-to-r from-emerald-400 to-green-500 hover:opacity-95 transition-opacity shadow-md"
```

```
    >
    <QrCode className="w-3.5 h-3.5" />
    <span>Slots & QR Monitor</span>
  </Link>
</div>
</div>
</div>
```

```
{/* Metrics Row */}
<div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4 gap-6 mb-8">
  <StatCard
    title="Online Microgrid Hubs"
    value={summary ? `${summary.activeStations} Active` : '--'}
    icon={Cpu}
    color="green"
    subtitle="Grid nodes accepting bookings"
    badge="Online"
  />
```

```
  <StatCard
    title="Approved Future Bookings"
    value={summary ? summary.approvedFutureReservationsCount : '--'}
    icon={Battery}
    color="blue"
    subtitle="Within 7-day future window"
    badge="Scheduled"
  />
```

```
  <StatCard
    title="Pending Bookings"
    value={summary ? summary.pendingReservationsCount : '--'}
    icon={Clock}
    color="amber"
    subtitle="Awaiting transfer window"
    badge="Upcoming"
  />
```

```

<StatCard
  title="Power Dispatched"
  value={summary ? `${summary.totalEnergyTradedKW.toFixed(1)} kW` : '--'}
  icon={Zap}
  color="purple"
  subtitle="Finalized via QR verification"
  badge="Completed"
/>
</div>

{/* Two Columns: Stations Readiness & Recent Bookings */}
<div className="grid grid-cols-1 lg:grid-cols-2 gap-8">
  {/* Active Stations Readiness */}
  <div className="glass-panel rounded-2xl p-6 border border-slate-800 shadow-xl">
    <div className="flex items-center justify-between pb-4 border-b border-slate-800">
      <div>
        <h2 className="text-base font-bold text-white flex items-center gap-2">
          <Cpu className="w-4 h-4 text-emerald-400" />
          Active Station Battery Bays
        </h2>
        <p className="text-xs text-slate-400 mt-0.5">
          Real-time storage slot availability across operating nodes.
        </p>
      </div>
      <div>
        <Link
          to="/operator/slots"
          className="text-xs text-emerald-400 hover:text-emerald-300 font-semibold flex items-center gap-1"
        >
          <span>View Slots</span>
          <ArrowUpRight className="w-3.5 h-3.5" />
        </Link>
      </div>
    </div>

    <div className="mt-4 space-y-3">
      {stations.length === 0 ? (
        <div className="text-center py-8 text-xs text-slate-500">
          No active stations found.
        </div>
      ) : (

```

```

stations.map((s) => (
  <div
    key={s.id}
    className="bg-slate-900/80 border border-slate-800 rounded-xl p-3.5 flex items-
center justify-between hover:border-slate-700 transition-colors"
  >
    <div>
      <div className="flex items-center space-x-2">
        <span className="text-xs font-bold text-white">{s.stationName}</span>
        <span className="text-[10px] font-mono px-1.5 py-0.5 rounded bg-slate-800 text-
slate-400">
          {s.stationCode}
        </span>
      </div>
      <div className="text-[11px] text-slate-400 mt-0.5">{s.locationName}</div>
    </div>

    <div className="text-right">
      <span className="text-xs font-bold text-emerald-400">
        {s.availableBatterySlots} / {s.totalBatterySlots} Available
      </span>
      <div className="text-[10px] text-slate-500 mt-0.5">
        Cap: {s.capacityKWh} kW/h
      </div>
    </div>
  </div>
))
)}
</div>
</div>

```

```

{ /* Recent Power Trading Bookings */ }
<div className="glass-panel rounded-2xl p-6 border border-slate-800 shadow-xl">
  <div className="flex items-center justify-between pb-4 border-b border-slate-800">
    <div>
      <h2 className="text-base font-bold text-white flex items-center gap-2">
        <Radio className="w-4 h-4 text-emerald-400" />
        Recent Reservation Activity
      </h2>
      <p className="text-xs text-slate-400 mt-0.5">

```

```

    Prosumer energy bookings awaiting or completing dispatch.
  </p>
</div>
<Link
  to="/operator/slots"
  className="text-xs text-emerald-400 hover:text-emerald-300 font-semibold flex items-
center gap-1"
  >
    <span>All Bookings</span>
    <ArrowUpRight className="w-3.5 h-3.5" />
  </Link>
</div>

<div className="mt-4 space-y-3">
  {recentReservations.length === 0 ? (
    <div className="text-center py-8 text-xs text-slate-500">
      No bookings recorded yet.
    </div>
  ) : (
    recentReservations.map((r) => (
      <div
        key={r.id}
        className="bg-slate-900/80 border border-slate-800 rounded-xl p-3.5 flex items-
center justify-between hover:border-slate-700 transition-colors text-xs"
      >
        <div>
          <div className="font-mono font-bold text-emerald-
400">{r.reservationNumber}</div>
          <div className="text-slate-300 text-[11px] mt-0.5">
            Prosumer: <span className="font-mono">{r.prosumerNIC}</span> &bull;
            {r.stationName}
          </div>
        </div>
        <div>
          <div className="text-right">
            <div className="font-bold text-amber-400">{r.energyAmountKW} kW</div>
            <span
              className={`inline-block text-[10px] font-semibold px-2 py-0.5 rounded-full mt-1
$ {
              r.status === 'Approved'

```

```

        ? 'bg-emerald-500/15 text-emerald-400'
        : r.status === 'Completed'
        ? 'bg-sky-500/15 text-sky-400'
        : 'bg-rose-500/15 text-rose-400'
      }}
    >
      {r.status}
    </span>
  </div>
</div>
))
})
</div>
</div>
</div>
</div>
);
};

```

export default OperatorDashboard;

SlotManagement.jsx

```

import React, { useEffect, useState } from 'react';
import { useAuth } from '../context/AuthContext';
import { stationsApi } from '../api/stationsApi';
import { reservationsApi } from '../api/reservationsApi';
import SlotScheduleList from '../components/operator/SlotScheduleList';
import ReservationMonitor from '../components/operator/ReservationMonitor';
import Toast from '../components/common/Toast';
import { Radio } from 'lucide-react';

const SlotManagement = () => {
  const { user } = useAuth();
  const [stations, setStations] = useState([]);
  const [selectedStationId, setSelectedStationId] = useState("");
  const [selectedDate, setSelectedDate] = useState(() => {
    return new Date().toISOString().split('T')[0];
  });
};

```

```

const [slots, setSlots] = useState([]);
const [reservations, setReservations] = useState([]);
const [loadingSlots, setLoadingSlots] = useState(false);
const [loadingReservations, setLoadingReservations] = useState(false);
const [toast, setToast] = useState(null);

// Fetch stations on load
useEffect(() => {
  stationsApi
    .getStations(true)
    .then((data) => {
      setStations(data);
      if (data.length > 0) {
        setSelectedStationId(data[0].id);
      }
    })
    .catch((err) => {
      setToast({ type: 'error', message: err.message || 'Failed to load active stations.' });
    });
}, []);

// Fetch slots whenever station or date changes
const fetchSlots = async (stationId, date) => {
  if (!stationId) return;
  setLoadingSlots(true);
  try {
    const data = await reservationsApi.getSlots(stationId, date);
    setSlots(data);
  } catch (err) {
    setToast({ type: 'error', message: err.message || 'Failed to load booking slots.' });
  } finally {
    setLoadingSlots(false);
  }
};

useEffect(() => {
  if (selectedStationId) {
    fetchSlots(selectedStationId, selectedDate);
  }
}, [selectedStationId, selectedDate]);

```

```

// Fetch live reservations
const fetchReservations = async () => {
  setLoadingReservations(true);
  try {
    const data = await reservationsApi.getReservations();
    setReservations(data);
  } catch (err) {
    setToast({ type: 'error', message: err.message || 'Failed to load reservations.' });
  } finally {
    setLoadingReservations(false);
  }
};

useEffect(() => {
  fetchReservations();
}, []);

// Batch generate slots handler
const handleGenerateDailySlots = async (stationId, targetDate) => {
  try {
    await reservationsApi.generateDailySlots({
      stationId,
      targetDate: new Date(targetDate).toISOString(),
      slotDurationMinutes: 60,
      slotCapacityKW: 25.0,
    });
    setToast({
      type: 'success',
      message: 'Daily slots batch generated successfully for operational hours.',
    });
    fetchSlots(stationId, targetDate);
  } catch (err) {
    setToast({ type: 'error', message: err.message || 'Failed to generate slots.' });
  }
};

// QR Verify and Complete handler
const handleVerifyAndComplete = async (payload) => {
  try {

```

```

const res = await reservationsApi.verifyAndComplete(payload);
setToast({
  type: 'success',
  message: `Energy transfer for reservation '${res.reservationNumber}' verified & completed
by Operator '${user?.nic}'.`,
});
fetchReservations();
if (selectedStationId) {
  fetchSlots(selectedStationId, selectedDate);
}
} catch (err) {
  setToast({ type: 'error', message: err.message || 'QR verification failed.' });
}
};

```

```

return (
  <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-8">
    {toast && <Toast type={toast.type} message={toast.message} onClose={() =>
setToast(null)} />}

```

```

{/* Page Title */}
<div className="mb-6">
  <h1 className="text-2xl font-black text-white tracking-tight flex items-center gap-2.5">
    <Radio className="w-6 h-6 text-emerald-400" />
    Microgrid Slot & Reservation Management
  </h1>
  <p className="text-xs text-slate-400 mt-1">
    Grid Operator console for viewing battery slot readiness, auditing prosumer trading
requests, and finalizing on-site QR transactions.
  </p>
</div>

```

```

{/* Slot Schedule List */}
<SlotScheduleList
  stations={stations}
  selectedStationId={selectedStationId}
  onSelectStation={(id) => setSelectedStationId(id)}
  slots={slots}
  loading={loadingSlots}
  selectedDate={selectedDate}

```

```

        onChange={ (d) => setSelectedDate(d)}
        onGenerateDailySlots={handleGenerateDailySlots}
      />

      { /* Live Reservation Monitor */}
      <ReservationMonitor
        reservations={reservations}
        loading={loadingReservations}
        onVerifyAndComplete={handleVerifyAndComplete}
        operatorNic={user?.nic || ""}
      />
    </div>
  );
};

export default SlotManagement;

```

ReservationMonitor.jsx

```

import React, { useState } from 'react';
import { Search, QrCode, CheckCircle, RefreshCw, Zap, ShieldCheck, Clock } from 'lucide-react';
import Modal from '../common/Modal';

const ReservationMonitor = ({
  reservations = [],
  loading = false,
  onVerifyAndComplete,
  operatorNic = "",
}) => {
  const [searchTerm, setSearchTerm] = useState("");
  const [statusFilter, setStatusFilter] = useState('ALL');

  // Verify and complete modal state
  const [isModalOpen, setIsModalOpen] = useState(false);
  const [qrTokenInput, setQrTokenInput] = useState("");
  const [deliveredKW, setDeliveredKW] = useState(10);
  const [operatorNotes, setOperatorNotes] = useState("");
  const [completing, setCompleting] = useState(false);

```

```

const filtered = reservations.filter((r) => {
  const matchesSearch =
    r.reservationNumber.toLowerCase().includes(searchTerm.toLowerCase()) ||
    r.prosumerNIC.toLowerCase().includes(searchTerm.toLowerCase()) ||
    r.stationName.toLowerCase().includes(searchTerm.toLowerCase());

  if (statusFilter === 'ALL') return matchesSearch;
  return matchesSearch && r.status === statusFilter;
});

const getStatusBadge = (status) => {
  switch (status) {
    case 'Approved':
      return (
        <span className="inline-flex items-center px-2.5 py-0.5 rounded-full text-xs font-
semibold bg-emerald-500/15 text-emerald-400 border border-emerald-500/30">
          Approved
        </span>
      );
    case 'Completed':
      return (
        <span className="inline-flex items-center px-2.5 py-0.5 rounded-full text-xs font-
semibold bg-sky-500/15 text-sky-400 border border-sky-500/30">
          Completed
        </span>
      );
    case 'Cancelled':
      return (
        <span className="inline-flex items-center px-2.5 py-0.5 rounded-full text-xs font-
semibold bg-rose-500/15 text-rose-400 border border-rose-500/30">
          Cancelled
        </span>
      );
    case 'Pending':
      return (
        <span className="inline-flex items-center px-2.5 py-0.5 rounded-full text-xs font-
semibold bg-amber-500/15 text-amber-400 border border-amber-500/30">
          Pending
        </span>
      );
  }

```

```

    default:
      return <span className="text-xs text-slate-400">{status}</span>;
    }
  };

const handleOpenCompleteModal = (res = null) => {
  if (res && res.qrCodeToken) {
    setQrTokenInput(res.qrCodeToken);
    setDeliveredKW(res.energyAmountKW || 10);
  } else {
    setQrTokenInput("");
    setDeliveredKW(10);
  }
  setOperatorNotes('Verified on-site by Grid Operator');
  setIsModalOpen(true);
};

const handleSubmitComplete = async (e) => {
  e.preventDefault();
  if (!qrTokenInput.trim()) return;

  setCompleting(true);
  try {
    await onVerifyAndComplete({
      qrToken: qrTokenInput.trim(),
      operatorId: operatorNic,
      energyDeliveredKWh: parseFloat(deliveredKW) || 0,
      notes: operatorNotes,
    });
    setIsModalOpen(false);
  } finally {
    setCompleting(false);
  }
};

const formatDateTime = (isoString) => {
  if (!isoString) return '--';
  const d = new Date(isoString);
  return `${d.toLocaleDateString()} ${d.toLocaleTimeString([], { hour: '2-digit', minute: '2-digit', hour12: true })}`;
};

```

```

};

return (
  <div className="glass-panel rounded-2xl p-6 border border-slate-800 shadow-xl mt-6">
    { /* Header and Controls */ }
    <div className="flex flex-col md:flex-row md:items-center justify-between gap-4 pb-6 border-b border-slate-800">
      <div>
        <h2 className="text-xl font-bold text-white tracking-tight flex items-center gap-2">
          <Zap className="w-5 h-5 text-emerald-400" />
          Power Trading Booking Monitor
        </h2>
        <p className="text-xs text-slate-400 mt-1">
          Real-time feed of solar reservations across microgrid hubs with QR validation dispatch.
        </p>
      </div>

      <div className="flex flex-wrap items-center gap-3">
        <div className="relative">
          <Search className="w-4 h-4 text-slate-400 absolute left-3 top-1/2 -translate-y-1/2" />
          <input
            type="text"
            placeholder="Search reservation or NIC..."
            value={searchTerm}
            onChange={(e) => setSearchTerm(e.target.value)}
            className="bg-slate-900 border border-slate-700 rounded-xl pl-9 pr-4 py-2 text-xs text-slate-100 placeholder-slate-500 focus:outline-none focus:border-emerald-500/60 w-56"
          />
        </div>

        <div className="flex items-center rounded-xl bg-slate-900 p-1 border border-slate-800 text-xs font-medium">
          {[ 'ALL', 'Approved', 'Completed', 'Cancelled' ].map((status) => (
            <button
              key={status}
              onClick={() => setStatusFilter(status)}
              className={`px-3 py-1.5 rounded-lg transition-all ${
                statusFilter === status
                  ? 'bg-emerald-500 text-slate-950 font-bold'
                  : 'text-slate-400 hover:text-white'
              }`}
            />
          ))}
        </div>
      </div>
    </div>
  )

```

```

    }}
  >
    {status}
  </button>
))}
</div>

{/* Quick QR Dispatch Button */}
<button
  onClick={() => handleOpenCompleteModal()}
  className="flex items-center space-x-1.5 px-3 py-2 rounded-xl text-xs font-bold bg-
gradient-to-r from-emerald-500 to-green-600 text-slate-950 hover:opacity-90 transition-opacity
shadow-md"
  >
    <QrCode className="w-4 h-4" />
    <span>Verify & Complete QR</span>
  </button>
</div>
</div>

{/* Table */}
<div className="overflow-x-auto mt-4">
  <table className="w-full text-left text-xs">
    <thead className="bg-slate-900/60 text-slate-400 uppercase font-semibold tracking-
wider border-b border-slate-800">
      <tr>
        <th className="py-3 px-4">Reservation #</th>
        <th className="py-3 px-4">Prosumer NIC</th>
        <th className="py-3 px-4">Hub / Station</th>
        <th className="py-3 px-4">Scheduled Slot</th>
        <th className="py-3 px-4">Trade Spec</th>
        <th className="py-3 px-4">Status</th>
        <th className="py-3 px-4 text-right">Operator Action</th>
      </tr>
    </thead>
    <tbody className="divide-y divide-slate-800/60 font-medium">
      {loading ? (
        <tr>
          <td colspan="7" className="text-center py-10 text-slate-400">
            <div className="flex items-center justify-center space-x-2">

```

```

        <RefreshCw className="w-4 h-4 animate-spin text-emerald-400" />
        <span>Querying bookings from central API...</span>
      </div>
    </td>
  </tr>
) : filtered.length === 0 ? (
  <tr>
    <td colspan="7" className="text-center py-10 text-slate-500">
      No reservations matching current filters.
    </td>
  </tr>
) : (
  filtered.map((res) => (
    <tr key={res.id} className="hover:bg-slate-900/40 transition-colors">
      <td className="py-3.5 px-4 font-mono font-bold text-emerald-400">
        {res.reservationNumber}
      </td>
      <td className="py-3.5 px-4 font-mono font-semibold text-slate-300">
        {res.prosumerNIC}
      </td>
      <td className="py-3.5 px-4 font-semibold text-white">
        {res.stationName}
      </td>
      <td className="py-3.5 px-4 text-slate-300">
        <div className="flex items-center space-x-1">
          <Clock className="w-3.5 h-3.5 text-slate-400" />
          <span>{formatDateTime(res.startTime)}</span>
        </div>
      </td>
      <td className="py-3.5 px-4">
        <span className="font-bold text-amber-400">{res.energyAmountKW}
kW</span>{' '}
        <span className="text-[11px] text-slate-400">({res.tradeType})</span>
      </td>
      <td className="py-3.5 px-4">{getStatusBadge(res.status)}</td>
      <td className="py-3.5 px-4 text-right">
        {res.status === 'Approved' ? (
          <button
            onClick={() => handleOpenCompleteModal(res)}

```

```

        className="inline-flex items-center space-x-1 px-2.5 py-1.5 rounded-lg text-xs
font-semibold bg-emerald-500/20 text-emerald-300 hover:bg-emerald-500/30 border border-
emerald-500/40 transition-colors"

```

```

        >
        <ShieldCheck className="w-3.5 h-3.5" />
        <span>Finalize</span>
    </button>
    ) : res.status === 'Completed' ? (
        <span className="text-[11px] text-slate-400">
            Done ({res.finalizedByOperatorNIC || 'Operator'})
        </span>
    ) : (
        <span className="text-[11px] text-slate-500">--</span>
    )}
</td>
</tr>
))
)}
</tbody>
</table>
</div>

{/* QR Verify and Complete Modal */}
<Modal
    isOpen={isModalOpen}
    onClose={() => setIsModalOpen(false)}
    title="Grid Operator - QR Code Transfer Verification"
    subtitle="Verify the prosumer cryptographic token and complete the battery transfer job."
    >
    <form onSubmit={handleSubmitComplete} className="space-y-4 text-xs">
        <div>
            <label className="block text-slate-300 font-semibold mb-1 flex items-center gap-1.5">
                <QrCode className="w-3.5 h-3.5 text-emerald-400" />
                Scanned / Raw QR Token
            </label>
            <textarea
                required
                rows={3}
                value={qrTokenInput}
                onChange={(e) => setQrTokenInput(e.target.value)}

```

```
placeholder="Paste or scan prosumer QR payload here (e.g. RES-
ID|NIC|STATION|SLOT|TIMESTAMP#SIGNATURE)..."
className="w-full bg-slate-950 border border-slate-700 rounded-xl p-3 text-white font-
mono text-xs placeholder-slate-500 focus:outline-none focus:border-emerald-500/70"
```

```
/>
</div>
```

```
<div className="grid grid-cols-2 gap-4">
  <div>
    <label className="block text-slate-300 font-semibold mb-1">
      Verified Delivered Energy (kW/h)
    </label>
    <input
      type="number"
      step="0.1"
      min="0.1"
      required
      value={deliveredKW}
      onChange={(e) => setDeliveredKW(e.target.value)}
      className="w-full bg-slate-950 border border-slate-700 rounded-xl px-3 py-2 text-
white placeholder-slate-500 focus:outline-none focus:border-emerald-500/70"
    />
  </div>
```

```
<div>
  <label className="block text-slate-300 font-semibold mb-1">
    Operator ID / NIC
  </label>
  <input
    type="text"
    disabled
    value={operatorNic}
    className="w-full bg-slate-950 border border-slate-700 rounded-xl px-3 py-2 text-
slate-400 font-mono opacity-60"
  />
</div>
</div>
```

```
<div>
  <label className="block text-slate-300 font-semibold mb-1">
```

```

      Operator Observations / Notes
    </label>
    <input
      type="text"
      value={operatorNotes}
      onChange={(e) => setOperatorNotes(e.target.value)}
      placeholder="Inspection completed, battery bay 4 engaged"
      className="w-full bg-slate-950 border border-slate-700 rounded-xl px-3 py-2 text-
white placeholder-slate-500 focus:outline-none focus:border-emerald-500/70"
    />
  </div>

  <div className="flex items-center justify-end space-x-3 pt-4 border-t border-slate-800">
    <button
      type="button"
      onClick={() => setIsModalOpen(false)}
      className="px-4 py-2 rounded-xl text-slate-400 hover:text-white hover:bg-slate-800
transition-colors"
    >
      Cancel
    </button>
    <button
      type="submit"
      disabled={completing}
      className="flex items-center space-x-1.5 px-5 py-2 rounded-xl font-bold text-slate-950
bg-gradient-to-r from-emerald-500 to-green-600 hover:opacity-95 transition-opacity
disabled:opacity-50 shadow-lg"
    >
      <CheckCircle className="w-4 h-4" />
      <span>{completing ? 'Verifying with API...' : 'Finalize Transfer'}</span>
    </button>
  </div>
</form>
</Modal>
</div>
);
};

export default ReservationMonitor;

```

8. References

- [1] Microsoft, "ASP.NET Core documentation," Microsoft Learn. [Online]. Available: <https://learn.microsoft.com/en-us/aspnet/core/>. [Accessed: Sep. 28, 2026].
- [2] Microsoft, "Host ASP.NET Core on Windows with IIS," Microsoft Learn. [Online]. Available: <https://learn.microsoft.com/en-us/aspnet/core/host-and-deploy/iis/>. [Accessed: Sep. 28, 2026].
- [3] MongoDB, Inc., "MongoDB C#.NET Driver documentation." [Online]. Available: <https://www.mongodb.com/docs/drivers/csharp/current/>. [Accessed: Sep. 28, 2026].
- [4] MongoDB, Inc., "MongoDB Manual." [Online]. Available: <https://www.mongodb.com/docs/manual/>. [Accessed: Sep. 28, 2026].
- [5] Google, "Android Developers documentation." [Online]. Available: <https://developer.android.com/>. [Accessed: Sep. 28, 2026].
- [6] Google, "Save data in a local database using Room." [Online]. Available: <https://developer.android.com/training/data-storage/room>. [Accessed: Sep. 28, 2026].
- [7] Google, "Set up Android Emulator networking" (10.0.2.2 host loopback). [Online]. Available: <https://developer.android.com/studio/run/emulator-networking>. [Accessed: Sep. 28, 2026].
- [8] JetBrains, "Kotlin documentation." [Online]. Available: <https://kotlinlang.org/docs/home.html>. [Accessed: Sep. 28, 2026].
- [9] Meta Open Source, "React documentation." [Online]. Available: <https://react.dev/>. [Accessed: Sep. 28, 2026].
- [10] Vite, "Vite guide." [Online]. Available: <https://vite.dev/guide/>. [Accessed: Sep. 28, 2026].
- [11] Tailwind Labs, "Tailwind CSS documentation." [Online]. Available: <https://tailwindcss.com/docs>. [Accessed: Sep. 28, 2026].

9. Individual Contribution

IT Number	Name	Contribution
IT22546920	Lakshan H.G.J.S.	Backoffice & Node/Admin: Backoffice Dashboard, Prosumer Management, Node Management and Staff Management screens (web); Stations and Users API (station CRUD, staff creation, approve, deactivate, reactivate); User and SolarStationInfo models.
IT22220424	Kaushalya P.L.P.D.	Auth, Operator & Mobile Infrastructure: Login and role-based routing (web); Auth API (login, JWT, registration); Login, Register and Profile screens, Operator screens, QR scanner, Google Maps, Retrofit and Room/SQLite (Android).
IT22070630	Perera K.D.N.	Reservation & Prosumer: Slot Management and Operator Dashboard (web); Reservations and Slots API with BookingService (7-day window and 12-hour notice rules); Create, Edit, Summary and History booking screens and QR display (Android); EnergyReservation and EnergyBookingSlot models.

10. Challenges

10.1. Verifying that Android Authentication Is Not Performed Locally

Early testing showed the Android app could log in with both Wi-Fi and mobile data disabled, which raised a concern that credentials might be validated against the local Room/SQLite database rather than the central Web API — a direct violation of the FAT-service constraint.

To investigate, the app's stored data was cleared and the login was re-tested with both Wi-Fi and mobile data disabled; login still succeeded. The Web API service on the host machine was then stopped entirely and the login was attempted again — at this point the app correctly returned an "Authentication Error".

Reviewing LoginActivity.kt, AuthRepository.kt and RetrofitClient.kt confirmed that AuthRepository.login() always calls the API's login endpoint and never validates credentials against local data. The earlier successful offline login was explained by the Android emulator's host-loopback address (10.0.2.2), which allows the emulator to reach the Web API running on the host machine independently of the device's own Wi-Fi/mobile-data radio state. This was a valuable lesson in not mistaking an emulator networking quirk for a genuine architectural flaw, and in designing a test (stopping the backend entirely) that isolates the actual variable being tested.

10.2. Enforcing the 12-Hour and 7-Day Rules Centrally

An early design question was where to enforce the 7-day scheduling window and 12-hour modification/cancellation notice — client-side for immediate user feedback, or server-side only. Client-side validation was rejected because it would place business logic outside the API, violating the FAT-service pattern the module's marking scheme explicitly rewards. The final design validates entirely in BookingService.cs on the server, with the client only reflecting the API's success or error response back to the user.

10.3. Coordinating Role-Based Access Across Two Independent Clients

Because Backoffice functionality is web-only while Prosumer functionality is mobile-only, and Grid Operator spans both, care was needed to keep role-routing logic consistent between the React and Kotlin codebases without duplicating business rules — this was solved by keeping all authorization decisions in the API's service layer and having each client's routing logic depend only on the role string returned at login, not on any locally cached permission list.

10.4. Google Maps API key configuration

Displaying nearby microgrid nodes on StationsMapActivity needed a Google Maps API key, and the first attempt showed a blank map with no markers even though the station data was loading

correctly from the API. Logcat showed an authorization error, which traced the problem to the Google Cloud Console configuration: the Maps SDK had to be explicitly enabled, and the key had to be declared in AndroidManifest.xml and restricted to the app's package name and signing certificate fingerprint. Since the debug and release keystores have different fingerprints, a key that worked on one member's machine initially failed on another's until both fingerprints were registered. To avoid committing the key to a public repository, it was moved to local.properties (git-ignored) and injected into the manifest at build time.

10.5. git workflow coordination across three members

Working across three members on one repository surfaced a few coordination issues. Early on, large, infrequent commits from a single member made it hard for the others to see what had changed or to merge their own work without conflicts, particularly in shared files like App.jsx and the navigation/routing setup. This was resolved by agreeing on smaller, feature-scoped commits, working in separate branches per member's domain, and merging through pull requests rather than pushing directly to main. A second issue was that two members editing the same Android module (data layer and UI) at the same time produced merge conflicts in Gradle and manifest files, which required a short discussion each time to agree on the correct resolution rather than accepting either side automatically.

11. Git Repository link

GitHub Link - <https://github.com/JordanCJ7/SolariX>